Fundamentals of Deep Learning

Lecture Notes

Muxammadrizo Maribjonov

November 5, 2026

Contents

1 From Logistic Regression to MLPs **4**

1.1 Recap of Logistic Regression 4

1.1.1 The problem: a decision is not a probability 4

1.1.2 Notation and the assumptions we are making 4

1.1.3 Inventing the sigmoid through odds and evidence 5

1.1.4 Geometry: why the boundary is a line 6

1.1.5 What does "maximally uncertain" actually mean? 7

1.1.6 Nonlinear boundaries without nonlinear parameters 7

1.1.7 From a probability model to the likelihood of an observation 8

1.1.8 Deriving the training objective from maximum likelihood 9

1.1.9 Why a logarithmic penalty makes sense 9

1.1.10 Why the best probability is honest, not always extreme 10

1.1.11 Deriving the gradient: why the residual is $p - y$ 11

1.1.12 Convexity: a proof, and what it does not promise 12

1.1.13 Computational complexity: what exactly is being computed? 13

1.1.14 Numerical stability: evaluate the mathematical quantity safely 14

1.1.15 Worked robotics setup: grasp success, cost, and evaluation 15

1.1.16 From-scratch practice and checks 16

1.1.17 Reasoning exercises 17

1.1.18 Worked answers and conceptual checkpoints 18

1.2 Cross Entropy, KL Divergence, and Maximum Likelihood 19

1.2.1 Start with a robot that must describe what it sees 19

1.2.2 Why should information be a negative logarithm? 19

1.2.3 Expectation turns surprise into entropy 20

1.2.4 Coding makes the average operational 21

1.2.5 Cross-entropy: keep the source, change the scoring model 22

1.2.6 KL divergence is the excess expected log loss 23

1.2.7 Why KL is directed and is not a distance metric 23

1.2.8 Conditioning: knowing the context changes uncertainty 24

1.2.9 Maximum likelihood: choosing a model after seeing data 25

1.2.10 The empirical distribution: turning a dataset into an expectation 25

1.2.11 Every step of the MLE–cross-entropy–KL equivalence 26

1.2.12 Worked MLEs: Bernoulli and categorical models 26

1.2.13 Continuous data: densities are not point probabilities 27

1.2.14 Interpreting the slide: approximating two modes by one Gaussian 28

1.2.15 Transfer question 1: why track KL rather than cross-entropy? 30

1.2.16 Transfer question 2: derive binary cross-entropy as a special case 30

1.2.17 Worked semantic-perception example: separate uncertainty from error 31

1.2.18 Implement the definitions without hiding their edge cases 32

1.2.19 Reasoning exercises 33

1.2.20 Worked answers and checks on interpretation . . . 34
1.3 Logistic Regression as a Neural Network: The Perceptron . . . 35
1.3.1 One model, two ways of seeing it . . . 35
1.3.2 A complete numerical forward pass . . . 36
1.3.3 Computational complexity: count the forward-pass work . . . 36
1.3.4 The same argument as a matrix multiplication . . . 36
1.3.5 A stable NumPy implementation . . . 37
1.3.6 Connection to learning and the next layer . . . 37
1.4 Multilayer Perceptrons . . . 38
1.4.1 Why add a hidden layer? . . . 38
1.4.2 Derive a layer from its individual neurons . . . 38
1.4.3 Compose two hidden layers and an output layer . . . 39
1.4.4 Why the hidden nonlinearity matters: a proof . . . 39
1.4.5 A constructive nonlinear representation: solve XOR by hand . . . 40
1.4.6 Time complexity for the lecture's two-hidden-layer question . . . 41
1.4.7 Memory complexity: count what must exist . . . 41
1.4.8 A practical sizing example . . . 42
1.4.9 Exercises and worked reasoning . . . 42
1.5 Matrix Dimensions . . . 43
1.5.1 A matrix is an organized collection of scalar equations . . . 43
1.5.2 Batching in the column-vector convention . . . 43
1.5.3 Transpose the whole equation to get row batches . . . 44
1.5.4 Work through every entry of a concrete batch . . . 45
1.5.5 The lecture's batched-perceptron repetition question . . . 45
1.5.6 General layer notation and a dimension-checking procedure . . . 46
1.5.7 General time complexity, then the equal-width answer . . . 46
1.5.8 General memory complexity: parameters, data, and lifetimes . . . 47
1.5.9 A numerical batch budget and what it teaches . . . 49
1.5.10 NumPy implementation: translate the equations literally . . . 49
1.5.11 Exercises with worked answers . . . 50
1.6 Other Activation Functions and Loss Functions . . . 50
1.6.1 Can neurons really add up to draw a curve? . . . 50
1.6.2 Build a piecewise-linear curve with exact weights . . . 51
1.6.3 What should we inspect in an activation function? . . . 52
1.6.4 Sigmoid and hyperbolic tangent . . . 53
1.6.5 Linear and rectified linear activations . . . 54
1.6.6 PReLU and ELU: what happens on the negative side? . . . 54
1.6.7 GELU and Swish: smooth input-dependent gates . . . 54
1.6.8 Output activations describe what predictions can mean . . . 55
1.6.9 Derive squared error from a Gaussian observation model . . . 55
1.6.10 Binary cross-entropy and why the output gradient simplifies . . . 56
1.6.11 Derive softmax from relative evidence and normalization . . . 56
1.6.12 Transfer question: why not a hard maximum? . . . 57
1.6.13 Transfer question: what if regression has a ReLU output? . . . 57
1.6.14 Real weights on Iris: one feature and a continuous target . . . 58
1.6.15 Real weights on Iris: hidden features and softmax species prediction . . . 59
1.6.16 Implementation and reconstruction tasks . . . 61
1.7 Representational Power of MLPs . . . 62
1.7.1 Separate three questions that are often mixed together . . . 62
1.7.2 Any Boolean function on a finite cube: an explicit construction . . . 62
1.7.3 Universal approximation: a precise statement . . . 62

1.7.4 A first-principles proof in one dimension using ReLU hinges 63
1.7.5 What universal approximation does not say 64
1.7.6 Depth: repeated composition can reuse a small computation 64
1.7.7 Interpreting the linear-region theorem in the slide 65
1.7.8 Capacity comparison and answers to the lecture questions 65
1.8 Summary, Further Reading, References 66
1.8.1 Reconstruct the chapter as one connected argument 66
1.8.2 Learning goals with precise qualifications 66
1.8.3 Practice without looking, then check the reasoning 67
1.8.4 Worked checks 67
1.8.5 Reproducible materials and further reading 67

2 MLPs and Backpropagation 69
2.1 Recap of Multilayer Perceptrons 69
2.2 Recap of Chain Rule of Calculus 69
2.3 Calculating Gradients with Backpropagation 69
2.4 Backprop for a Linear Layer in Matrix Form 69
2.5 Full Matrix Form of Backpropagation Equations 69
2.6 Summary, Further Reading, References 69

Chapter 1

From Logistic Regression to MLPs

1.1 Recap of Logistic Regression

The question behind every formula

A classifier must decide what to do, but first it can estimate what might be true. Logistic regression separates these two tasks: a model assigns a probability to a binary outcome, and a decision rule turns that probability into an action. The sigmoid specifies the probability model; cross-entropy measures how much probability that model assigned to the outcome actually observed.

1.1.1 The problem: a decision is not a probability

Imagine a robot about to grasp a cup. From its camera and estimated gripper pose, it measures a centering error and an angular misalignment. The outcome is $Y = 1$ for a successful grasp and $Y = 0$ for a failure. We would like to answer both "How likely is success?" and "Should the robot attempt this grasp?" These are different questions. A success probability of 0.7 might justify a cheap retryable grasp but not a grasp whose failure breaks an expensive object.

Why not just fit a line to the labels? A real-valued affine prediction can be negative or greater than one. Why not immediately threshold that line? A hard threshold throws away the distinction between a barely positive score and a strongly positive score. It also gives a piecewise constant classification error, which offers little useful gradient information. We therefore seek a smooth conditional probability model before choosing a decision rule.

This section develops the connected chain

$$\text{features} \longrightarrow \text{score} \longrightarrow \text{probability} \longrightarrow \text{likelihood} \longrightarrow \text{loss} \longrightarrow \text{learning and decisions.}$$

We first understand a fixed model. Only later do we vary its parameters to train it. The examples and plots here are constructed teaching examples, not empirical claims about a deployed robot.

1.1.2 Notation and the assumptions we are making

Let $x = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$ denote the measured features and let $Y \in \{0, 1\}$ be a random outcome. Define the augmented input and parameter vector

$$\tilde{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \in \mathbb{R}^{d+1}, \qquad w = \begin{bmatrix} b \\ a \end{bmatrix}, \qquad z = w^\top \tilde{x} = b + a^\top x.$$

The slides call the bias w_0 and use $x_0 = 1$; thus their $w^\top x$ is our $w^\top \tilde{x}$. We keep the tilde to distinguish the actual feature space from the augmented space. The scalar z is the *score* or *logit*.

Symbol	Meaning
Y	Random binary label before observing the outcome
y	Observed label, exactly zero or one
$q(x)$	True conditional probability $\Pr(Y = 1 \mid X = x)$
$p_w(x)$	Model probability, also written $h_w(x)$
$\hat{y}$	In the slides, the probability $h_w(x)$
$\hat{c}$	Hard class prediction, zero or one
$\ell(p, y)$	Loss on one observation
$J(w)$	Mean loss over the training data

Definitions, assumptions, consequences. That a binary conditional distribution is Bernoulli is a consequence of its two possible outcomes: it is completely specified by one probability. That its *log-odds is affine in the chosen features* will be our modeling assumption. That independent observations give a product likelihood is an additional sampling assumption. Cross-entropy then follows from maximum likelihood; its convexity follows from calculus. These statements play different logical roles.

Conditioning matters. We model $Y \mid X = x$, not one identical Bernoulli law for all inputs. At different inputs the success probabilities can differ. Treating Y probabilistically does not assert that the world is fundamentally random. Unmeasured friction, occlusion, noisy sensors, or missing features can make an outcome unpredictable from the information we have.

1.1.3 Inventing the sigmoid through odds and evidence

A useful probability must stay in $[0, 1]$, while a linear score can take any real value. Many maps could enforce this range. Why use this particular one?

For $0 < p < 1$, define the *odds* as $p/(1 - p)$. If $p = 0.8$, the odds are 4: success has four times the probability of failure. Odds live in $(0, \infty)$; logarithms turn this into the whole real line. The *log-odds* is

$$\operatorname{logit}(p) = \log \frac{p}{1 - p}.$$

We now *assume* that evidence measured in log-odds is additive:

$$\log \frac{p_w(x)}{1 - p_w(x)} = b + \sum_{j=1}^{d} a_j x_j = z. \tag{1.1}$$

Solving, rather than memorizing, gives

$$\frac{p}{1-p} = e^z \quad \Longrightarrow \quad p = e^z(1-p) \quad \Longrightarrow \quad p(1+e^z) = e^z \quad \Longrightarrow \quad \boxed{p_w(x) = \sigma(z) = \frac{1}{1+e^{-z}}.}$$

This is an equivalence: affine log-odds and a sigmoid of an affine score describe exactly the same model. Binary labels alone do not force this choice; a probit link, for example, would define a different model.

Equation (1.1) also explains the parameters. Increasing x_j by one unit with other features held fixed multiplies the odds by e^{a_j}. It does *not* add a_j to the probability. The bias is the log-odds at $x = 0$, so $p_w(0) = \sigma(b)$. This interpretation depends on units and on whether zero is meaningful for the features.

Why additive log-odds is a reasonable idea. Bayes' rule gives, wherever the densities are positive,

$$\log \frac{\Pr(Y=1 \mid x)}{\Pr(Y=0 \mid x)} = \log \frac{\Pr(Y=1)}{\Pr(Y=0)} + \log \frac{p(x \mid Y=1)}{p(x \mid Y=0)}.$$

Under conditional independence of features, the last term becomes a sum of featurewise log-likelihood ratios. This motivates adding evidence. Logistic regression itself does *not* require conditional feature independence; it models the final conditional log-odds directly. This is a conceptual motivation, not a claim about the historical invention of the method.

A nontrivial case where the affine assumption is exact. Suppose $X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma)$, with the same positive-definite covariance for both classes and priors $\pi_k > 0$. The normalizing constants cancel in the density ratio, leaving

$$\begin{aligned}\log \frac{q(x)}{1-q(x)} &= \log \frac{\pi_1}{\pi_0} - \frac{1}{2}(x-\mu_1)^\top \Sigma^{-1}(x-\mu_1) + \frac{1}{2}(x-\mu_0)^\top \Sigma^{-1}(x-\mu_0) \\ &= (\mu_1-\mu_0)^\top \Sigma^{-1} x - \frac{1}{2}(\mu_1^\top \Sigma^{-1}\mu_1 - \mu_0^\top \Sigma^{-1}\mu_0) + \log \frac{\pi_1}{\pi_0}.\end{aligned}$$

The quadratic terms in x cancel. Thus a Gaussian generative assumption can produce precisely a logistic conditional model. Unequal covariances generally leave quadratic terms. Logistic regression need not fit either Gaussian density to learn an affine log-odds model.

1.1.4 Geometry: why the boundary is a line

For equal costs of the two mistakes, predict class one when $p_w(x) \geq 1/2$. To understand its geometry, differentiate:

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)(1-\sigma(z)) > 0 \quad \text{for finite } z.$$

The sigmoid is strictly increasing and $\sigma(0) = 1/2$. Therefore

$$\hat{c} = \mathbf{1}\{p_w(x) \geq 1/2\} = \mathbf{1}\{b + a^\top x \geq 0\}. \tag{1.2}$$

Its decision boundary is $b + a^\top x = 0$. In two dimensions this is a line; in three, a plane; in d dimensions, a hyperplane, provided $a \neq 0$. Strictly, $b + a^\top x$ is *affine* in x and linear in $\tilde{x}$. The informal phrase "linear classifier" includes an affine bias.

For the slide's $w = (-3, 1, 1)^\top$, we obtain

$$z = -3 + x_1 + x_2, \qquad x_2 = 3 - x_1.$$

At $(1,1)$ the score is -1 and $p \approx 0.269$; at $(1,2)$ it is 0 and $p = 0.5$; at $(2,2)$ it is 1 and $p \approx 0.731$; at $(4,4)$ it is 5 and $p \approx 0.9933$. We use class one for ties by convention. Another tie convention changes only points exactly on the boundary.

Direction, distance, and confidence. Moving by a displacement v changes the score by $a^\top v$. If $a^\top v = 0$, we remain on a constant-score hyperplane. Thus a is normal to the boundary. For $a \neq 0$ the signed perpendicular distance from x to the boundary is

$$\delta(x) = \frac{b + a^\top x}{\|a\|_2}, \qquad p_w(x) = \sigma(\|a\|_2 \delta(x)).$$

The score is a *scaled* distance, not a distance in physical units. Scaling all weights by $c > 0$ preserves the boundary but changes the probabilities to $\sigma(cz)$. As c grows, predictions away from the boundary approach zero or one. The same classification accuracy can hide very different probability quality. If $c < 0$, the boundary stays but class assignments reverse.

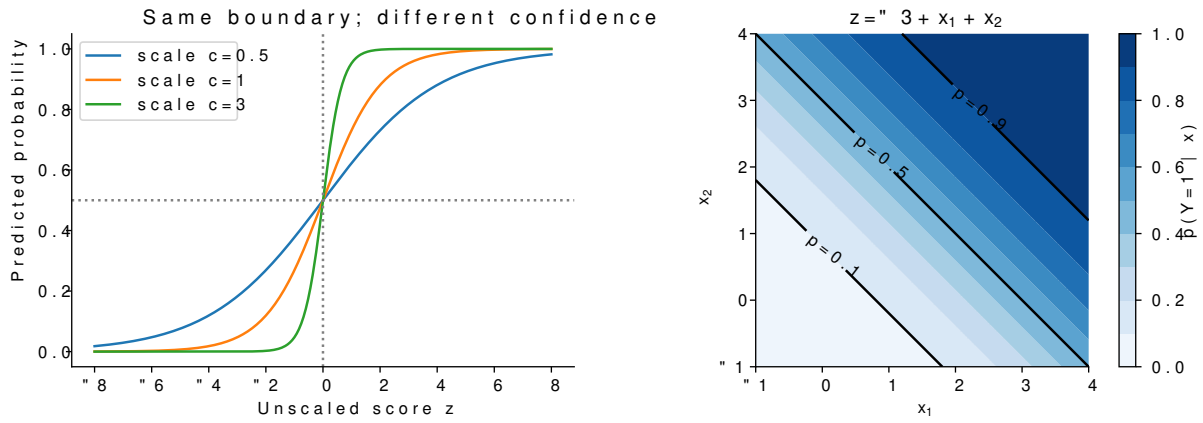


Figure 1.1: Left: rescaling a logit changes confidence without moving the zero threshold. Right: parallel probability contours around the boundary $x_1 + x_2 = 3$. The sigmoid bends the probability response, not this boundary.

Edge cases. If $a = 0$ and $b \neq 0$, the prediction is constant and there is no boundary. If $a = 0$ and $b = 0$, every point has probability $1/2$. The normalized-distance formula cannot be used in either case.

1.1.5 What does "maximally uncertain" actually mean?

For a fixed model w, $p_w(x)$ is a deterministic number at a fixed input. It specifies a distribution over Y, not a random choice of weights. The model's Bernoulli variance is

$$\mathrm{Var}(Y \mid x; w) = p(1-p) = \frac{1}{4} - (p - \tfrac{1}{2})^2,$$

which is largest at $p = 1/2$. Its predictive entropy, in nats, is

$$H(p) = -p \log p - (1-p) \log(1-p).$$

Differentiation gives

$$H'(p) = \log \frac{1-p}{p}, \qquad H''(p) = -\frac{1}{p} - \frac{1}{1-p} < 0.$$

So entropy has a unique maximum $H(1/2) = \log 2$. This is the precise sense in which the model is maximally uncertain at its 0.5 boundary.

Caution

A fitted logistic probability is not a guarantee of correctness. A probability near one may be unjustified on inputs unlike the training data. Predictive Bernoulli entropy does not separately measure uncertainty about the fitted parameters, missing model structure, or distribution shift. A decision boundary at a cost-dependent threshold other than 0.5 is not a maximum-entropy contour.

1.1.6 Nonlinear boundaries without nonlinear parameters

Replace raw features with a fixed map $\phi(x)$ and set $p_w(x) = \sigma(b + a^\top \phi(x))$. The boundary is affine in feature space, but can be curved in the original coordinates. For example,

$$\phi(x) = \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}, \qquad z = 4(x_1^2 + x_2^2 - 1).$$

The 0.5 boundary is the circle $x_1^2 + x_2^2 = 1$; outside it the model predicts class one. In (ϕ_1, ϕ_2) coordinates it is the line $\phi_1 + \phi_2 = 1$. This is logistic regression with nonlinear basis functions. The score remains linear in its trainable coefficients.

A second example is XOR on $x_1, x_2 \in \{-1, 1\}$, with class one for opposite signs. A feature $x_1 x_2$ allows the score $z = -kx_1x_2$, $k > 0$, which separates all four points. An affine score in x_1, x_2 alone cannot: the midpoint of the two class-one points and the midpoint of the two class-zero points are both the origin, contradicting strict separation by an affine function.

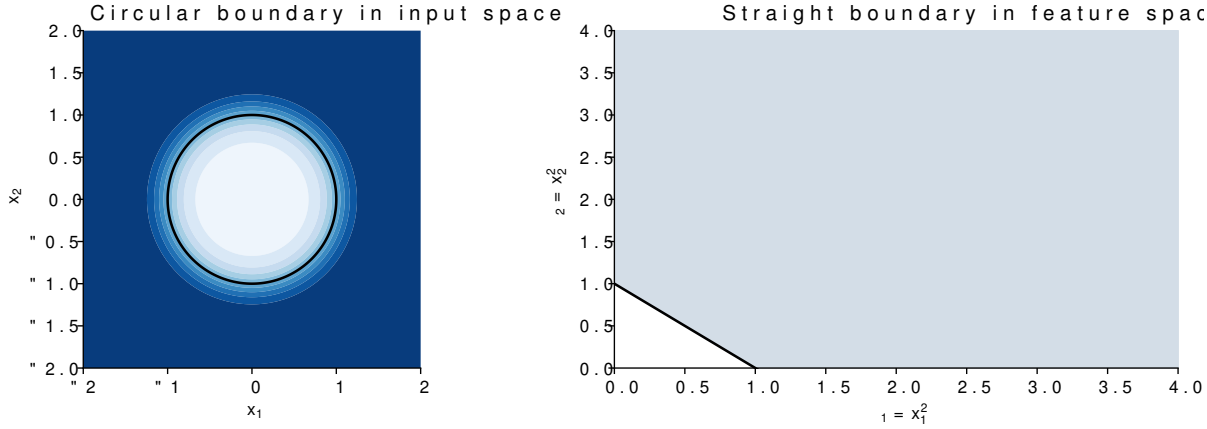


Figure 1.2: One model viewed in two coordinate systems. The curved boundary in input space becomes straight after the fixed feature transformation. Only the nonnegative quadrant is reachable in this feature space.

1.1.7 From a probability model to the likelihood of an observation

Once an input x is fixed, a Bernoulli model assigns

$$p_w(Y = y \mid x) = \begin{cases} p_w(x), & y = 1, \\ 1 - p_w(x), & y = 0. \end{cases}$$

Because y is binary, the two cases can be encoded as

$$p_w(Y = y \mid x) = p_w(x)^y \big(1 - p_w(x)\big)^{1-y}. \tag{1.3}$$

Check both values: when $y = 1$, the second exponent is zero; when $y = 0$, the first is zero. The formula does not make the label continuous. It uses the label as a selector of the probability assigned to the observed event.

Probability versus likelihood. With w fixed, this expression is a probability distribution over y; its two values sum to one. After observing (x, y), the very same expression viewed as a function of w is a *likelihood*. It asks which parameter choices would make the observed outcome plausible. It is not a probability distribution over w and need not integrate to one over parameter space. A Bayesian posterior over w would additionally require a prior and normalization.

For example, if a grasp succeeds, a model predicting $p = 0.9$ assigns likelihood 0.9, while one predicting $p = 0.2$ assigns 0.2. If that grasp fails, the likelihoods reverse their preference: 0.1 versus 0.8. A model is judged by what it said about the event that happened, not just by the thresholded class.

1.1.8 Deriving the training objective from maximum likelihood

Let $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ and $p_n = p_w(x_n)$. Assume the labels are conditionally independent across observations given their inputs and the common parameter w. An i.i.d. sample of pairs is a familiar sufficient setup. Then

$$\mathcal{L}(w; \mathcal{D}) = \prod_{n=1}^{N} p_n^{y_n} (1 - p_n)^{1-y_n}.$$

The product is a joint conditional probability of the observed labels. Adjacent video frames or repeated trials on the same object can violate the independence assumption. The sum loss can still be used as an objective in that case, but the simple independent-data likelihood interpretation and effective sample size need care.

We maximize this likelihood. Since log is strictly increasing, it preserves which positive value is largest. Since $\log(ab) = \log a + \log b$, it converts products into sums:

$$\begin{aligned}
\arg\max_w \mathcal{L}(w; \mathcal{D}) &= \arg\max_w \log \mathcal{L}(w; \mathcal{D}) \\
&= \arg\max_w \sum_{n=1}^{N} [y_n \log p_n + (1 - y_n) \log(1 - p_n)] \\
&= \arg\min_w \left\{ -\frac{1}{N} \sum_{n=1}^{N} [y_n \log p_n + (1 - y_n) \log(1 - p_n)] \right\}.
\end{aligned}$$

Negation turns maximization into minimization. Dividing by positive N does not change the minimizers; it expresses an average per observation. It does change gradient magnitudes, so the same numerical learning rate need not behave the same for sum and mean losses.

Definition 1.1 (Binary cross-entropy)**.** For one observation,

$$\boxed{\ell(p, y) = -y \log p - (1 - y) \log(1 - p).} \tag{1.4}$$

For a dataset, the mean negative log-likelihood is

$$\boxed{J(w) = \frac{1}{N} \sum_{n=1}^{N} \ell(p_w(x_n), y_n).} \tag{1.5}$$

All logarithms in this section are natural, so losses are measured in nats.

The brackets in (1.5) matter: the minus sign applies to both label-dependent log terms. Changing the logarithm base only multiplies the objective by a positive constant, leaving its minimizers unchanged.

1.1.9 Why a logarithmic penalty makes sense

The quantity $-\log r$ measures the *surprise* of an event to which the model assigned probability r. An event assigned probability one produces no surprise; an event assigned a tiny probability produces a large surprise. For independent events, surprise adds because their probabilities multiply. This is why the negative logarithm is so natural for a joint probability objective.

Probability of observed outcome r	0.9	0.5	0.01
Surprise $-\log r$	0.1054	0.6931	4.6052

For $y = 1$, $\ell = -\log p$ decreases as p increases, and $\partial \ell / \partial p = -1/p$. For $y = 0$, $\ell = -\log(1 - p)$ increases with p, and $\partial \ell / \partial p = 1/(1 - p)$. A confidently wrong prediction is much more expensive than an uncertain one. Correct classification alone does not make the loss zero: $p = 0.51$ and $p = 0.99$ both predict one, but their losses for $y = 1$ are about 0.673 and 0.010.

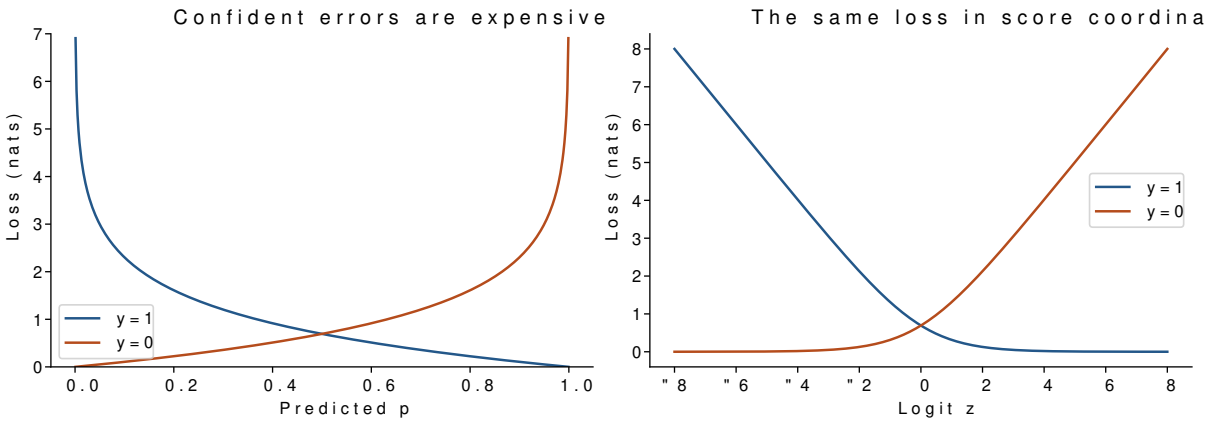


Figure 1.3: Cross-entropy in probability coordinates (left) and logit coordinates (right). The probability loss diverges at a wrong endpoint. In logit coordinates the wrong-direction loss grows approximately linearly for large magnitude.

The lecture's two-prediction question. For labels $(0, 1)$, prediction $(1, 1)$ assigns zero joint probability to the observed data. Its mean BCE is infinite, because the first observation has $-\log(1-1) = +\infty$. Prediction $(0.5, 0.5)$ assigns joint probability 0.25, and its mean loss is $-\frac{1}{2}\log(0.25) = \log 2 \approx 0.6931$. Cross-entropy therefore prefers the second prediction. Exact endpoints are not produced by a finite mathematical logit, but can occur through rounding. With $(1-\varepsilon, 1-\varepsilon)$ instead, the mean loss is $-\frac{1}{2}\log[\varepsilon(1-\varepsilon)]$, which diverges as $\varepsilon \downarrow 0$. At an endpoint, interpret zero-weight log terms by limits; do not compute $0 \log 0$ directly in floating-point code.

1.1.10 Why the best probability is honest, not always extreme

For a single observed positive label the loss prefers $p \to 1$. Does this mean training should always produce extreme probabilities? Consider repeated outcomes at the same input with true success probability q. The *expected* loss of reporting p is

$$r(p; q) = -q \log p - (1-q) \log(1-p).$$

For $0 < q < 1$,

$$\frac{\partial r}{\partial p} = -\frac{q}{p} + \frac{1-q}{1-p} = \frac{p-q}{p(1-p)}.$$

It is negative for $p < q$, zero for $p = q$, and positive for $p > q$. Hence the unique minimum is the true conditional probability $p = q$. For $q = 0$ or 1, the minimum is at the corresponding endpoint, approached as a limit by finite logits. This property is called a *strictly proper scoring rule*.

Suppose ten otherwise identical conditions yield seven successes and three failures. Their mean loss is $-0.7 \log p - 0.3 \log(1-p)$, minimized at $p = 0.7$, with logit $\log(7/3) \approx 0.8473$. Memorizing the latest outcome is not the same as modeling the underlying probability.

The irreducible minimum is $r(q; q) = H(q)$, and the excess expected loss is

$$r(p; q) - H(q) = q \log \frac{q}{p} + (1-q) \log \frac{1-q}{1-p} = D_{\mathrm{KL}}(\mathrm{Bern}(q) \,\|\, \mathrm{Bern}(p)) \geq 0.$$

The nonnegativity also follows from the minimization just proved. This gives a bridge to Section 1.2: cross-entropy is entropy plus the cost of using the wrong probabilities. A one-hot observed label is an empirical point mass, whose cross-entropy with the model is precisely the single-observation negative log-likelihood.

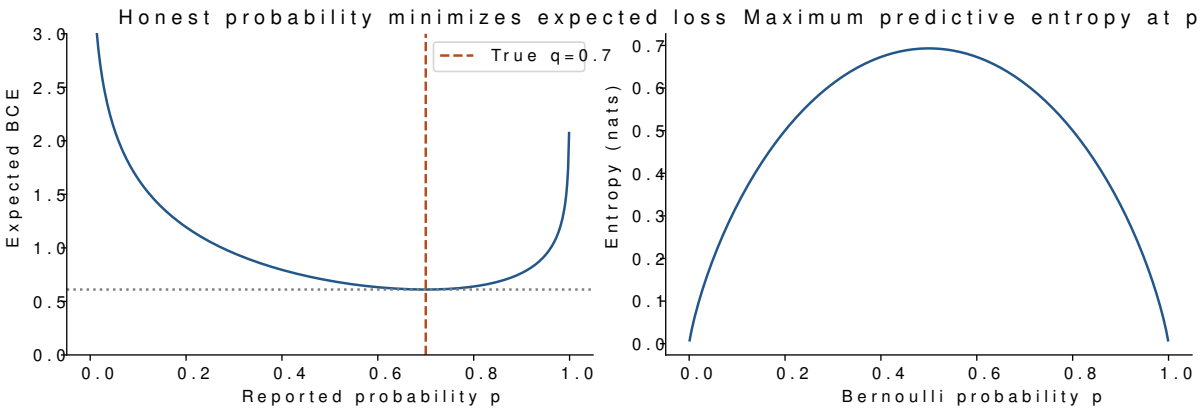


Figure 1.4: Left: when the true success frequency is 0.7, reporting 0.7 minimizes expected loss. The dotted horizontal line is the irreducible entropy. Right: entropy peaks at 0.5, quantifying uncertainty in a Bernoulli prediction.

Empirical versus population risk. Training minimizes a sample average. The quantity we ultimately care about is

$$R(w) = \mathbb{E}_{X,Y}[\ell(p_w(X), Y)] = \mathbb{E}_X[-q(X)\log p_w(X) - (1-q(X))\log(1-p_w(X))]\,.$$

This follows by first averaging over the two values of Y at each fixed X, then averaging over inputs. The weights are fixed while this expectation is evaluated; they vary when comparing models during optimization. Finite data, misspecified affine log-odds, and distribution shift can keep p_w far from q. A proper loss encourages honest probabilities in the population; it does not promise that every fitted finite-sample model is calibrated.

1.1.11 Deriving the gradient: why the residual is $p - y$

We want to adjust each weight in the direction that reduces the mean loss. For one observation, apply the chain rule through $w \to z \to p \to \ell$:

$$\begin{aligned}
\frac{\partial \ell}{\partial p} &= -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}, \\
\frac{\partial p}{\partial z} &= p(1-p), \\
\frac{\partial \ell}{\partial z} &= p - y, \\
\frac{\partial z}{\partial w_j} &= \tilde{x}_j, \qquad \boxed{\frac{\partial \ell}{\partial w_j} = (p-y)\tilde{x}_j.}
\end{aligned}$$

The apparent complicated fractions cancel because of the sigmoid–Bernoulli pairing. In particular, even when a confidently wrong sigmoid saturates, the BCE gradient with respect to its logit does not vanish: it approaches -1 for a very wrong positive label and $+1$ for a very wrong negative label.

For $y = 1$ and $p = 0.2$, $p - y = -0.8$. Gradient descent increases the score in the direction of this input, making the observed success more probable. More precisely, for a single-sample update $w^+ = w - \eta(p-y)\tilde{x}$,

$$z^+ = (w^+)^\top \tilde{x} = z + \eta(y-p)\|\tilde{x}\|^2.$$

For a positive label this increases its score; for a negative label it decreases it. Other examples may pull in conflicting directions, which is why learning requires a compromise rather than independent adjustments to every prediction.

Matrix form and dimensions. Let $X \in \mathbb{R}^{N\times(d+1)}$ have row n equal to $\tilde{x}_n^\top$, and let $y, p, z \in \mathbb{R}^N$. Then

$$z = Xw, \qquad p = \sigma(z) \quad \text{elementwise}, \qquad \boxed{\nabla J(w) = \frac{1}{N} X^\top (p - y).}$$

The gradient has shape $(d+1)$: $(d+1) \times N$ times $N \times 1$. The bias gradient is $N^{-1}\sum_n (p_n - y_n)$ because column zero of X is one. A full-batch update is

$$w_{t+1} = w_t - \eta \frac{1}{N} X^\top (\sigma(Xw_t) - y).$$

All predictions in this update are evaluated at the *current* weights w_t. Predictions change after the update. For a uniformly sampled minibatch of size B, replace the full average by its batch average. This is an unbiased estimator of the empirical gradient under uniform sampling. SGD is one possible solver, not part of the definition of logistic regression.

One complete update by hand. Take two observations $x^{(1)} = (0, 0), y_1 = 0$ and $x^{(2)} = (1, 2), y_2 = 1$, and initialize all three weights to zero. Both probabilities are 0.5, so

$$X = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 2 \end{bmatrix}, \quad p - y = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}, \quad \nabla J = \begin{bmatrix} 0 \\ -0.25 \\ -0.5 \end{bmatrix}.$$

With $\eta = 1$, $w^+ = (0, 0.25, 0.5)^\top$. The scores become $(0, 1.25)$ and the probabilities $(0.5, 0.7773)$. The mean loss decreases from 0.6931 to $\frac{1}{2}[0.6931 + \log(1 + e^{-1.25})] \approx 0.4725$. This separable toy dataset has no finite unregularized optimum; it illustrates an update, not convergence to finite weights.

1.1.12 Convexity: a proof, and what it does not promise

Differentiate the gradient again. Since $\partial p_n / \partial w = p_n(1 - p_n)\tilde{x}_n$,

$$\nabla^2 J(w) = \frac{1}{N} X^\top D X, \qquad D = \operatorname{diag}\big(p_n(1 - p_n)\big). \tag{1.6}$$

For any $v \in \mathbb{R}^{d+1}$,

$$v^\top \nabla^2 J(w) v = \frac{1}{N} \sum_{n=1}^{N} p_n(1 - p_n)(\tilde{x}_n^\top v)^2 \geq 0.$$

Thus the Hessian is positive semidefinite and J is convex in w. Every finite local minimum is global. This proof still works for any *fixed* nonlinear feature map, since its design matrix remains fixed. It does not establish convexity for a multilayer network whose feature representation is trained too.

For finite logits, all diagonal entries of D are positive. If X has full column rank, the Hessian is positive definite and J is strictly convex at every finite w, giving at most one finite minimizer. If columns are redundant, there may be whole flat directions. Strict convexity does not by itself guarantee that a minimum is attained.

The separability trap. Write $t_n = 2y_n - 1 \in \{-1, 1\}$. If some v strictly separates all training examples, $t_n \tilde{x}_n^\top v > 0$ for every n, then

$$J(cv) = \frac{1}{N} \sum_n \log(1 + e^{-c t_n \tilde{x}_n^\top v}) \longrightarrow 0 \quad (c \to \infty).$$

For any finite weights each term is strictly positive. The loss has infimum zero but no finite minimizer. Classification can become perfect while weight norms keep increasing. Quasi-separation creates related nonexistence issues. Therefore "convex" is not equivalent to "a unique finite solution exists."

A common remedy is an L^2 penalty on slopes,

$$J_\lambda(w) = J(w) + \frac{\lambda}{2}\|a\|_2^2, \qquad \nabla J_\lambda = \nabla J + \lambda(0, a^\top)^\top.$$

It discourages arbitrarily steep confidence transitions. With both classes present and fixed finite features, penalizing all slopes with $\lambda > 0$ also controls the unpenalized intercept and gives a finite unique optimum. If every label is one class, an unpenalized bias can still diverge. Penalizing all coordinates, including the bias, makes the objective coercive and strongly convex for $\lambda > 0$, guaranteeing a unique finite minimizer. The penalty changes the estimation objective: it is no longer unregularized maximum likelihood. Its strength must be interpreted relative to sum versus mean loss.

Choosing a learning rate from curvature. Because $p_n(1 - p_n) \le 1/4$,

$$0 \preceq \nabla^2 J(w) \preceq \frac{1}{4N} X^\top X, \qquad L = \frac{\|X\|_2^2}{4N}$$

is a global gradient-Lipschitz bound (spectral norm). Smoothness gives

$$J(w - \eta g) \le J(w) - \eta(1 - L\eta/2)\|g\|^2, \qquad g = \nabla J(w).$$

Thus $0 < \eta < 2/L$ yields descent when $g \neq 0$; $\eta \le 1/L$ is a convenient conservative choice. Add λ to the bound for the stated L^2 penalty. This full-batch bound does not mean every stochastic step decreases the loss. Standardizing feature scales often improves conditioning and makes the optimization more efficient.

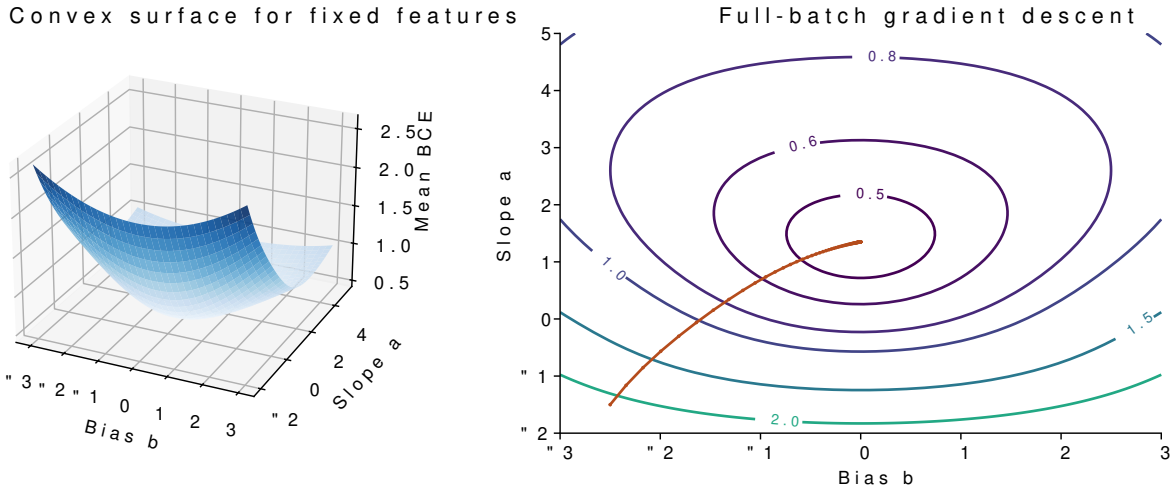


Figure 1.5: Mean BCE for a nonseparable one-feature dataset, with bias and slope as the two axes. The contour plot shows gradient descent from $(-2.5, -1.5)$. The surface and the contours describe the same objective in parameter space, not the decision boundary in input space.

1.1.13 Computational complexity: what exactly is being computed?

For dense data with N observations and d raw features:

Operation	Output	Time
Augmented matrix–vector product Xw	N logits	$\Theta(N(d+1))$
Elementwise sigmoid	N probabilities	$\Theta(N)$
Loss terms and reduction, given logits	One scalar	$\Theta(N)$
Gradient product $X^\top(p-y)$	$d+1$ derivatives	$\Theta(N(d+1))$

So computing BCE *from raw inputs and weights* is $\Theta(Nd)$ for $d \geq 1$. If probabilities have already been computed and the question only counts the loss reduction, it is $\Theta(N)$. A full-batch gradient step is also $\Theta(Nd)$; T steps cost $\Theta(TNd)$. We treat scalar arithmetic and transcendental functions as constant-cost operations in this usual model.

Storing dense X costs $\Theta(Nd)$ memory, and vectorized predictions need $\Theta(N)$ additional memory. Minibatches can reduce the working prediction storage to $\Theta(B)$; streaming avoids storing the full dataset. Sparse matrix products can cost $\Theta(\mathrm{nnz}(X)+N)$ instead. Explicitly forming a dense Hessian costs $O(Nd^2)$ time and $O(d^2)$ extra space, which explains why first-order methods are attractive when d is large.

1.1.14 Numerical stability: evaluate the mathematical quantity safely

The direct expression $-y\log\sigma(z) - (1-y)\log(1-\sigma(z))$ is fragile in floating point. A large positive score can round $\sigma(z)$ to one, and then $\log(1-p)$ becomes $\log 0$. Large negative scores can overflow e^{-z}. These are numerical failures, not infinite mathematical losses at finite z.

Use identities before computing. Since

$$\log\sigma(z) = z - \log(1+e^z), \qquad \log(1-\sigma(z)) = -\log(1+e^z),$$

substitution into BCE yields

$$\ell(z,y) = \log(1+e^z) - yz = \mathrm{softplus}(z) - yz. \tag{1.7}$$

A stable softplus is $\max(z,0) + \log(1+e^{-|z|})$. For binary labels an even cleaner form is

$$\boxed{\ell(z,y) = \log(1+e^{(1-2y)z}) = \mathrm{softplus}(-tz), \quad t = 2y-1.}$$

The scalar tz is the signed margin: a large positive margin means confidently correct, a negative margin means wrong. NumPy's `logaddexp(0, u)` computes $\log(1+e^u)$ stably. The signed-margin form also avoids subtracting two large nearly equal numbers for a correctly classified positive example.

```python
import numpy as np

def sigmoid(z):
    z = np.asarray(z, dtype=float)
    return np.exp(-np.logaddexp(0.0, -z))

def loss_and_gradient(X, y, w):
    # X: (N, d+1), y: (N,), w: (d+1,)
    z = X @ w
    loss = np.logaddexp(0.0, (1.0 - 2.0*y)*z).mean()
    gradient = X.T @ (sigmoid(z) - y) / len(y)
    return loss, gradient
```

At $z = -1000, y = 1$, the loss is approximately 1000, not a NaN. Clipping probabilities to $[\varepsilon, 1-\varepsilon]$ can prevent a log failure, but changes the objective near endpoints and can make a stated gradient inconsistent with that clipped objective. Computing from logits is preferable when available. Keep y one-dimensional with the code above: mixing shape $(N,1)$ labels and $(N,)$ probabilities can accidentally broadcast to an (N,N) array.

1.1.15 Worked robotics setup: grasp success, cost, and evaluation

We now connect the equations to an entire modeling workflow. Suppose a robot uses two nonnegative measurements: centering error e in millimeters and angular error θ in degrees. For an interpretable synthetic example, assume the true conditional success probability is

$$q(e, \theta) = \sigma(3 - 0.25e - 0.08\theta). \tag{1.8}$$

These coefficients are illustrative, not measured hardware parameters. Real success also depends on object geometry, friction, contact, and perception. The model says that one extra millimeter multiplies success odds by $e^{-0.25} \approx 0.779$, while one extra degree multiplies them by $e^{-0.08} \approx 0.923$, holding the other feature fixed.

At $e = 4, \theta = 10$, the logit is 1.2 and $q \approx 0.7685$. At $e = 8, \theta = 20$, it is -0.6 and $q \approx 0.3543$. With equal error costs, the success boundary is

$$0.25e + 0.08\theta = 3.$$

This line is a contour of equal probability, not a physical law of grasping.

Derive the action threshold from consequences. Suppose accepting a grasp that fails costs C_{FP}, while rejecting a grasp that would succeed costs C_{FN}. Assume the two correct decisions have zero cost. Given a success probability p, the expected costs are

$$\text{accept: } C_{\mathrm{FP}}(1 - p), \qquad \text{reject: } C_{\mathrm{FN}}p.$$

Accept if the first is smaller, giving

$$p \geq \tau = \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}, \qquad z \geq \log \frac{\tau}{1 - \tau}. \tag{1.9}$$

With $C_{\mathrm{FP}} = 8$ and $C_{\mathrm{FN}} = 1$, $\tau = 8/9 \approx 0.8889$ and the score threshold is $\log 8 \approx 2.0794$. Equation (1.8) then permits acceptance only when

$$0.25e + 0.08\theta \leq 3 - \log 8 \approx 0.9206.$$

The earlier 0.7685 example would pass a 0.5 rule but fail this cost-sensitive rule. The probability model has not changed; the action criterion has. In deployment this rule relies on useful probability estimates and appropriate costs. It is not a safety guarantee.

A complete reproducible experiment. The companion script generates 1,200 trials using seed 7, with independent $e \sim U(0, 20)$, $\theta \sim U(0, 40)$ and $Y \mid e, \theta \sim \mathrm{Bern}(q(e, \theta))$. It randomly splits them into 720 training, 240 validation, and 240 test observations. This random split is appropriate for this independently generated simulation. On real robot logs, split by object, session, or time as appropriate so near-duplicate frames and trials do not leak across sets.

Compute each feature's mean μ_j and standard deviation s_j on the training set only. Transform all sets using $u_j = (x_j - \mu_j)/s_j$ and prepend the bias column. Fit the standardized logistic model using full-batch gradient descent and a fixed illustrative slope penalty $\lambda = 0.02$. The script uses the curvature bound above to choose a conservative step size, with 1,800 updates. A practical study would select the penalty using validation data too.

Standardization changes coordinates, not the expressive family. If the learned score is $b_s + \sum_j a_{s,j}(x_j - \mu_j)/s_j$, its physical-unit coefficients are

$$a_j = \frac{a_{s,j}}{s_j}, \qquad b = b_s - \sum_j \frac{a_{s,j}\mu_j}{s_j}.$$

However, an L^2 penalty in standardized coordinates does change the implicit penalty on physical-unit coefficients. This is one reason scaling is a modeling choice as well as a numerical convenience.

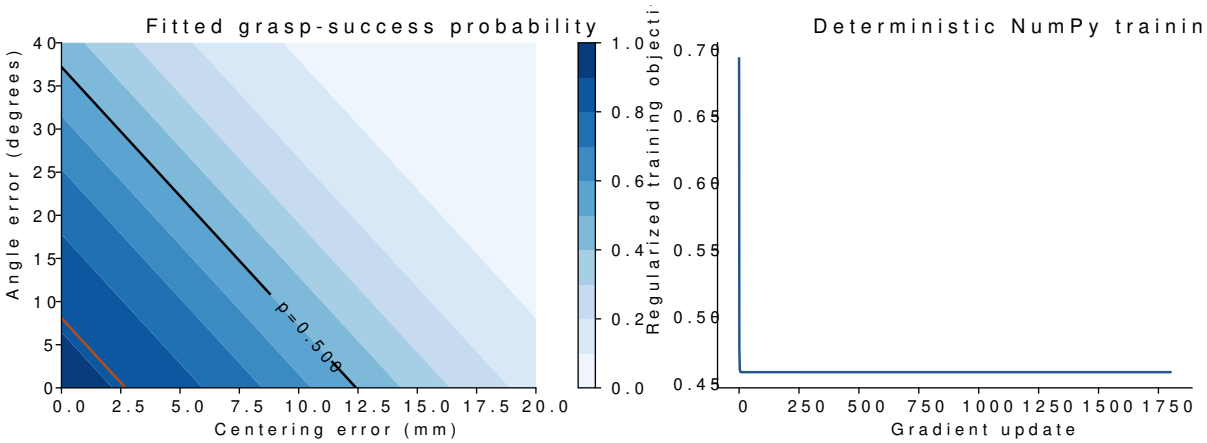


Figure 1.6: Synthetic robot experiment. Left: the fitted success field in physical units, with the 0.5 and 8/9 thresholds. Right: the regularized training objective decreases under the chosen full-batch step size.

Evaluate probabilities and decisions separately. Report unpenalized test BCE to measure probability quality, accuracy for a stated threshold, and empirical decision cost for the actual action criterion. A reliability diagram groups predicted probabilities into bins and compares each bin's mean probability with its observed success frequency. Small bins are noisy; an apparent deviation from the diagonal is not automatically systematic miscalibration. The script records each bin's sample count.

The script also selects a threshold from $0.05, 0.06, \ldots, 0.95$ using validation cost alone, then evaluates it on the test data. An empirically selected threshold can differ from 8/9 because of finite samples and imperfect probabilities. Do not select it by looking at test results: the test set is for the final assessment. For this seeded run the unpenalized test BCE is 0.4317 nats and accuracy at 0.5 is 80.0%. The validation-selected threshold is 0.84.

Decision threshold	Mean test cost
0.5	0.7542
8/9 (cost-derived)	0.3250
0.84 (validation-selected)	0.3708

Here validation tuning does not beat the cost-derived threshold on the test sample. This is a useful reminder that selecting a threshold on a finite validation set is itself an estimation problem. The full numerical output is in the adjacent `results.json`; all reported quantities can be regenerated from the code.

What would break this setup? A new gripper or different object material can change $q(x)$ even when the two features look familiar. Missing interactions may make affine log-odds inadequate. Selection bias arises if training only records grasps the old policy chose to attempt; rejected grasps may have no observed outcome. Label errors, rare failures, correlated trials, and feature drift also matter. More confident numbers alone solve none of these problems. The model's meaning is conditional on its observations, data collection, and deployment distribution.

1.1.16 From-scratch practice and checks

The executable companion is `deep_learning/code/logreg_1_1.py`. It uses NumPy for the model and optimization, and Matplotlib for figures; it uses no machine-learning library. Run it from the repository root:

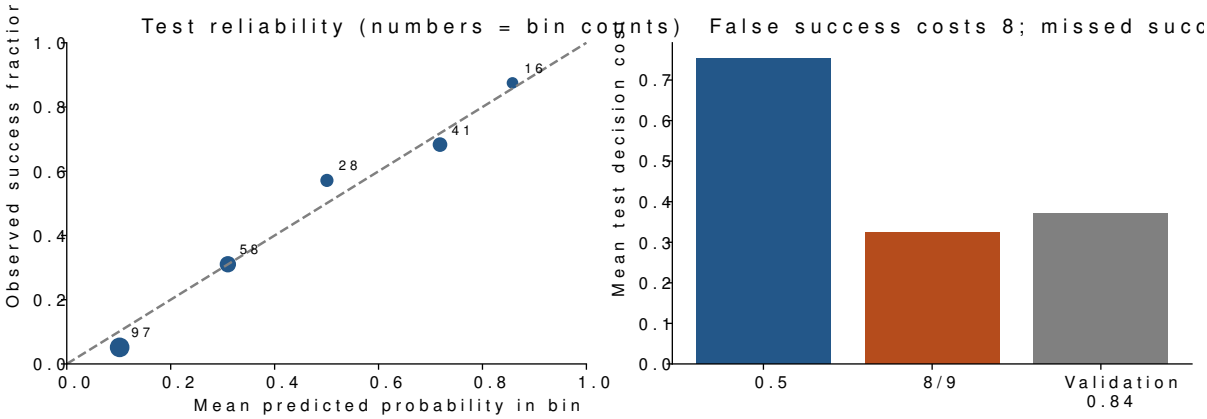


Figure 1.7: Held-out assessment for synthetic grasp trials. Left: a reliability diagram, with counts showing how much evidence supports each bin. Right: mean cost at a default threshold, the analytically derived cost threshold, and a validation-selected threshold. These are one simulation's measurements, not claims of general superiority or hardware performance.

```
python deep_learning/code/logreg_1_1.py
make dl
```

It regenerates seven vector-PDF figures and a JSON result summary in `deep_learning/figures/logreg_1_1/`. The chapter can be compiled with the committed figures without running Python. The full script includes an L^2-regularized objective, training, data splitting, standardization, evaluation, and the following meaningful numerical checks:

1. Compare the analytic gradient against centered finite differences, $[J(w + \varepsilon e_j) - J(w - \varepsilon e_j)]/(2\varepsilon)$.
2. Check the Hessian's eigenvalues on a small dataset and verify that the chosen full-batch updates do not increase the regularized objective.
3. Evaluate extreme logits ± 1000 and require finite losses and probabilities.
4. Check $\sigma(0) = 1/2$ and $\sigma(-z) = 1 - \sigma(z)$.

These checks catch implementation mistakes; they do not replace the mathematical proofs or establish generalization to a robot.

Suggested implementation order. First reproduce the two-point hand update using only arrays. Then implement stable sigmoid and BCE, verify the gradient numerically, add repeated updates, and plot the training loss. Only after that add standardization, a validation split, and decision costs. Predict what each change should do before running it. This makes the code an experiment on the mathematical ideas rather than a recipe.

1.1.17 Reasoning exercises

Try these before reading the worked answers below. They are independent of the main derivation, so the conceptual narrative need not be interrupted.

Exercise 1.1 (Same classes, different probabilities)**.** For $w = (-3, 1, 1)^\top$, compare w and $10w$ at $(1, 2)$ and $(2, 2)$. Do the boundaries, hard predictions, and losses for a positive label agree?

Exercise 1.2 (Recover a probability model)**.** An intercept-only dataset contains 20 successes and 30 failures. Derive the maximum-likelihood probability and intercept. What happens if all 50 succeed?

Exercise 1.3 (Costs change a boundary)**.** A classifier has score $z = -3 + x_1 + x_2$. Derive the action boundary when a false positive costs nine times a false negative. Which side predicts one?

Exercise 1.4 (Feature maps and convexity)**.** Use features (x_1, x_2, x_1x_2) to express XOR on $\{-1, 1\}^2$. Explain why the training objective is convex in its coefficients, despite the nonlinear boundary. Would the same proof apply if the features were outputs of a trained hidden layer?

Exercise 1.5 (Complexity and data shape)**.** For $N = 10^6$ and $d = 100$, identify the dominant forward-pass operation. Explain why storing labels as $(N, 1)$ while predictions have shape $(N,)$ is dangerous. How does a batch of size B change working memory and per-step time?

1.1.18 Worked answers and conceptual checkpoints

1. Scaling. At $(1, 2)$, both logits are zero and both probabilities are 0.5. At $(2, 2)$, the logits are 1 and 10, with probabilities about 0.7311 and 0.99995. Positive-label losses are $\log(1 + e^{-1}) \approx 0.3133$ and $\log(1 + e^{-10}) \approx 0.0000454$. Boundaries and hard classes agree. If the label at $(2, 2)$ were zero instead, the larger scale would be far more costly: about 10.00005 versus 1.3133. Confidence has consequences.

2. Intercept-only estimation. The loss is $-0.4 \log p - 0.6 \log(1 - p)$; differentiating gives the minimum $p = 0.4$. Thus $b = \log(0.4/0.6) = \log(2/3) \approx -0.4055$. If all labels are one, the loss decreases as $b \to +\infty$, and no finite unregularized optimum exists.

3. Costs. The threshold is $\tau = 9/10$, so the logit threshold is $\log 9$. The new boundary is $x_1 + x_2 = 3 + \log 9 \approx 5.1972$, with class one on the greater-than side. It is parallel to the equal-cost boundary and requires stronger evidence.

4. Fixed features. Take zero bias and zero coefficients on x_1, x_2, with coefficient $-k$ on x_1x_2, $k > 0$. The four signed scores have the required signs. The Hessian proof applies to the resulting fixed design matrix. With a trainable hidden layer, the feature matrix itself depends on parameters, and the positive semidefinite form above is no longer the full Hessian.

5. Computation. The dense product Xw uses roughly $N(d + 1)$ multiply–accumulate operations, about 10^8 here. Broadcasting $(N,)$ minus $(N, 1)$ can create an (N, N) array, which is both mathematically wrong and enormous. A dense batch step costs $\Theta(Bd)$ time and stores $\Theta(Bd)$ feature entries if batches are streamed, plus parameters and $\Theta(B)$ temporary predictions. An epoch still processes N observations and costs $\Theta(Nd)$.

Reconstruct the method from its meaning

An affine score describes additive log-odds. The sigmoid converts that evidence into a Bernoulli probability. Independence turns observed-label probabilities into a product likelihood, and the negative logarithm turns that product into an additive loss. The gradient $X^\top(p - y)/N$ adjusts evidence according to probability errors. Convexity follows from nonnegative Bernoulli variances, but finite solutions require more than convexity. Finally, the decision threshold comes from the consequences of actions, not from the probability model alone.

Scope and reading. This section expands the supplied lecture excerpts (slides 5–10 and their questions). Section 1.2 can develop entropy, KL divergence, and maximum likelihood more generally; the later sections can reinterpret the same affine–sigmoid computation as a neural-network unit. General background texts already present in this repository include Bishop [1] and Goodfellow et al. [5]. All plots in this section are generated by its companion script from explicit formulas or synthetic data.

1.2 Cross Entropy, KL Divergence, and Maximum Likelihood

Three quantities and three questions

Entropy asks: how unpredictable is the source itself? Cross-entropy asks: how surprised will a particular model be by that source? KL divergence asks: how much extra surprise does using that model create? Maximum likelihood chooses the model that assigns the greatest joint probability, or density, to the observed data. These ideas connect through one operation: averaging negative log probabilities under the distribution that supplies the observations.

1.2.1 Start with a robot that must describe what it sees

A mobile robot sends a stream of corridor events: clear path, person, cart, or blocked path. Suppose their long-run probabilities are

$$P = (1/2, 1/4, 1/8, 1/8).$$

Sending two bits for every event works: there are four possibilities. But the outcomes are not equally common. Could we give frequent outcomes shorter messages and rare ones longer messages, reducing the average communication cost?

Now imagine the robot's model predicts a different frequency distribution Q. Even if the communication channel is perfect, a code designed for Q can be wasteful on events actually drawn from P. This is a concrete way to separate *unavoidable uncertainty* from *avoidable model mismatch*. The same distinction applies to prediction: a noisy task can have a positive best possible log loss, while a confident but wrong model adds further loss.

The narrative in this section is

$$\text{surprise} \longrightarrow \text{average surprise} \longrightarrow \text{mismatch} \longrightarrow \text{estimating a model from data.}$$

We first work on a finite outcome space. Continuous measurements need additional care and are treated explicitly later. Throughout, P is the distribution that supplies outcomes and Q is the distribution used to score them, unless an expression explicitly swaps their roles.

1.2.2 Why should information be a negative logarithm?

Suppose an event has probability p. We seek a numerical measure $I(p)$ of the information gained when learning that it occurred. A plausible set of requirements is:

1. A certain event gives no new information: $I(1) = 0$.
2. A rarer event is more informative: $I(p)$ decreases as p increases.
3. For independent events, information adds: $I(pq) = I(p) + I(q)$.
4. The measure varies continuously with probability.

The independence clause is essential: probabilities multiply for independent events. For dependent events, the relevant factorization is $P(A, B) = P(A)P(B \mid A)$, so information adds using a conditional probability.

To see what these requirements imply, set $p = e^{-t}$ and define $F(t) = I(e^{-t})$ for $t \geq 0$. Then

$$F(t + s) = F(t) + F(s).$$

For positive integers, repeated addition gives $F(n) = nF(1)$. Dividing an interval into equal parts gives $F(m/n) = (m/n)F(1)$ for nonnegative rationals. Continuity extends this to real $t \geq 0$. Therefore $F(t) = ct$, with $c > 0$ by the rarity requirement, and

$$\boxed{I(p) = -c \ln p.}$$

The choice of c fixes the unit. With base-two logarithms, a fair coin outcome has information $-\log_2(1/2) = 1$ bit. With natural logarithms the unit is a nat. Since $\log_2 p = \ln p / \ln 2$, a value in nats divided by $\ln 2$ is its value in bits. Information here is about reduction of probabilistic uncertainty, not the semantic importance of a message. A rare sensor glitch can have high surprise without being useful for navigation.

One event versus a distribution. For $P = (1/2, 1/4, 1/8, 1/8)$, the four event surprises are $1, 2, 3, 3$ bits. A particular observed event has one of these values. Entropy will be the average of those values before we know which event will occur. Do not confuse the information of an outcome with the uncertainty of the whole distribution.

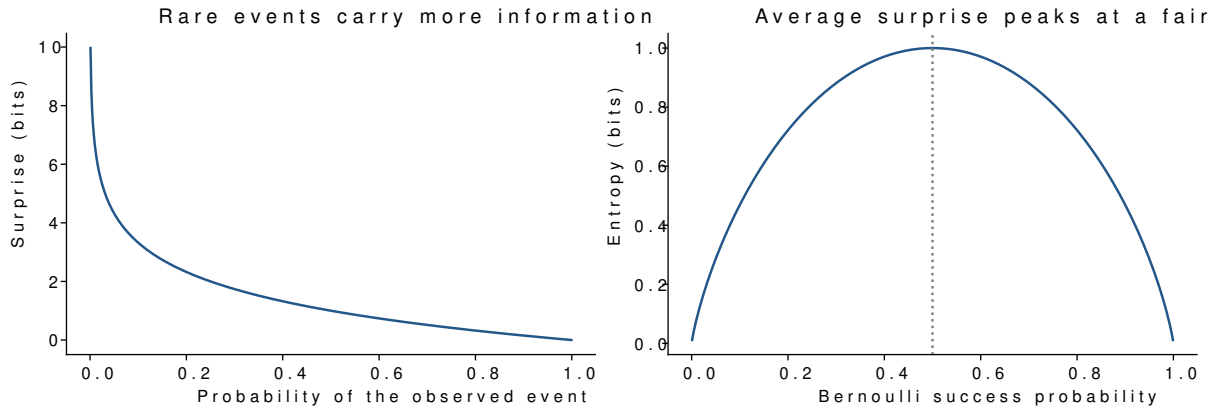


Figure 1.8: Left: surprise of one event grows as its probability decreases. Right: the average surprise of a Bernoulli source is largest at equal probabilities. These curves answer different questions: event surprise and source uncertainty.

1.2.3 Expectation turns surprise into entropy

For a finite random variable X with outcomes $x_1, \ldots, x_K$, let $p_k = P(X = x_k)$. By the definition of expectation, any function $f(X)$ has mean $\sum_k p_k f(x_k)$. Choosing $f(x) = -\log P(x)$ gives

$$\boxed{H(P) = \mathbb{E}_{X \sim P}[-\log P(X)] = -\sum_{k=1}^{K} p_k \log p_k.} \tag{1.10}$$

The weights outside the logarithm describe *how often* an event occurs; the logarithm describes *how surprising* that event is. Entropy is not $-\log \mathbb{E}[P(X)]$, and not an unweighted average over the list of possible outcomes. Moving a nonlinear logarithm outside an expectation changes the quantity.

For the robot stream, in bits,

$$H(P) = \tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = 1.75.$$

This is below the two bits needed by an equiprobable four-outcome source. A deterministic source has entropy zero: it never surprises an observer who knows the distribution. On a K-outcome space,

$$0 \le H(P) \le \log K,$$

with the upper bound attained by the uniform distribution. We will prove the upper bound from KL nonnegativity below. Zero-probability events contribute zero by the continuous extension $\lim_{t\downarrow 0} t \log t = 0$.

A thought experiment. Imagine predicting the next symbol from a fair coin source versus a coin that comes up heads 99% of the time. For the biased coin, usually there is almost nothing new to learn, but occasionally a tail is very surprising. Entropy balances these two facts through a frequency-weighted average. It does not simply count the number of logically possible outcomes.

1.2.4 Coding makes the average operational

A binary *prefix code* is a collection of bit strings in which no complete codeword is the prefix of another. This allows concatenated messages to be decoded without separators. For the robot stream, consider these three codes:

Event	True P	P-matched code	Uniform code	R-matched code
Clear	1/2	0	00	110
Person	1/4	10	01	111
Cart	1/8	110	10	10
Blocked	1/8	111	11	0

The last code matches $R = (1/8, 1/8, 1/4, 1/2)$. All three are valid prefix codes; they simply allocate short words differently. Their expected lengths under P are, respectively,

$$1.75, \qquad 2, \qquad \tfrac{1}{2}(3) + \tfrac{1}{4}(3) + \tfrac{1}{8}(2) + \tfrac{1}{8}(1) = 2.625 \text{ bits}.$$

The first code is most efficient for this source. The last makes the common "clear" event expensive and saves bits on a rare event instead.

Why ideal lengths are negative log probabilities. A complete binary code with lengths ℓ_k satisfies $\sum_k 2^{-\ell_k} = 1$. Think of each prefix as selecting a fraction $2^{-\ell_k}$ of the infinite binary tree. Assigning probability q_k the fraction q_k suggests the ideal length $\ell_k = -\log_2 q_k$. The code tree has only integer depths, however; these ideal lengths need not be integers. The dyadic probabilities in the table were chosen so they are.

More generally, the Kraft inequality says prefix-code lengths satisfy $S = \sum_k 2^{-\ell_k} \le 1$. Set $r_k = 2^{-\ell_k}/S$. Then, in bits,

$$\mathbb{E}_P[\ell_X] = -\sum_k p_k \log_2 r_k - \log_2 S = H(P) + D_{\mathrm{KL}}(P\|R) - \log_2 S \ge H(P).$$

This lower bound uses the KL nonnegativity proved shortly. Conversely, choosing $\ell_k = \lceil -\log_2 p_k \rceil$ gives a prefix code with

$$H(P) \le \mathbb{E}_P[\ell_X] < H(P) + 1$$

when the source has positive probabilities on its supported outcomes. For blocks of n independent events, applying the construction to the block probabilities gives an overhead of less than one bit *per block*, hence less than $1/n$ bit per event.

Read the slide's coding statement precisely

Entropy is an ideal average code length and an asymptotically achievable rate for an i.i.d. source, not necessarily the exact length of a one-symbol binary code. A Bernoulli source can have entropy 0.2 bits even though a single nontrivial binary symbol needs an integer-length codeword. Block coding is how fractional rates become meaningful. Dependencies require conditional or joint coding; the marginal entropy alone need not be the best rate for a sequence.

1.2.5 Cross-entropy: keep the source, change the scoring model

Suppose outcomes still come from P, but we evaluate them using probabilities $q_k = Q(X = x_k)$. The average model-assigned surprise is

$$\boxed{H(P, Q) = \mathbb{E}_{X\sim P}[-\log Q(X)] = -\sum_k p_k \log q_k.} \tag{1.11}$$

The first argument determines the averaging weights. The second determines the logarithmic score. Swapping the arguments changes both roles. This distinction is the single most useful way to read a cross-entropy expression.

For the robot source and a uniform model, every event receives two bits of surprise, so $H(P, Q) = 2$. For R above, $H(P, R) = 2.625$. In both cases the actual source remains P and has the same entropy 1.75.

For a general model Q with positive probability wherever P has positive probability, the ideal Q-based length is $-\log_2 q_k$. Rounded Shannon lengths have expected length between $H(P, Q)$ and $H(P, Q) + 1$ bits. This is an ideal or asymptotic mismatched-coding interpretation; an arbitrary integer code described as "optimized for Q" need not have length exactly $-\log_2 q_k$.

Support is part of the meaning. If $p_k > 0$ but $q_k = 0$, an event that really can happen has been declared impossible. Its log loss is infinite, so $H(P, Q) = +\infty$. If $p_k = 0$, that outcome is never scored under P and its term is defined to contribute zero. Do not evaluate $0 \log 0$ naively in numerical code.

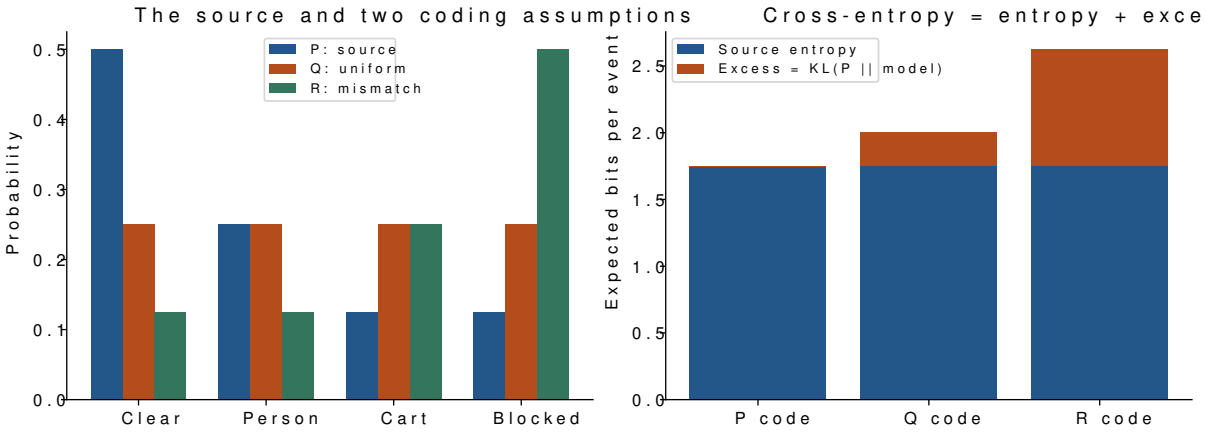


Figure 1.9: The same source is scored using three models. The unavoidable source entropy is unchanged. The additional orange part of each expected code length is KL divergence. The probabilities are dyadic, so the displayed coding costs are exact for the prefix codes in the table.

1.2.6 KL divergence is the excess expected log loss

Subtract the source entropy from cross-entropy, assuming the expressions are finite, and collect the logarithms:

$$\begin{aligned} H(P,Q) - H(P) &= \sum_k p_k(\log p_k - \log q_k) \\ &= \sum_k p_k \log \frac{p_k}{q_k} = \mathbb{E}_{X\sim P}\left[\log \frac{P(X)}{Q(X)}\right] =: D_{\mathrm{KL}}(P\|Q). \end{aligned} \tag{1.12}$$

Equivalently, $D_{\mathrm{KL}}(P\|Q) = -\mathbb{E}_P \log[Q(X)/P(X)]$, which matches the slide. Thus

$$\boxed{H(P,Q) = H(P) + D_{\mathrm{KL}}(P\|Q).}$$

For infinite quantities, do not treat $+\infty - \infty$ as a legitimate subtraction; define KL through the log-ratio expression with suitable support and integrability conditions.

Each individual log ratio may be positive or negative: a model can assign *more* probability than P to an outcome. KL is the average over P. Normalization forces a tradeoff: giving extra probability to some outcomes leaves less for others. The average cannot be negative.

Proof of nonnegativity, including zero probabilities. The elementary inequality $\log t \le t - 1$ for $t > 0$ follows by minimizing $t - 1 - \log t$: its derivative is $1 - 1/t$, and its minimum at $t = 1$ is zero. Let $S = \{k : p_k > 0\}$, and first suppose $q_k > 0$ on S. Applying the inequality to $t = q_k/p_k$ gives

$$\begin{aligned} -D_{\mathrm{KL}}(P\|Q) &= \sum_{k\in S} p_k \log \frac{q_k}{p_k} \\ &\le \sum_{k\in S} p_k \left(\frac{q_k}{p_k} - 1\right) = \sum_{k\in S} q_k - 1 \le 0. \end{aligned}$$

Equality requires $q_k = p_k$ for every $k \in S$ and no probability mass outside S; hence $P = Q$. If Q has a zero on positive P mass, KL is infinite and the inequality still holds. This proves both nonnegativity and the equality case.

Consequences. The best possible predictive distribution under expected log loss is $Q = P$. Moreover, using the uniform distribution $U_k = 1/K$,

$$D_{\mathrm{KL}}(P\|U) = \sum_k p_k \log p_k + \log K = \log K - H(P) \ge 0,$$

proving the maximum-entropy bound. An unrestricted distribution family can attain zero KL; a restricted model family may not contain P and may have a positive minimum mismatch.

1.2.7 Why KL is directed and is not a distance metric

For a concrete example, use natural logs and $P = (0.8, 0.2)$, $Q = (0.5, 0.5)$. Then

$$\begin{aligned} D_{\mathrm{KL}}(P\|Q) &= 0.8 \ln 1.6 + 0.2 \ln 0.4 \approx 0.192745, \\ D_{\mathrm{KL}}(Q\|P) &= 0.5 \ln 0.625 + 0.5 \ln 2.5 \approx 0.223144. \end{aligned}$$

Different sampling distributions produce different averages. KL is not symmetric and therefore cannot be a metric; it also does not generally satisfy the triangle inequality. It is nonnegative and identifies equality, but those two properties alone do not make a geometric distance.

For an explicit triangle-inequality counterexample, take $A = (1, 0)$, $B = (1/2, 1/2)$, and $C = (1/4, 3/4)$. Then $D(A\|C) = \ln 4 \approx 1.3863$, while $D(A\|B) + D(B\|C) = \ln 2 + \frac{1}{2}\ln(4/3) \approx 0.8370$. The direct divergence exceeds the sum through the intermediate distribution.

The support asymmetry is even sharper. For $P = (1, 0)$ and $Q = (1/2, 1/2)$, $D_{\mathrm{KL}}(P\|Q) = \ln 2$, while $D_{\mathrm{KL}}(Q\|P) = +\infty$. Under P only the first event is sampled. Under Q the second event occurs with positive probability but is impossible under P.

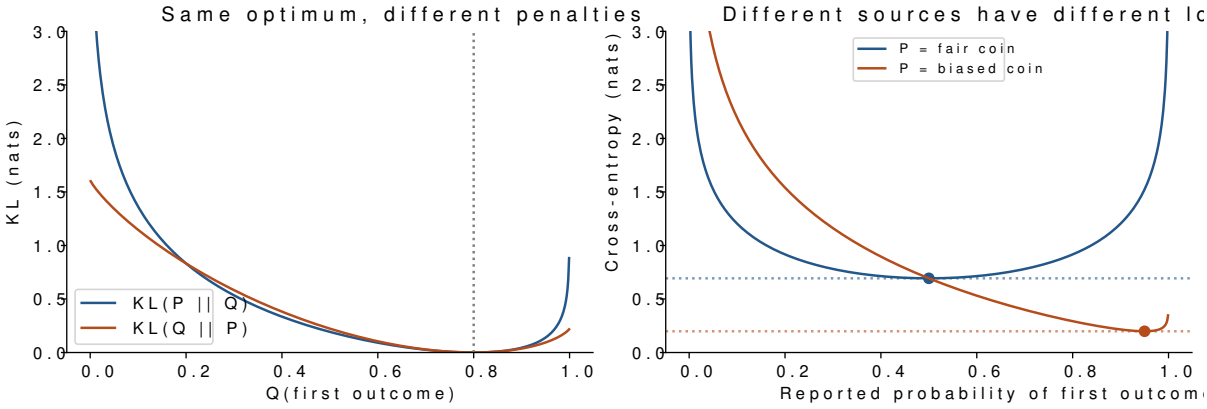


Figure 1.10: Left: the two directions have the same zero when the distributions match, but different penalties elsewhere. Right: different true sources have different minimum cross-entropies. Subtracting each source's entropy isolates model mismatch. Natural logs are used in both panels.

1.2.8 Conditioning: knowing the context changes uncertainty

A robot's observations are not drawn from one label distribution independently of its inputs. Let X denote the observed context and Y a semantic label. The relevant uncertainty is the conditional entropy

$$H_P(Y \mid X) = \mathbb{E}_{X \sim P_X}\left[-\sum_y P(y \mid X) \log P(y \mid X)\right].$$

We average uncertainty *within* each context over how often contexts occur. Using $P(x, y) = P(x)P(y \mid x)$ and taking negative logs and expectations gives the chain rule

$$H_P(X, Y) = H_P(X) + H_P(Y \mid X).$$

The expected information gained about Y from knowing X is

$$I_P(X; Y) = H_P(Y) - H_P(Y \mid X) = D_{\mathrm{KL}}(P_{XY}\|P_X P_Y) \geq 0.$$

This follows by expanding the joint log ratio and collecting the marginal and conditional terms. Conditioning reduces uncertainty *on average*; a particular unusual context can have more label entropy than the marginal.

For a conditional model $Q_\theta(y \mid x)$, expected log loss decomposes as

$$\underbrace{\mathbb{E}_{P_{XY}}[-\log Q_\theta(Y \mid X)]}_{\text{conditional cross-entropy}} = H_P(Y \mid X) + \mathbb{E}_{P_X} D_{\mathrm{KL}}(P(\cdot \mid X)\|Q_\theta(\cdot \mid X)). \tag{1.13}$$

To derive it, at each fixed x add and subtract $-\sum_y P(y \mid x) \log P(y \mid x)$, then average over x. This is the version most directly relevant to supervised learning.

1.2.9 Maximum likelihood: choosing a model after seeing data

We now move from two known distributions to an unknown source. Let $\mathcal{D} = (x^{(1)}, \ldots, x^{(m)})$ be observations drawn independently from a common but unknown distribution P_{data}. A parametric family Q_θ is a set of candidate distributions indexed by θ.

Before observing data, $Q_\theta(x)$ is a probability as a function of x for a fixed parameter. After observing the data, the same formula, considered as a function of θ, is a *likelihood*. The data are held fixed while we compare models:

$$\mathcal{L}(\theta; \mathcal{D}) = Q_\theta(\mathcal{D}), \qquad \hat{\theta}_{\text{ML}} \in \arg\max_\theta \mathcal{L}(\theta; \mathcal{D}).$$

We use membership in the argmax set because a maximizer might not be unique. There can also be no finite maximizer, as with separable unregularized logistic regression in Section 1.1. An *estimator* is a rule mapping a dataset to a parameter value; before sampling, it is itself random. Its realized output is an estimate.

Likelihood is not a posterior probability of θ. A posterior requires a prior through Bayes' rule. A high likelihood also does not by itself establish that the family is correct, or that the estimate will generalize.

Why independence appears in the derivation. Under the i.i.d. assumption,

$$Q_\theta(\mathcal{D}) = \prod_{i=1}^{m} Q_\theta(x^{(i)}).$$

For correlated observations, one instead needs an appropriate joint model, for example

$$Q_\theta(x^{(1)}) \prod_{i=2}^{m} Q_\theta(x^{(i)} \mid x^{(1:i-1)}).$$

The logarithm still turns a product into a sum, but the factors are conditional. Treating adjacent robot-video frames as independent is a modeling assumption, not an automatic consequence of having stored them in separate rows.

1.2.10 The empirical distribution: turning a dataset into an expectation

For discrete outcomes, define

$$\hat{P}_m(x) = \frac{1}{m} \sum_{i=1}^{m} \mathbf{1}\{x^{(i)} = x\}.$$

It is the observed relative frequency of each outcome. Repeated observations count repeatedly. For any function f,

$$\begin{aligned} \mathbb{E}_{\hat{P}_m}[f(X)] &= \sum_x \left[\frac{1}{m} \sum_i \mathbf{1}\{x^{(i)} = x\} \right] f(x) \\ &= \frac{1}{m} \sum_i \sum_x \mathbf{1}\{x^{(i)} = x\} f(x) = \frac{1}{m} \sum_i f(x^{(i)}). \end{aligned}$$

This is why a sample average can be written as an expectation: we have changed the averaging distribution to the empirical one. It is not an assertion that $\hat{P}_m$ equals the unknown true distribution.

For the sequence (A, A, A, B, C), the empirical distribution over (A, B, C) is $(3/5, 1/5, 1/5)$, not $(1/3, 1/3, 1/3)$. If we instead regard the five observation *indices* as equally likely, each has mass $1/5$; mapping indices to their values combines the repeated A outcomes. Keeping those spaces distinct avoids many empirical-entropy mistakes.

1.2.11 Every step of the MLE–cross-entropy–KL equivalence

For a discrete i.i.d. model with probabilities positive on observed outcomes,

$$\begin{aligned}
\arg\max_\theta Q_\theta(\mathcal{D}) &= \arg\max_\theta \prod_{i=1}^m Q_\theta(x^{(i)}) && \text{(independence)}\\
&= \arg\max_\theta \sum_{i=1}^m \log Q_\theta(x^{(i)}) && \text{(monotone log)}\\
&= \arg\max_\theta \frac{1}{m}\sum_{i=1}^m \log Q_\theta(x^{(i)}) && \text{(positive scaling)}\\
&= \arg\max_\theta \mathbb{E}_{\hat{P}_m} \log Q_\theta(X) && \text{(empirical mean)}\\
&= \arg\min_\theta \mathbb{E}_{\hat{P}_m}[-\log Q_\theta(X)] && \text{(negation)}\\
&= \arg\min_\theta H(\hat{P}_m, Q_\theta) && \text{(definition)}\\
&= \arg\min_\theta \{H(\hat{P}_m, Q_\theta) - H(\hat{P}_m)\} && \text{(constant in } \theta)\\
&= \arg\min_\theta D_{\mathrm{KL}}(\hat{P}_m \| Q_\theta). && (1.14)
\end{aligned}$$

These are equal sets of optimizers, not equal numerical objectives. In particular, the sum of log probabilities equals m times the empirical expectation, not the expectation itself. The factor disappears only because a positive constant does not change the argmax. Likewise, removing empirical entropy changes the objective value but not its optimizer or gradient.

Which way does the KL point? The empirical distribution comes first: it supplies the data being averaged. The model comes second: its probabilities are evaluated on those data. This is forward KL, $D_{\mathrm{KL}}(\hat{P}_m \| Q_\theta)$, not $D_{\mathrm{KL}}(Q_\theta \| \hat{P}_m)$. The order is determined by the likelihood calculation, not by a freely chosen convention.

Population interpretation. If the relevant expectations are finite, the population objective is

$$\mathbb{E}_{P_{\text{data}}}[-\log Q_\theta(X)] = H(P_{\text{data}}) + D_{\mathrm{KL}}(P_{\text{data}} \| Q_\theta).$$

Training substitutes a sample average for this unknown expectation. A law of large numbers gives convergence for a fixed θ under suitable assumptions; consistency of the *optimizer* requires additional conditions, such as identifiability and control of the objective over the family. Finite-sample maximum likelihood does not automatically find the true distribution. If the family is misspecified, even a population-optimal parameter can retain positive KL.

1.2.12 Worked MLEs: Bernoulli and categorical models

For a Bernoulli model $Q_\theta(1) = \theta$, let s of m labels be one. The likelihood of the observed ordered sequence is

$$\mathcal{L}(\theta) = \theta^s (1-\theta)^{m-s}.$$

If only the count is observed, its binomial probability includes $\binom{m}{s}$, but this factor is independent of θ and leaves the estimate unchanged. For $0 < s < m$,

$$\frac{d}{d\theta}\log \mathcal{L}(\theta) = \frac{s}{\theta} - \frac{m-s}{1-\theta} = 0 \quad \Longrightarrow \quad \hat{\theta} = \frac{s}{m}.$$

The second derivative is $-s/\theta^2 - (m-s)/(1-\theta)^2 < 0$, proving this is the unique interior maximum. Seven successes in ten trials therefore give 0.7. With all successes the maximum is at

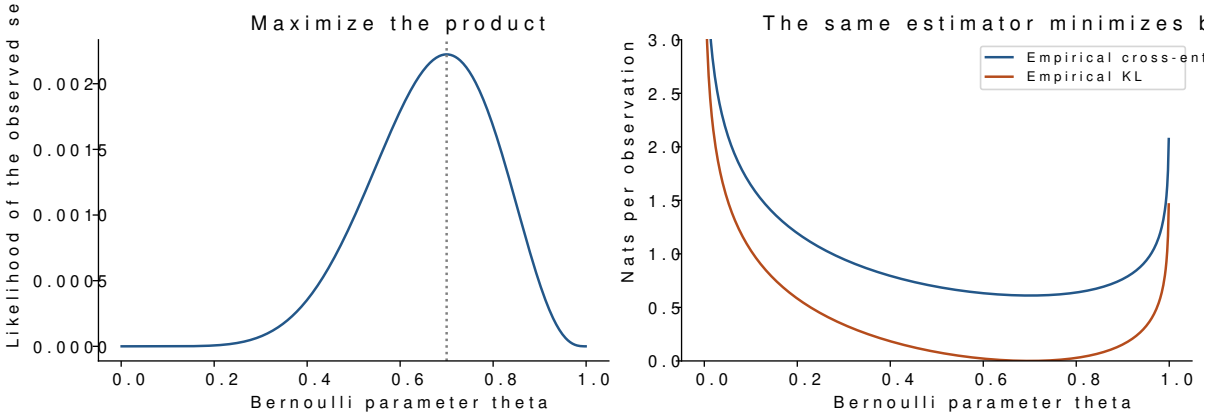


Figure 1.11: Seven successes and three failures. The product likelihood peaks at 0.7. Mean negative log-likelihood and empirical KL have the same minimizer; their vertical separation is the entropy of the empirical Bernoulli distribution.

$\theta = 1$ if the closed interval $[0, 1]$ is allowed; a finite-logit parameterization cannot reach it and only approaches the supremum. Parameter space matters.

For a K-class categorical model with counts n_k and probabilities q_k, maximize $\sum_k n_k \log q_k$ subject to $\sum_k q_k = 1$. When all counts are positive, the Lagrangian

$$\Lambda(q, \lambda) = \sum_k n_k \log q_k + \lambda \left(1 - \sum_k q_k\right)$$

gives $n_k/q_k = \lambda$. Summing $q_k = n_k/\lambda$ gives $\lambda = m$, so $q_k = n_k/m$. Concavity of the log objective makes this a global optimum. Zero-count classes obtain zero probability in the unrestricted closed simplex; this can produce infinite future log loss if a previously unseen class appears. Pseudocounts or priors can address that issue, but then the estimator is no longer this unregularized maximum-likelihood estimator. With a finite softmax parameterization, zero probabilities are approached only as limits.

1.2.13 Continuous data: densities are not point probabilities

A continuous measurement such as position has $P(X = x) = 0$ at every individual point, so a continuous-data likelihood uses a *density* $q_\theta(x)$, not point mass. For a small measurement bin of width Δ,

$$P_\theta(x \le X < x + \Delta) \approx q_\theta(x)\Delta.$$

With a fixed measurement scale, the product of Δ factors does not depend on θ, explaining why maximizing the density likelihood is meaningful. A density can exceed one, and a negative log density can be negative.

For densities p, q relative to the same base measure, define differential entropy and cross-entropy when the integrals exist:

$$h(P) = -\int p(x) \log p(x)\, dx, \qquad h(P, Q) = -\int p(x) \log q(x)\, dx.$$

Unlike discrete entropy, differential entropy can be negative and depends on units. For $U = aX + b$, $a \neq 0$, change of variables yields

$$p_U(u) = \frac{p_X((u - b)/a)}{|a|}, \qquad h(U) = h(X) + \log |a|.$$

Converting meters to centimeters adds $\log 100$ nats. Differential entropy by itself is therefore not an absolute number of bits needed to encode a real number exactly. Fine quantization gives, under appropriate regularity, $H(X_\Delta) \approx h(X) + \log(1/\Delta)$ in one dimension.

By contrast,

$$D_{\mathrm{KL}}(P\|Q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx$$

is invariant under a common invertible smooth coordinate transformation: the Jacobian factors cancel inside the density ratio, and the outer measure changes consistently. It remains nonnegative, and it equals $h(P, Q) - h(P)$ when that subtraction is well defined.

A subtle limit of the empirical-KL shorthand

For continuous observations, the empirical measure is $\hat{P}_m = m^{-1} \sum_i \delta_{x^{(i)}}$, a collection of point masses. A smooth density model assigns zero probability to that finite set, whereas $\hat{P}_m$ assigns it probability one. Thus $D_{\mathrm{KL}}(\hat{P}_m\|Q_\theta) = +\infty$ in the literal measure-theoretic sense. The finite sample negative log-density is still a valid MLE objective, but it is not a finite KL between those two measures. Use the population-density KL interpretation, or explicitly discretize/smooth consistently. The discrete chain in (1.14) should not be transferred blindly to Dirac empirical measures and continuous densities.

This issue does not invalidate conditional classification: for each observed input, the predicted distribution over a finite set of labels is discrete. The per-example categorical cross-entropy remains well defined even when the input itself is continuous.

1.2.14 Interpreting the slide: approximating two modes by one Gaussian

Suppose a robot's position belief has two plausible locations because two corridors look similar. Use the explicit illustrative density

$$p(x) = \tfrac{1}{2}\mathcal{N}(x; -3, 0.7^2) + \tfrac{1}{2}\mathcal{N}(x; 3, 0.7^2).$$

We are forced to summarize it with a single Gaussian $q_{\mu,s}(x) = \mathcal{N}(x; \mu, s^2)$, $s > 0$. No member of this family can represent both narrow peaks faithfully. Which compromise does each KL choose?

Forward KL: derive the moment-matching solution. Minimizing $D_{\mathrm{KL}}(P\|Q_{\mu,s})$ is equivalent to minimizing $\mathbb{E}_P[-\log q_{\mu,s}(X)]$, because $h(P)$ is constant. The Gaussian negative log density gives

$$h(P, Q_{\mu,s}) = \frac{1}{2}\log(2\pi s^2) + \frac{\mathbb{E}_P[(X-\mu)^2]}{2s^2}.$$

Let $m_P = \mathbb{E}_P X$ and $v_P = \mathrm{Var}_P X$. Expanding $X - \mu = (X - m_P) + (m_P - \mu)$ and noting the cross term has mean zero gives

$$\mathbb{E}_P[(X-\mu)^2] = v_P + (m_P - \mu)^2.$$

For any s, the minimizing mean is $\mu = m_P$. Write $v = s^2$; then

$$\frac{d}{dv}\left[\frac{1}{2}\log(2\pi v) + \frac{v_P}{2v}\right] = \frac{v - v_P}{2v^2}.$$

It is negative below v_P and positive above, so the minimizing variance is $v = v_P$ when $v_P > 0$. For our symmetric mixture,

$$\boxed{\mu^* = 0, \qquad (s^*)^2 = 3^2 + 0.7^2 = 9.49, \qquad s^* \approx 3.0806.}$$

The broad fitted Gaussian covers both modes, even though it assigns appreciable density to the low-probability region between them. Ignoring either mode would make many P-drawn observations extremely surprising. That is what forward KL penalizes. The mean position zero is a statistical summary, not necessarily a physically plausible robot location.

Reverse KL: why a single mode can win. The reverse objective is

$$D_{\mathrm{KL}}(Q_{\mu,s}\|P) = \mathbb{E}_Q[\log q_{\mu,s}(X) - \log p(X)] = -h(Q_{\mu,s}) - \mathbb{E}_Q \log p(X).$$

Now the fitted model supplies the samples. A broad Q generates many samples between the two peaks, where $p(x)$ is tiny, producing a large penalty. A narrow Q centered near one peak rarely visits the valley or the other peak. Missing the other mode is less directly penalized because expectations are under Q. The $-h(Q)$ term matters too: it discourages collapsing a continuous Gaussian to an infinitesimally narrow spike. Reverse KL is not merely "maximize p at the model mean."

Near the left mode, if the peaks are well separated, $p(x) \approx \frac{1}{2}\mathcal{N}(x; -3, 0.7^2)$. Choosing Q close to that component makes $\log(q/p) \approx \log 2$, so its reverse KL is roughly 0.6931 nats. The companion script searches Gaussian mean and scale on a grid, refines the best grid neighborhood, and evaluates expectations by Gauss–Hermite quadrature. It finds a fit near $(\mu, s) = (-3, 0.7)$ with reverse KL about 0.69312. By symmetry a corresponding fit near $(3, 0.7)$ has the same value. This numerical search supports the displayed example; it is not a theorem that reverse KL always selects exactly one mode or a proof of global optimality for every mixture.

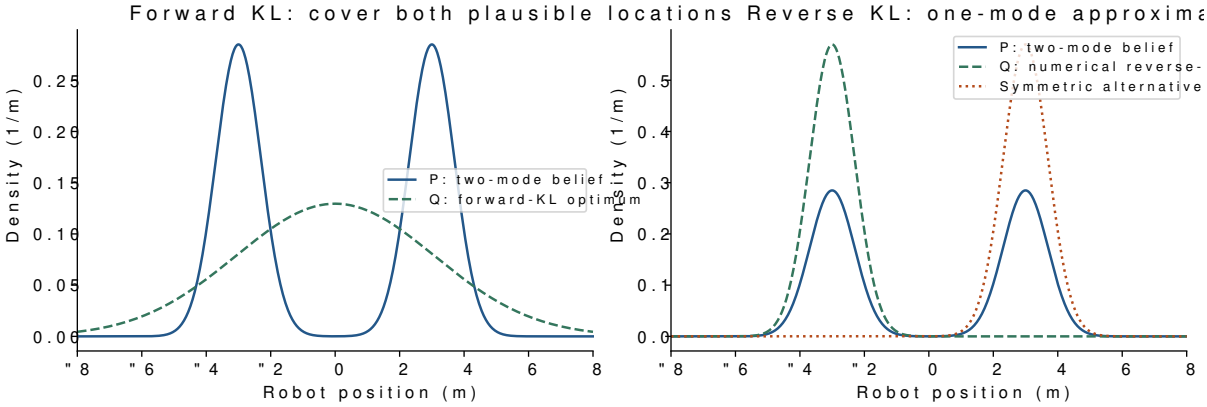


Figure 1.12: A constructed counterpart of the slide's qualitative example, with explicit distributions. Forward KL has an analytically derived Gaussian optimum. Reverse KL has two symmetric one-mode candidates found numerically. The mismatch is unavoidable because a single Gaussian cannot express this two-mode belief.

When the slogan fails. "Forward KL covers modes; reverse KL seeks modes" is a useful tendency for restricted approximating families and well-separated targets. If the family contains P, both directions can achieve the exact match. Overlapping modes, unequal mixture weights, or a richer family can change the compromise. Neither direction is universally better. For localization, throwing away a plausible mode can hide ambiguity, while a broad Gaussian can place probability in impossible corridors. A multimodal representation may be preferable to either single-Gaussian approximation.

Why maximum likelihood is the left-hand case. MLE averages $-\log q_\theta(x)$ over *observed data*. It asks whether the model makes the data plausible, giving forward KL in the population/discrete interpretation. It does not sample from Q_θ and ask whether those samples are

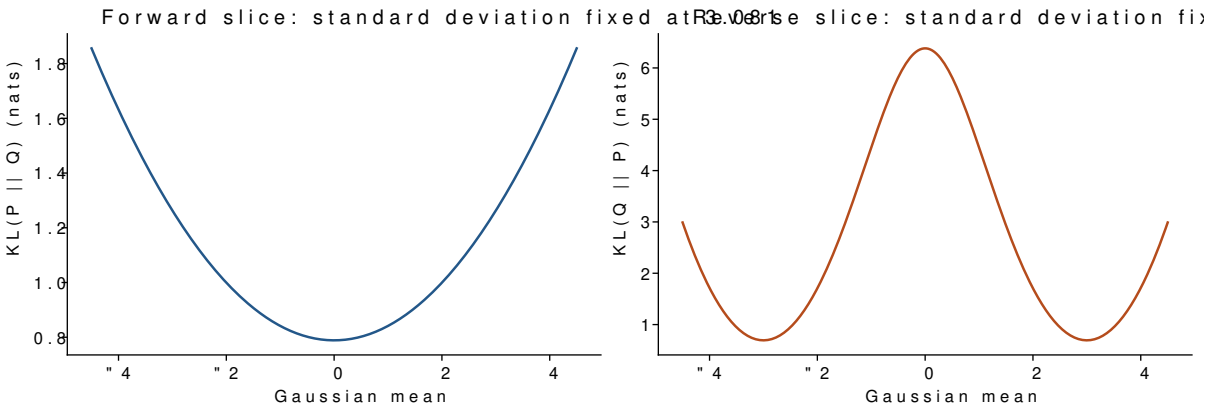


Figure 1.13: One-dimensional slices through two different objectives. Each panel holds the standard deviation at its displayed fitted value and varies the mean. The forward slice favors the midpoint; the reverse slice favors either peak. These slices are not the full two-parameter optimization landscapes.

plausible under P. Reverse-KL objectives arise in other settings, but do not follow from the likelihood derivation just given.

1.2.15 Transfer question 1: why track KL rather than cross-entropy?

Because KL removes the uncertainty floor. A perfect model of a fair binary source still has cross-entropy $\log 2 \approx 0.6931$ nats. A perfect model of a source with probabilities $(0.95, 0.05)$ has cross-entropy about 0.1985 nats. Both have KL zero. A larger cross-entropy does not necessarily mean the model is further from its source; the source may simply be less predictable.

For a fixed known P, KL states the excess expected loss relative to the unrestricted optimum $Q = P$. For example, if $H(P) = 0.60$ and $H(P, Q) = 0.65$, the model pays 0.05 extra nats per observation. Cross-entropy alone would not reveal that gap unless we knew the entropy floor.

Three qualifications make this useful in practice. First, with P fixed, subtracting $H(P)$ leaves gradients and optimizer locations unchanged. Computing KL does not improve optimization by itself. Second, if the model family cannot represent P, its best attainable KL may be positive. The actual optimization gap within that family is

$$D_{\mathrm{KL}}(P\|Q_\theta) - \inf_{\vartheta} D_{\mathrm{KL}}(P\|Q_\vartheta),$$

not simply $D_{\mathrm{KL}}(P\|Q_\theta)$. Third, true P and its entropy are usually unknown. Held-out samples let us estimate cross-entropy, while estimating entropy accurately can be difficult, especially in high dimensions.

If labels are represented by one-hot targets, the target entropy for each example is zero, so per-example cross-entropy and KL to that point mass are numerically the same. That does not prove the true conditional label entropy is zero. With noisy or ambiguous labels, the real uncertainty can remain positive. Known simulator distributions, repeated outcomes at the same finite context, or soft target distributions are settings in which the entropy subtraction can be directly informative.

1.2.16 Transfer question 2: derive binary cross-entropy as a special case

For an input x_n, the model's distribution over labels $0, 1$ is

$$Q_n = (1 - \hat{y}_n, \hat{y}_n), \qquad \hat{y}_n = p_w(Y = 1 \mid x_n).$$

The observed label $y_n \in \{0, 1\}$ defines a target distribution concentrated on that outcome:

$$P_n = (1 - y_n, y_n).$$

Substituting these two probability vectors into the general finite-outcome cross-entropy, with the sum over *labels*, gives

$$\begin{aligned} H(P_n, Q_n) &= -\sum_{k \in \{0,1\}} P_n(k) \log Q_n(k) \\ &= -(1 - y_n) \log(1 - \hat{y}_n) - y_n \log \hat{y}_n. \end{aligned}$$

Averaging over the N observations gives exactly the lecture formula:

$$\boxed{L(w) = -\frac{1}{N} \sum_{n=1}^{N} \left[y_n \log \hat{y}_n + (1 - y_n) \log(1 - \hat{y}_n) \right].}$$

There are two different averages: the inner sum is over possible labels at a fixed input; the outer sum is over dataset examples. The labels and predictions across all examples are not themselves one normalized probability vector.

An equivalent joint construction avoids ambiguity: put mass $1/N$ on each example index n, and define $P(n, k) = N^{-1}\mathbf{1}\{k = y_n\}$ and $Q_w(n, k) = N^{-1}Q_n(k)$. Then

$$H(P) = \log N, \quad H(P, Q_w) = \log N + L(w), \quad D_{\mathrm{KL}}(P \| Q_w) = L(w).$$

This index-based joint space is not the same as grouping repeated inputs into an empirical conditional label distribution, whose entropy may be positive. Both constructions produce the same averaged conditional log loss; they assign entropy to different random variables.

Soft targets and more than two classes. If a target supplies a probability $r_n \in [0, 1]$ rather than a hard label, replace y_n by r_n. The same formula becomes the cross-entropy between two Bernoulli distributions. Its KL equals that BCE minus the target entropy. For K classes, target probabilities r_{nk} and predictions q_{nk} give

$$L = -\frac{1}{N} \sum_{n} \sum_{k=1}^{K} r_{nk} \log q_{nk}.$$

With one-hot targets this reduces to the negative log probability of the correct class. Label smoothing changes the target distribution, hence also its entropy and the optimum; it is not merely a different algebraic form of the original hard-label objective.

1.2.17 Worked semantic-perception example: separate uncertainty from error

Consider a robot semantic classifier with labels free space, person, and cart. For a transparent teaching example, the input has only two discrete contexts: open floor and occluded view. In 100 observations of each context, the label counts are

Context	Free	Person	Cart
Open floor	80	15	5
Occluded view	10	20	70

The empirical conditional distributions are $\hat{P}_A = (0.80, 0.15, 0.05)$ and $\hat{P}_B = (0.10, 0.20, 0.70)$. Suppose a model predicts $Q_A = (0.70, 0.20, 0.10)$ and $Q_B = (0.20, 0.30, 0.50)$. Both contexts have weight $1/2$ in this dataset. Directly from the counts,

$$\mathrm{NLL}_{\mathrm{mean}} = -\frac{1}{200} \sum_{c \in \{A,B\}} \sum_{k} n_{ck} \log Q_c(k) \approx 0.764413 \text{ nats.}$$

The empirical conditional entropy and mismatch are

$$\hat{H}(Y \mid X) = \tfrac{1}{2} H(\hat{P}_A) + \tfrac{1}{2} H(\hat{P}_B) \approx 0.707344,$$
$$\hat{D} = \tfrac{1}{2} D_{\mathrm{KL}}(\hat{P}_A \| Q_A) + \tfrac{1}{2} D_{\mathrm{KL}}(\hat{P}_B \| Q_B) \approx 0.057069.$$

They add to 0.764413, as predicted by (1.13). An unrestricted categorical model per context would fit the empirical frequencies and reduce the empirical KL to zero, but its cross-entropy would remain 0.707344. The remaining label variation cannot be predicted from these two context categories in this empirical model.

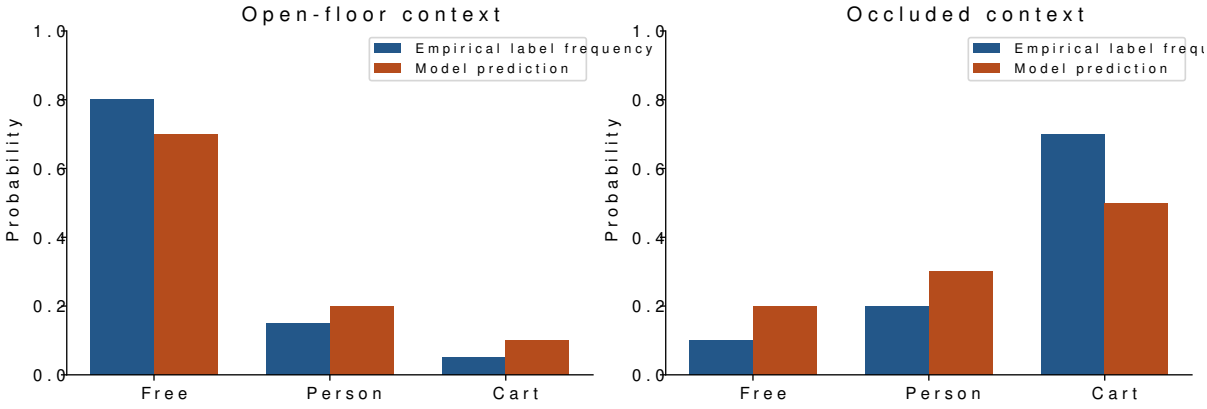


Figure 1.14: The robot example's target frequencies and model probabilities. The occluded context underpredicts carts; the resulting conditional KL measures that mismatch separately from the label variability within each context.

This is an in-sample decomposition of constructed counts, not an estimate of irreducible uncertainty on all future robot observations. Richer inputs might resolve ambiguity; finite frequencies are noisy; context proportions can shift. For a real study, fit models on training data, compare predictive log loss on held-out objects or sessions, and report how any entropy estimate was obtained. Do not estimate an entropy floor from the same overfit model and then call it known ground truth.

1.2.18 Implement the definitions without hiding their edge cases

The companion uses NumPy for the definitions and numerical experiments. A stable finite-distribution KL calculation must ignore zero P terms while returning infinity when positive P meets zero Q:

```
def kl(p, q):
    p = np.asarray(p, dtype=float)
    q = np.asarray(q, dtype=float)
    # Validate matching shapes, nonnegativity and sums of one first.
    active = p > 0
    if np.any(q[active] == 0):
        return float("inf")
    return np.sum(p[active] *
                  (np.log(p[active]) - np.log(q[active])))
```

The full script validates probability vectors and supports bits or nats. Using a mask before evaluating logarithms avoids invalid inactive terms. Adding an arbitrary epsilon to every probability changes the distributions and may destroy normalization. If smoothing is intended, define and normalize the smoothed model explicitly. For network outputs, compute log probabilities from logits using stable log-sum-exp; avoid taking logs after probabilities have rounded to zero.

In the mixture example, use

$$\log p(x) = \operatorname{logaddexp}\big(\log \mathcal{N}(x; -3, 0.7^2), \log \mathcal{N}(x; 3, 0.7^2)\big) - \log 2.$$

Computing each tiny density and adding them first can underflow in the tails. For reverse KL, write $X = \mu + sZ$, $Z \sim \mathcal{N}(0, 1)$, and integrate over Z. Gauss–Hermite quadrature approximates this expectation without truncating the real line to a plotting window. The script checks the selected candidate with both 120 and 240 quadrature nodes; their reverse-KL estimates differ by less than 10^{-8} for this example. Numerical integration and grid search remain approximations, not symbolic proofs of an optimum.

Reproduce and inspect. From the repository root:

```
python deep_learning/code/information_1_2.py
make dl
```

Seven vector figures and a numerical summary are written under `deep_learning/figures/information_1_2/`. Committed figures allow the notes to compile without running Python. The script checks the entropy decomposition, KL support cases, empirical-NLL equivalence, the BCE special case, and coordinate invariance for a Gaussian KL. It uses no machine-learning library and labels all examples as constructed data.

1.2.19 Reasoning exercises

Try these before the worked answers. Explain what distribution supplies the expectation in every expression, rather than starting with algebra alone.

Exercise 1.6 (An exact code)**.** For $P = (1/2, 1/4, 1/8, 1/8)$, use the three codebooks above to verify each expected length. How many extra ideal bits does the uniform code use over 1,000 i.i.d. events on average? Does this say every sequence uses exactly that many extra bits?

Exercise 1.7 (Asymmetry and impossible events)**.** Evaluate both KL directions for $P = (1, 0)$ and $Q = (1/2, 1/2)$ in bits. Which expectation encounters an impossible event, and under which model?

Exercise 1.8 (MLE without a remembered formula)**.** For a categorical dataset with counts $(12, 6, 2)$, derive its unrestricted MLE using a Lagrange multiplier. What changes if the last count is zero?

Exercise 1.9 (Two sources, the same reported loss)**.** Two models each have cross-entropy 0.7 nats. Their source entropies are 0.69 and 0.2. Which model is closer in forward KL to its own source? Can you infer which has higher classification accuracy?

Exercise 1.10 (Moments and modes)**.** For $P = \frac{1}{2}\mathcal{N}(-a, s_0^2) + \frac{1}{2}\mathcal{N}(a, s_0^2)$, derive the forward-KL Gaussian mean and variance. Explain what happens as $a \to 0$ and why this limits a universal "reverse KL chooses one peak" slogan.

Exercise 1.11 (One-hot targets are not a population claim)**.** A repeated input produces label one in 70% of trials. What probability minimizes expected BCE? Why can every observed one-hot target have zero entropy while the conditional source entropy is positive?

1.2.20 Worked answers and checks on interpretation

1. Coding. The averages are 1.75, 2, and 2.625 bits per event. The uniform code costs $0.25 \times 1000 = 250$ extra bits in expectation. Individual sequences have fluctuating outcome counts, so the realized difference need not equal 250. The statement concerns an average under the source distribution.

2. Support. $D_{\mathrm{KL}}(P\|Q) = \log_2(1/(1/2)) = 1$ bit. The reverse direction is infinite: Q samples the second event with probability $1/2$, but P assigns it zero probability. In the forward direction that event is never sampled.

3. Categorical MLE. The stationary equations give $q_k = n_k/\lambda$ and $\lambda = 20$, hence $(0.6, 0.3, 0.1)$. If the dataset instead has counts $(12, 6, 0)$, its total is 18 and the closed-simplex optimum is $(2/3, 1/3, 0)$. A finite softmax can approach but not reach this boundary. A future third-class observation would have infinite loss under the unsmoothed boundary solution.

4. Excess loss. The KL values are 0.01 and 0.5 nats. The first is much closer in this directed log-loss sense, even though raw cross-entropies agree. Classification accuracy cannot be inferred from these quantities alone: it discards confidence and also depends on a decision rule and the source distribution.

5. Gaussian approximation. The mean is zero by symmetry. The second moment is $a^2 + s_0^2$, so the variance is also $a^2 + s_0^2$. As $a \to 0$, the mixture becomes one Gaussian and both KL directions can achieve zero with that same Gaussian. The two-mode behavior is about approximating a separated multimodal target with a restricted family.

6. Label noise. The expected loss is $-0.7 \log p - 0.3 \log(1 - p)$ and is minimized at $p = 0.7$. Each realized label defines a point mass with entropy zero, but before the outcome is known the conditional label distribution is Bernoulli(0.7), with entropy about 0.610864 nats. An observed target and the process generating successive targets are different distributions.

A way to reconstruct the whole section

Read the expectation first: it tells you who supplies the outcomes. Then read the logarithm: it tells you whose probabilities judge them. If both are P, you measure entropy. If P supplies outcomes and Q scores them, you measure cross-entropy. Their excess is forward KL. Replacing the unknown source expectation by an empirical average gives the negative log-likelihood used for learning. Always identify the outcome space, conditioning, support, and log unit before manipulating the formula.

Relation to the lecture and further reading. This section expands the supplied entropy, KL, and maximum-likelihood slides, including both transfer questions and the qualitative two-mode diagram. The new diagrams use explicitly specified distributions and reproducible calculations. For the lecture's textbook context, see Goodfellow et al. [5], Chapter 3, Section 3.13 and Figure 3.6, and Chapter 5, Section 5.5 (especially the maximum-likelihood equivalences). The finite-code qualifications and the continuous empirical-measure distinction above specify the assumptions needed to interpret the concise slide statements carefully.

1.3 Logistic Regression as a Neural Network: The Perceptron

1.3.1 One model, two ways of seeing it

Logistic regression asks for the probability of a binary outcome given measured features. A neural-network diagram exposes the computations that produce that probability. It does not, by itself, introduce a different statistical model. For $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$, and bias $b \in \mathbb{R}$,

$$z = \mathbf{w}^\mathsf{T}\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b, \tag{1.15}$$

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{1.16}$$

The score z is called the *pre-activation* or *logit*; p is the activation and the model's estimate of $P(Y = 1 \mid \mathbf{x})$. The bias is the same quantity denoted w_0 in the preceding sections. The first operation is technically *affine*, since the bias allows a nonzero output at zero input.

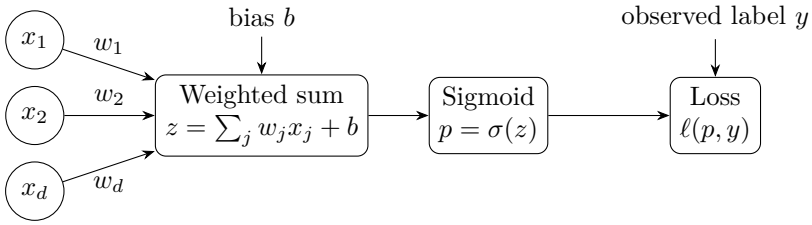


Figure 1.15: A sigmoid neuron and its training loss. The label enters the loss computation, not the prediction. Intermediate inputs are omitted when $d > 3$.

Why can z be interpreted as accumulated evidence? From the sigmoid,

$$1 - p = \frac{e^{-z}}{1 + e^{-z}}, \qquad \frac{p}{1 - p} = e^{z}, \qquad \log \frac{p}{1 - p} = z = b + \sum_j w_j x_j.$$

Thus an increase Δ in feature x_j, with all other features fixed, changes log-odds by $w_j\Delta$ and multiplies odds by $e^{w_j\Delta}$. The sigmoid translates this additive evidence into a number between zero and one. This is a modeling assumption about conditional log-odds, not a theorem that every classification problem has this structure.

What does "perceptron" mean here?

The classical perceptron uses a hard threshold, $\hat{y} = \mathbf{1}\{z \geq 0\}$, and its classical learning rule differs from logistic regression training. The lecture diagram uses the term for a sigmoid neuron. Logistic regression is exactly this sigmoid neuron trained with binary cross-entropy. Hard threshold outputs are not suitable probability estimates for this loss: a confidently wrong prediction gives infinite negative log-likelihood, and the threshold is not differentiable at zero and has zero derivative elsewhere.

At the usual probability threshold $1/2$, sigmoid monotonicity implies

$$p \geq \tfrac{1}{2} \iff z \geq 0.$$

The decision boundary remains $\mathbf{w}^\mathsf{T}\mathbf{x} + b = 0$. A nonlinear output activation therefore does not automatically create a nonlinear decision boundary in the original input features. Hidden nonlinear features, introduced in later sections, can change that geometry.

1.3.2 A complete numerical forward pass

Consider an illustrative grasp-success model using standardized contact-quality and pose-error features. Set

$$\mathbf{x} = \begin{bmatrix} 1.2 \\ 0.5 \end{bmatrix}, \qquad \mathbf{w} = \begin{bmatrix} 2 \\ -1.5 \end{bmatrix}, \qquad b = -0.4.$$

The signs encode the model's assumptions: better contact increases predicted success, while larger pose error reduces it. These are illustrative weights, not a validated robot controller. Compute

$$z = 2(1.2) - 1.5(0.5) - 0.4 = 1.25, \qquad p = \sigma(1.25) \approx 0.7773.$$

If the grasp succeeds ($y = 1$), its loss is $-\log p \approx 0.2519$ nats. If it fails ($y = 0$), the same prediction incurs $-\log(1-p) \approx 1.5019$ nats. The forward prediction is identical in both cases; the observed outcome determines how it is scored.

1.3.3 Computational complexity: count the forward-pass work

Assume a dense dataset with N examples and $d \geq 1$ features, and fixed-precision arithmetic where a scalar arithmetic operation, exponential, or logarithm has constant cost in N and d. For one example, computing z uses d multiplications, $d-1$ additions to sum the products, and one bias addition. This is $\Theta(d)$ work. The sigmoid then operates on one scalar, so it costs $\Theta(1)$. Binary cross-entropy also uses only a fixed number of scalar operations:

$$\ell(p, y) = -y \log p - (1 - y) \log(1 - p).$$

Repeating for all examples and averaging gives

$$J(\mathbf{w}, b) = \frac{1}{N} \sum_{n=1}^{N} \ell(p_n, y_n).$$

Operation	One example	Entire dataset
Weighted sum and bias	$\Theta(d)$	$\Theta(Nd)$
Sigmoid activation	$\Theta(1)$	$\Theta(N)$
Per-example cross-entropy	$\Theta(1)$	$\Theta(N)$
Sum and average losses	—	$\Theta(N)$

Consequently,

$$\boxed{\Theta(Nd + N) = \Theta(Nd), \quad \text{hence } O(Nd).}$$

The weighted sums dominate because every example must combine all d input features. If both N and d double, the dominant work increases by a factor of four. For example, $N = 10{,}000$ and $d = 100$ require 10^6 weighted feature products; doubling both requires 4×10^6.

1.3.4 The same argument as a matrix multiplication

Store examples as rows:

$$X = \begin{bmatrix} \mathbf{x}_1^\mathsf{T} \\ \vdots \\ \mathbf{x}_N^\mathsf{T} \end{bmatrix} \in \mathbb{R}^{N \times d}, \quad \mathbf{w} \in \mathbb{R}^d, \quad \mathbf{z} = X\mathbf{w} + b\mathbf{1} \in \mathbb{R}^N.$$

Matrix–vector multiplication computes N dot products, each of length d: $\Theta(Nd)$. Applying sigmoid elementwise produces $\mathbf{p} \in \mathbb{R}^N$ in $\Theta(N)$ time, followed by $\Theta(N)$ loss evaluation and reduction. Vectorization can improve execution speed through optimized numerical kernels; it does not remove the Nd arithmetic dependence.

What exactly is being counted?

Starting from inputs and weights, computing the full loss costs $O(Nd)$. If probabilities are already available, computing cross-entropy alone costs $O(N)$. This is the cost of one evaluation, not the whole optimization process. A mini-batch of B examples costs $O(Bd)$. Sparse matrix–vector multiplication can reduce the full forward-pass cost to $O(\mathrm{nnz}(X) + N)$, where $\mathrm{nnz}(X)$ counts stored nonzero entries.

1.3.5 A stable NumPy implementation

Directly taking logarithms of computed probabilities can fail when floating-point sigmoid outputs round to zero or one. Substitute $p = \sigma(z)$ into the loss instead:

$$\log p = z - \log(1 + e^z), \tag{1.17}$$

$$\log(1 - p) = -\log(1 + e^z), \tag{1.18}$$

$$\ell(z, y) = \log(1 + e^z) - yz. \tag{1.19}$$

For binary y, an equivalent expression is

$$\ell(z, y) = \log(1 + e^{(1-2y)z}).$$

Evaluate it using `np.logaddexp`, which avoids forming an overflowing exponential. This form also avoids subtracting two large nearly equal terms for a confident correct prediction.

```
import numpy as np

def binary_neuron_loss(X, y, w, b):
    # X: (N, d); y: (N,), binary; w: (d,); b: scalar
    z = X @ w + b                          # O(Nd)
    signed_logits = (1.0 - 2.0*y) * z   # O(N)
    losses = np.logaddexp(0.0, signed_logits)  # O(N)
    return losses.mean()                # O(N)

X = np.array([[1.2, 0.5]])
w = np.array([2.0, -1.5])
assert np.isclose(binary_neuron_loss(X, np.array([1.]),
                                     w, -0.4), 0.2519290813)
assert np.isclose(binary_neuron_loss(X, np.array([0.]),
                                     w, -0.4), 1.5019290813)
```

Materializing logits and losses uses $O(N)$ additional memory beyond the dataset and parameters. Processing one example at a time permits a running sum with constant extra workspace beyond the current input and weights; mini-batches trade memory against throughput.

1.3.6 Connection to learning and the next layer

The chain rule gives a particularly simple error signal:

$$\frac{\partial \ell}{\partial p} = \frac{p - y}{p(1 - p)}, \qquad \frac{\partial p}{\partial z} = p(1 - p), \qquad \frac{\partial \ell}{\partial z} = p - y.$$

These identities hold for finite logits, hence $0 < p < 1$. Since $\partial z/\partial w_j = x_j$ and $\partial z/\partial b = 1$,

$$\nabla_{\mathbf{w}} J = \frac{1}{N} X^{\mathsf{T}}(\mathbf{p} - \mathbf{y}), \qquad \frac{\partial J}{\partial b} = \frac{1}{N} \sum_n (p_n - y_n).$$

The gradient also costs $O(Nd)$. Thus a full-batch gradient step, including its forward pass, has the same asymptotic order; T such steps cost $O(TNd)$. Convergence determines the required T and is a separate question.

A single sigmoid neuron learns one affine score. Placing several neurons in a hidden layer lets the model construct several features and combine them. If there are h hidden neurons, their dense forward computation is $XW + \mathbf{1}\mathbf{b}^\mathsf{T}$ with $W \in \mathbb{R}^{d\times h}$, costing $O(Ndh)$. This explains both the computational growth and the next conceptual step toward a multilayer perceptron.

1.4 Multilayer Perceptrons

1.4.1 Why add a hidden layer?

A sigmoid neuron can turn an affine score into a probability, but its $1/2$ decision boundary is still a hyperplane in its input features. If the relationship between those features and the label cannot be separated by a hyperplane, adjusting the sigmoid neuron alone cannot solve the representation problem.

Consider two binary measurements x_1, x_2 and a label indicating that exactly one measurement is active:

$$(0,0) \mapsto 0, \qquad (1,0) \mapsto 1, \qquad (0,1) \mapsto 1, \qquad (1,1) \mapsto 0.$$

This is XOR. To classify it with $\hat{y} = \mathbf{1}\{w_1x_1 + w_2x_2 + b \geq 0\}$ would require

$$b < 0, \quad w_1 + b \geq 0, \quad w_2 + b \geq 0, \quad w_1 + w_2 + b < 0.$$

The middle inequalities imply $w_1 + w_2 + 2b \geq 0$, hence $w_1 + w_2 + b \geq -b > 0$, contradicting the last inequality. This is a limitation of the model family, not a failure of an optimizer.

One response is to invent a nonlinear feature, such as x_1x_2. Another is to let the model learn features. An MLP does this by composing affine maps and nonlinear activations. Its final sigmoid neuron is logistic regression on the last hidden representation rather than on the raw measurements.

A hidden layer changes the coordinates used for prediction

A hidden layer computes $\mathbf{h}(\mathbf{x};\theta)$, a collection of parameterized features. The output model combines those features. Joint learning adjusts both how the input is represented and how the representation is used. A hidden unit is called hidden because it is an internal computation, not an observed target variable.

1.4.2 Derive a layer from its individual neurons

We follow the lecture convention here: inputs are columns, and each neuron's incoming weight vector is a column of W. For k_1 neurons,

$$W^{(1)} = [\mathbf{w}_1^{(1)} \ \cdots \ \mathbf{w}_{k_1}^{(1)}] \in \mathbb{R}^{d\times k_1}, \qquad \mathbf{b}^{(1)} \in \mathbb{R}^{k_1}.$$

Neuron j computes

$$z_j^{(1)} = \sum_{i=1}^{d} W_{ij}^{(1)} x_i + b_j^{(1)}, \qquad h_j^{(1)} = g^{(1)}(z_j^{(1)}).$$

Stacking these k_1 equations gives

$$\underbrace{\mathbf{z}^{(1)}}_{k_1\times 1} = \underbrace{W^{(1)\mathsf{T}}}_{k_1\times d}\underbrace{\mathbf{x}}_{d\times 1} + \underbrace{\mathbf{b}^{(1)}}_{k_1\times 1}, \tag{1.20}$$

$$\underbrace{\mathbf{h}^{(1)}}_{k_1\times 1} = g^{(1)}\left(\underbrace{\mathbf{z}^{(1)}}_{k_1\times 1}\right). \tag{1.21}$$

The activation is applied elementwise and preserves shape. The weights and bias are learned parameters; $\mathbf{z}^{(1)}$ and $\mathbf{h}^{(1)}$ depend on the current example. All neurons in this layer read the same input vector. They do not receive separate examples.

A fully connected layer connects every input coordinate to every output neuron. Consequently it has dk_1 weights and k_1 biases. The number of connections counts weights, not independent input data points.

1.4.3 Compose two hidden layers and an output layer

Let the output have c coordinates. Set $\mathbf{h}^{(0)} = \mathbf{x}$. Then

$$\mathbf{z}^{(1)} = W^{(1)\mathsf{T}}\mathbf{x} + \mathbf{b}^{(1)}, \qquad \mathbf{h}^{(1)} = g^{(1)}(\mathbf{z}^{(1)}), \tag{1.22}$$

$$\mathbf{z}^{(2)} = W^{(2)\mathsf{T}}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}, \qquad \mathbf{h}^{(2)} = g^{(2)}(\mathbf{z}^{(2)}), \tag{1.23}$$

$$\mathbf{z}^{(3)} = W^{(3)\mathsf{T}}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}, \qquad \hat{\mathbf{y}} = g^{(3)}(\mathbf{z}^{(3)}). \tag{1.24}$$

Here $W^{(2)} \in \mathbb{R}^{k_1\times k_2}$ and $W^{(3)} \in \mathbb{R}^{k_2\times c}$. The output activation depends on the task: a sigmoid for one binary probability, softmax for mutually exclusive classes, or often the identity for unconstrained regression. Hidden activations need not match the output activation.

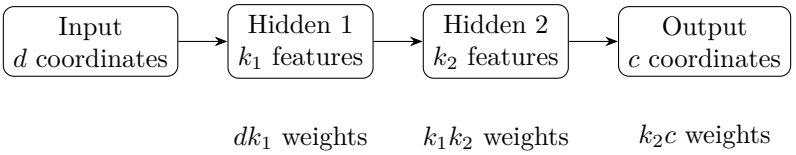


Figure 1.16: A forward pass has three affine maps when there are two hidden layers and an output layer. The input is not counted as a computed layer. Each arrow denotes a full matrix transformation followed by an activation.

For one hidden layer, the nested form is

$$\hat{\mathbf{y}} = g^{(2)}\Big(W^{(2)\mathsf{T}}g^{(1)}(W^{(1)\mathsf{T}}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}\Big).$$

The nesting records a dependency: the second layer must wait for the first layer's features. Neurons within a layer can be evaluated together, while successive layers depend on earlier results. A network with only affine maps and elementwise activations also processes different batch examples independently.

1.4.4 Why the hidden nonlinearity matters: a proof

Suppose two consecutive layers have identity activations. Write their affine maps as $A_1\mathbf{x} + \mathbf{b}_1$ and $A_2\mathbf{h} + \mathbf{b}_2$. Substitution gives

$$A_2(A_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{A_2A_1}_{A_{\text{eff}}}\mathbf{x} + \underbrace{(A_2\mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}_{\text{eff}}}.$$

Induction collapses any finite stack of affine maps into a single affine map. Adding only linear hidden layers therefore cannot solve XOR; with a final sigmoid it is still logistic regression on an affine score. Factorization can affect optimization or impose a rank constraint, but does not produce nonlinear decision geometry.

With $g(z) = \max(0, z)$ (ReLU), each unit can turn off on one side of its own hyperplane. Different inputs activate different subsets of units. Within a fixed activation pattern the map is affine; across patterns it is piecewise affine. This permits several boundary pieces in input space. ReLU is differentiable except at zero, so the slides' differentiability requirement should be read with this common almost-everywhere qualification; learning implementations choose a derivative convention at the kink.

1.4.5 A constructive nonlinear representation: solve XOR by hand

Choose two ReLU features

$$h_1 = \max(0, x_1 - x_2), \qquad h_2 = \max(0, x_2 - x_1).$$

Their sum is $|x_1 - x_2|$. At the four binary inputs:

(x_1, x_2)	h_1	h_2	$h_1 + h_2$	y
$(0, 0)$	0	0	0	0
$(1, 0)$	1	0	1	1
$(0, 1)$	0	1	1	1
$(1, 1)$	0	0	0	0

Now a single output neuron suffices:

$$z_{\text{out}} = 10(h_1 + h_2 - 0.5), \qquad p = \sigma(z_{\text{out}}).$$

The negative examples have $p = \sigma(-5) \approx 0.0067$, and the positive examples have $p = \sigma(5) \approx 0.9933$. In hidden space the boundary is the line $h_1 + h_2 = 0.5$. Its preimage in input space is $|x_1 - x_2| = 0.5$, comprising two parallel lines. The output remains linear in hidden features even though the boundary is not a single line in the original inputs.

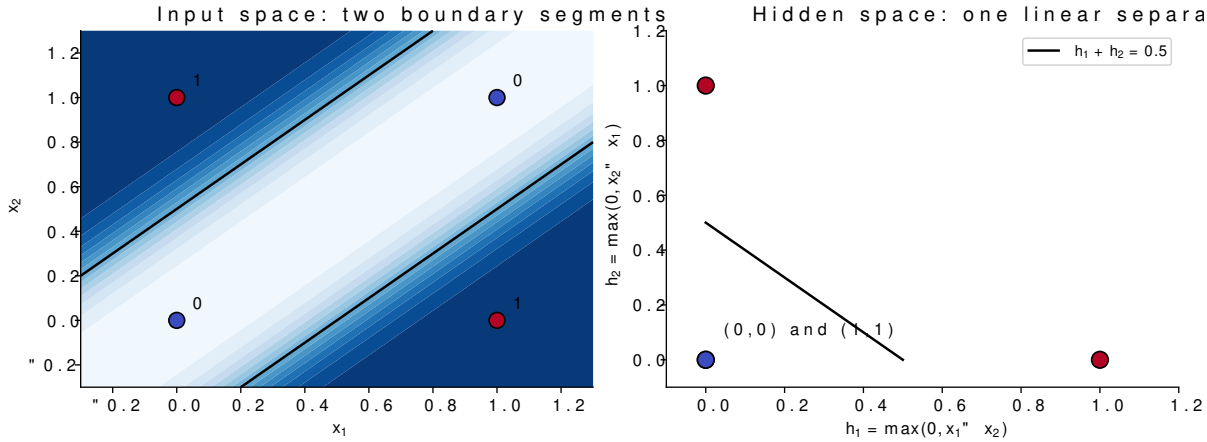


Figure 1.17: An explicit feature map makes XOR linearly separable. Colors on the left background show the output probability; labeled points are the four binary cases. Both negative examples map to the same hidden point. These features and weights are constructed by hand, demonstrating representability rather than proving a training algorithm will find them.

This toy problem has a useful interpretation for perception: two sensors may disagree in a way that predicts a fault, even when either sensor reading alone is uninformative. The learned feature needs to detect a relationship between inputs. Real sensor errors are not literally noiseless XOR, but the example isolates the need for interactions.

Calling hidden units "adaptable basis functions" emphasizes this feature-building role. For a scalar output logit,

$$z_{\text{out}} = \beta_0 + \sum_j \beta_j \phi_j(\mathbf{x}; \theta),$$

where ϕ_j are hidden activations. Unlike fixed polynomial features, their parameters can be learned. They need not form a basis in the strict linear-algebra sense: they may be redundant, dependent, or inactive. Also, learning the hidden parameters jointly generally removes logistic regression's convexity guarantee.

1.4.6 Time complexity for the lecture's two-hidden-layer question

Assume dense matrices, ordinary matrix multiplication, fixed-precision scalar arithmetic, and elementwise activations with constant work per coordinate. Standard softmax also takes linear work in its number of output coordinates. Keep c explicit because the question does not specify output width.

For one example, layer 1 multiplies a $k_1 \times d$ matrix by a $d \times 1$ vector. The hint gives $O(k_1 d \cdot 1)$. Equivalently, k_1 neurons each compute a dot product with d terms. Apply the same argument at the next layers:

Layer	Matrix product	Product cost	Bias/activation cost
First hidden	$(k_1 \times d)(d \times 1)$	$O(dk_1)$	$O(k_1)$
Second hidden	$(k_2 \times k_1)(k_1 \times 1)$	$O(k_1 k_2)$	$O(k_2)$
Output	$(c \times k_2)(k_2 \times 1)$	$O(k_2 c)$	$O(c)$

Add sequential work; do not multiply the layer costs together:

$$T = O(dk_1 + k_1 k_2 + k_2 c + k_1 + k_2 + c) = \boxed{O(dk_1 + k_1 k_2 + k_2 c)}.$$

The last simplification assumes positive integer widths. For a scalar binary output ($c = 1$), a useful explicit answer is

$$\boxed{T = O(dk_1 + k_1 k_2 + k_2) = O(dk_1 + k_1 k_2).}$$

If the intended forward pass stops at the second hidden representation, omit the output-layer term. Writing the architecture next to the answer prevents ambiguity. These counts describe arithmetic work, not exact GPU latency or a promise of ideal parallel speedup.

1.4.7 Memory complexity: count what must exist

Time counts operations; memory counts stored numbers. A multiplication can perform many operations without storing every intermediate scalar product. In the lecture hint, $(n \times m)(m \times k)$ stores two operands and a result, giving $O(nm + mk + nk)$, not $O(nmk)$ storage.

Keep all network parameters resident. The exact number of parameters is

$$P = \underbrace{dk_1 + k_1 k_2 + k_2 c}_{\text{weights}} + \underbrace{k_1 + k_2 + c}_{\text{biases}}.$$

The input has d numbers. If all preactivations and activations are retained separately, they use $2(k_1 + k_2 + c)$ numbers, including the final output. Thus one explicit storage model is

$$S_{\text{cache}} = P + d + 2(k_1 + k_2 + c).$$

An implementation may alias some buffers or save only what a particular activation needs, changing constants. The inclusive big-O answer is

$$\boxed{S = O(dk_1 + k_1k_2 + k_2c + d + k_1 + k_2 + c).}$$

For $c = 1$ this becomes

$$\boxed{S = O(dk_1 + k_1k_2 + d + k_1 + k_2)}.$$

For positive widths, the product terms dominate these vector terms, so $O(dk_1 + k_1k_2)$ is also valid. Showing the unsimplified expression demonstrates that input, output, biases, and intermediate vectors were included as requested.

Do not call activation memory the total memory

$O(d + k_1 + k_2 + c)$ accounts for input and activations only. It omits the matrices needed to execute the forward pass. Conversely, parameters are stored once, not once per example or once every time they are used. Gradients and optimizer state are separate training allocations and are not required for a forward-only inference calculation.

For inference, once layer 2 has consumed $\mathbf{h}^{(1)}$, that activation can be released if no later operation needs it. With only two hidden layers, caching versus buffer reuse does not change the order of the vector terms. With variable depth and large batches, the distinction becomes significant; Section 1.5 makes it explicit.

1.4.8 A practical sizing example

Suppose a grasp-success predictor takes 12 measured features, such as geometric clearance, contact estimates, and pose uncertainty summaries. Choose the illustrative architecture

$$12 \longrightarrow 32 \longrightarrow 16 \longrightarrow 1.$$

It uses $12(32) + 32(16) + 16(1) = 912$ weight multiplications per example, plus the corresponding additions and activations. It has $32 + 16 + 1 = 49$ biases, hence $P = 961$ parameters. Retaining input and all Z/H vectors uses

$$961 + 12 + 2(32 + 16 + 1) = 1071$$

scalars. At four bytes per scalar this model accounts for 4,284 bytes. This is an array-element count, not actual process memory: array objects, kernel workspace, allocator overhead, and library code are excluded.

For 256 candidate grasps evaluated together, there are $256(912) = 233{,}472$ weight products, while there are still only 961 parameters. The network is reused across candidates. Its predictions would require training and validation before being meaningful; the example here is for architecture and resource accounting.

1.4.9 Exercises and worked reasoning

1. Why does depth without nonlinearity not solve XOR? Compose the affine maps including their biases. The result is one affine map, whose threshold is a single hyperplane; the four XOR inequalities remain contradictory. A final sigmoid changes probabilities but not that limitation.

2. What if both hidden widths are doubled? For fixed d, c, the first-layer products double, the hidden-to-hidden products quadruple, and the output-layer products double. The total factor depends on which term dominates. It is not always exactly four.

3. Is one hidden neuron enough for XOR with ReLU and a sigmoid output? No. Its logit is $a\max(0, \mathbf{w}^\mathsf{T}\mathbf{x} + b) + c$, a monotone function of the one affine score (possibly decreasing if $a < 0$). Thresholding it gives a half-space, all inputs, or no inputs, with possible boundary ties. XOR requires more than one such half-space decision. The two-unit construction above shows sufficiency.

4. Can a large activation value be interpreted as a probability? Not in general. ReLU activations can exceed one, can be rescaled with compensating downstream weights, and need not represent a named semantic concept. The probabilistic interpretation comes from the chosen output distribution and loss, not merely from being a neuron.

1.5 Matrix Dimensions

1.5.1 A matrix is an organized collection of scalar equations

Dimension checking is a way to keep track of meaning: which index identifies an example, which identifies a feature, and which identifies an output neuron? Rather than remembering a transpose by appearance, derive the expression from one output coordinate.

Let d be the input width and q the number of neurons. In the lecture convention, neuron j has a column weight vector $\mathbf{w}_j \in \mathbb{R}^{d\times 1}$. Its score is

$$z_j = \mathbf{w}_j^\mathsf{T}\mathbf{x} + b_j = \sum_{i=1}^{d} w_{ij}x_i + b_j.$$

Here i is summed out: it indexes the features contributing to one neuron. The index j survives: it identifies the resulting score. To compute all scores, collect weight vectors into columns,

$$W_s = [\mathbf{w}_1 \ \cdots \ \mathbf{w}_q] \in \mathbb{R}^{d\times q}.$$

Then

$$\underbrace{\mathbf{z}}_{q\times 1} = \underbrace{W_s^\mathsf{T}}_{q\times d}\underbrace{\mathbf{x}}_{d\times 1} + \underbrace{\mathbf{b}}_{q\times 1}, \qquad \underbrace{\mathbf{h}}_{q\times 1} = g(\underbrace{\mathbf{z}}_{q\times 1}).$$

The subscript s marks the slides' weight storage, to keep it distinct from code notation below. Matrix multiplication performs all the dot products at once; it does not define a different operation from the scalar equations.

1.5.2 Batching in the column-vector convention

Place B examples in columns:

$$X_c = [\mathbf{x}^{(1)} \ \cdots \ \mathbf{x}^{(B)}] \in \mathbb{R}^{d\times B}.$$

The batch preactivation matrix has one neuron per row and one example per column:

$$\underbrace{Z_c}_{q\times B} = \underbrace{W_s^\mathsf{T}}_{q\times d}\underbrace{X_c}_{d\times B} + \underbrace{\mathbf{b}\mathbf{1}_B^\mathsf{T}}_{q\times B}, \qquad H_c = g(Z_c) \in \mathbb{R}^{q\times B}. \tag{1.25}$$

Entry (j, n) is

$$(Z_c)_{jn} = \sum_i (W_s)_{ij}(X_c)_{in} + b_j.$$

The same b_j is added for every example n. Formally, a $q \times 1$ matrix cannot be added to a $q \times B$ matrix by ordinary matrix addition. The slides' shorthand "$+\mathbf{b}$" means broadcasting, made explicit here by $\mathbf{b}\mathbf{1}_B^\mathsf{T}$. The repeated columns are not different trainable biases.

1.5.3 Transpose the whole equation to get row batches

Code often places examples in rows. Define

$$X_r = X_c^\mathsf{T} \in \mathbb{R}^{B\times d}, \qquad Z_r = Z_c^\mathsf{T} \in \mathbb{R}^{B\times q}.$$

Using $(AB)^\mathsf{T} = B^\mathsf{T}A^\mathsf{T}$ gives

$$\begin{aligned} Z_r &= \left(W_s^\mathsf{T} X_c + \mathbf{b}\mathbf{1}_B^\mathsf{T}\right)^\mathsf{T} && (1.26)\\ &= X_c^\mathsf{T} W_s + \mathbf{1}_B \mathbf{b}^\mathsf{T} && (1.27)\\ &= X_r W_s + \mathbf{1}_B \mathbf{b}^\mathsf{T}. && (1.28) \end{aligned}$$

Since an elementwise activation commutes with transposition, $H_r = g(Z_r) = H_c^\mathsf{T}$. Nothing about the model has changed.

PyTorch's dense-layer parameter storage uses one neuron's weights per row. Write that matrix as

$$W_p = W_s^\mathsf{T} \in \mathbb{R}^{q\times d}.$$

Consequently, the familiar code expression is

$$\boxed{\underbrace{Z_r}_{B\times q} = \underbrace{X_r}_{B\times d}\underbrace{W_p^\mathsf{T}}_{d\times q} + \underbrace{\mathbf{1}_B \mathbf{b}^\mathsf{T}}_{B\times q}.} \tag{1.29}$$

This is the formula $Z = XW^\mathsf{T} + B_{\text{bias}}$ in the lecture's coding notation. We use B_{bias} for the expanded bias matrix so that B alone can denote batch size.

Object	Lecture columns	PyTorch-style rows
Batch input	$X_c : d \times B$	$X_r : B \times d$
Stored weights	$W_s : d \times q$	$W_p : q \times d$
Bias as a mathematical column	$\mathbf{b} : q \times 1$	$\mathbf{b} : q \times 1$
Preactivations	$Z_c : q \times B$	$Z_r : B \times q$
Affine computation	$W_s^\mathsf{T} X_c + \mathbf{b}\mathbf{1}_B^\mathsf{T}$	$X_r W_p^\mathsf{T} + \mathbf{1}_B \mathbf{b}^\mathsf{T}$

In NumPy and PyTorch, the bias is typically a one-dimensional array of shape `(q,)`. It broadcasts over batch rows. A one-dimensional array has no separate row or column orientation; explicitly reshape it when doing column-batch arithmetic.

Two independent layout choices

Changing examples from columns to rows does not logically force a particular weight layout. One could use row batches with $W_s \in \mathbb{R}^{d\times q}$ and write $X_r W_s$, or with $W_p \in \mathbb{R}^{q\times d}$ and write $X_r W_p^\mathsf{T}$. Both are correct. The letters alone do not specify the convention: the index meanings and dimensions do. PyTorch is not a different mathematics.

The official `torch.nn.Linear` documentation stores weights with output features in rows and input features in columns, using transformation $xA^\mathsf{T} + b$. It also permits leading dimensions beyond a single batch dimension: the last dimension is the feature dimension. Thus "PyTorch inputs are row vectors" is a useful two-dimensional batch convention, not a restriction on every supported tensor.

1.5.4 Work through every entry of a concrete batch

Take two examples, three input features, and two neurons:

$$X_r = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 0 & 2 \end{bmatrix}, \quad W_p = \begin{bmatrix} 1 & -1 & 2 \\ 0 & 3 & -1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}.$$

The product and broadcast bias are

$$X_r W_p^{\mathsf{T}} = \begin{bmatrix} 5 & 3 \\ 3 & -2 \end{bmatrix}, \qquad \mathbf{1}_2 \mathbf{b}^{\mathsf{T}} = \begin{bmatrix} 0.5 & -0.5 \\ 0.5 & -0.5 \end{bmatrix}.$$

For example, the first example's second neuron gives $1(0) + 2(3) + 3(-1) - 0.5 = 2.5$. Every other cell follows the same rule:

$$Z_r = \begin{bmatrix} 5.5 & 2.5 \\ 3.5 & -2.5 \end{bmatrix}, \qquad H_r = \mathrm{ReLU}(Z_r) = \begin{bmatrix} 5.5 & 2.5 \\ 3.5 & 0 \end{bmatrix}.$$

Row 2 always belongs to example 2; column 2 always belongs to neuron 2. This semantic identity is as important as whether an array multiplication happens to run.

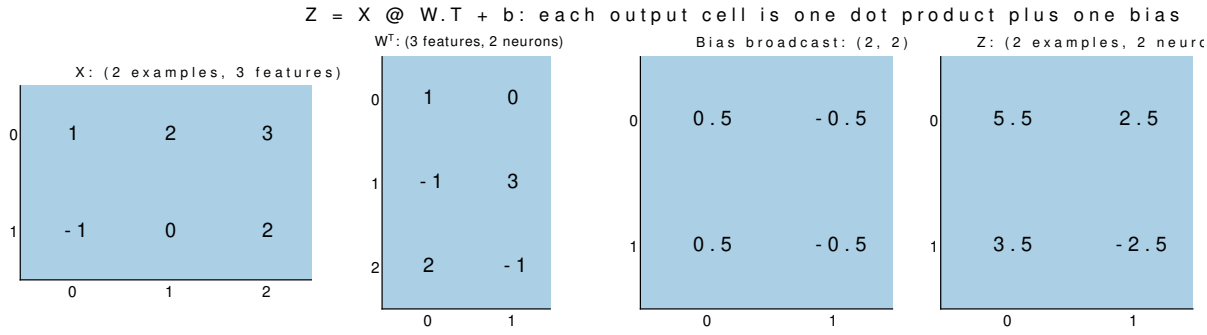


Figure 1.18: The batch computation with actual numbers. Output entry (n, j) combines row n of X with column j of W^{T}, then adds bias b_j.

1.5.5 The lecture's batched-perceptron repetition question

For one sigmoid output neuron and B examples of dimension d, the two requested formulas in row-batch notation are

$$\underbrace{Z}_{B\times 1} = \underbrace{X}_{B\times d}\,\underbrace{W^{\mathsf{T}}}_{d\times 1} + \underbrace{\mathbf{1}_B}_{B\times 1}\,\underbrace{b}_{\text{scalar}}, \qquad W \in \mathbb{R}^{1\times d}, \tag{1.30}$$

$$\underbrace{H}_{B\times 1} = \sigma(\underbrace{Z}_{B\times 1}). \tag{1.31}$$

Equivalently, with examples in columns,

$$\underbrace{Z_c}_{1\times B} = \underbrace{\mathbf{w}^{\mathsf{T}}}_{1\times d}\,\underbrace{X_c}_{d\times B} + \underbrace{b\mathbf{1}_B^{\mathsf{T}}}_{1\times B}, \qquad H_c = \sigma(Z_c) \in \mathbb{R}^{1\times B}.$$

The B probabilities are predictions for different examples, not probabilities of B mutually exclusive classes; there is no reason for them to sum to one.

1.5.6 General layer notation and a dimension-checking procedure

Let there be L computed layers with widths $n_1, \dots, n_L$, and set $n_0 = d$. In row-batch notation,

$$H^{(0)} = X \in \mathbb{R}^{B\times d}, \qquad W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \qquad \mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}.$$

For $\ell = 1, \dots, L$,

$$\underbrace{Z^{(\ell)}}_{B\times n_\ell} = \underbrace{H^{(\ell-1)}}_{B\times n_{\ell-1}} \underbrace{W^{(\ell)\mathsf{T}}}_{n_{\ell-1}\times n_\ell} + \underbrace{\mathbf{1}_B \mathbf{b}^{(\ell)\mathsf{T}}}_{B\times n_\ell}, \tag{1.32}$$

$$\underbrace{H^{(\ell)}}_{B\times n_\ell} = g^{(\ell)}(Z^{(\ell)}). \tag{1.33}$$

To reconstruct this without memory: first decide what an output entry means, (n, j). Then write its feature sum $Z^{(\ell)}_{nj} = \sum_i H^{(\ell-1)}_{ni} W^{(\ell)}_{ji} + b^{(\ell)}_j$. The shared feature index i must be contracted; the example and neuron indices survive. This forces the product and its dimensions.

A useful invariant is that the batch dimension stays B through an ordinary MLP. It is the feature width that changes. A square matrix can hide an incorrect transpose; use nonsquare test cases and check values, not shapes alone.

1.5.7 General time complexity, then the equal-width answer

For a batch, layer ℓ multiplies $(B \times n_{\ell-1})(n_{\ell-1} \times n_\ell)$. The ordinary algorithm computes Bn_ℓ outputs, each using $n_{\ell-1}$ products, so its work is $O(Bn_{\ell-1}n_\ell)$. Bias addition and an elementwise activation each cost $O(Bn_\ell)$. Summing across layers gives

$$T = O\left(B\sum_{\ell=1}^{L} n_{\ell-1}n_\ell + B\sum_{\ell=1}^{L} n_\ell\right) = O\left(B\sum_{\ell=1}^{L} n_{\ell-1}n_\ell\right).$$

The simplification assumes positive widths and ordinary coordinatewise activations (or linear-cost softmax). It does not cover arbitrary operations such as self-attention.

For the lecture question, define M to be the number of *computed layers*, all of width k, including the output and excluding the input. Then $n_0 = d$ and $n_1 = \cdots = n_M = k$. The first layer costs $O(Bdk)$ and there are $M - 1$ subsequent k-to-k maps:

$$\boxed{T = O(Bdk + B(M-1)k^2).}$$

For $M \geq 2$ it is common to write $O(Bdk + BMk^2)$. Keeping $M - 1$ makes the $M = 1$ case transparent: it is one layer, so the time is $O(Bdk)$, with no hidden k^2 product.

If "M layers" means M hidden layers instead

State the convention. With $M \geq 1$ hidden layers of width k and an additional output of width c,

$$T = O(B[dk + (M-1)k^2 + kc]).$$

For binary classification $c = 1$, the last term is Bk. For $c = k$, there are M width-k transitions after the input map. Neither output width nor whether it is counted should be silently assumed.

1.5.8 General memory complexity: parameters, data, and lifetimes

Assume parameters for every layer are stored in memory. Count scalar elements; multiply by bytes per element for a storage estimate. The parameter count is

$$P = \sum_{\ell=1}^{L} (n_{\ell-1} n_\ell + n_\ell).$$

It has no factor of B: all examples share the same weights and biases. The input takes Bd elements. The final output takes Bn_L elements. The storage of intermediate activations depends on their lifetimes.

Case A: retain the layer activations. A simple forward cache stores $Z^{(\ell)}$ and $H^{(\ell)}$ for all layers, for later inspection or differentiation. It requires

$$S_{\text{cache}} = P + Bd + 2B \sum_{\ell=1}^{L} n_\ell = O\left(P + Bd + B \sum_{\ell=1}^{L} n_\ell \right).$$

If only one of Z or H is needed, the factor two may disappear. This is still forward-pass storage, not a count of gradients and optimizer buffers. For M computed layers of width k,

$$P = dk + (M-1)k^2 + Mk,$$

so the explicit inclusive answer is

$$\boxed{S_{\text{cache}} = O(dk + (M-1)k^2 + Mk + Bd + BMk).} \tag{1.34}$$

For $B \geq 1$, the Mk bias term is bounded by BMk, yielding $O(dk + (M-1)k^2 + Bd + BMk)$. This is the standard answer when "memory of the forward pass" includes retaining all layer results. It follows the hint by counting parameters, inputs, and outputs rather than multiplying their sizes.

Case B: forward-only inference with buffer reuse. An ordinary sequential MLP only needs the previous activation to compute the next one. Once consumed, earlier buffers can be discarded. A conservative accounting keeps the original input resident, plus current layer input, preactivation, and output. Therefore

$$S_{\text{infer}} = O\left(P + Bd + B \max_{1 \leq \ell \leq L} (n_{\ell-1} + n_\ell) \right).$$

Any double counting of the original input changes only a constant. For equal width,

$$\boxed{S_{\text{infer}} = O(dk + (M-1)k^2 + Mk + Bd + Bk).} \tag{1.35}$$

The parameters still grow with depth, but the live activation buffers need not. The two answers describe different implementations; a question that does not specify retention is best answered by stating the assumption and giving the distinction.

Do not sum each layer's full memory formula mechanically and count the same weights or boundary activation twice as distinct objects. Count distinct stored arrays, then decide which coexist. Likewise, multiplying a $B \times d$ batch by a $d \times k$ matrix does not require a $B \times d \times k$ tensor of all scalar products; standard matrix multiplication accumulates into the $B \times k$ output.

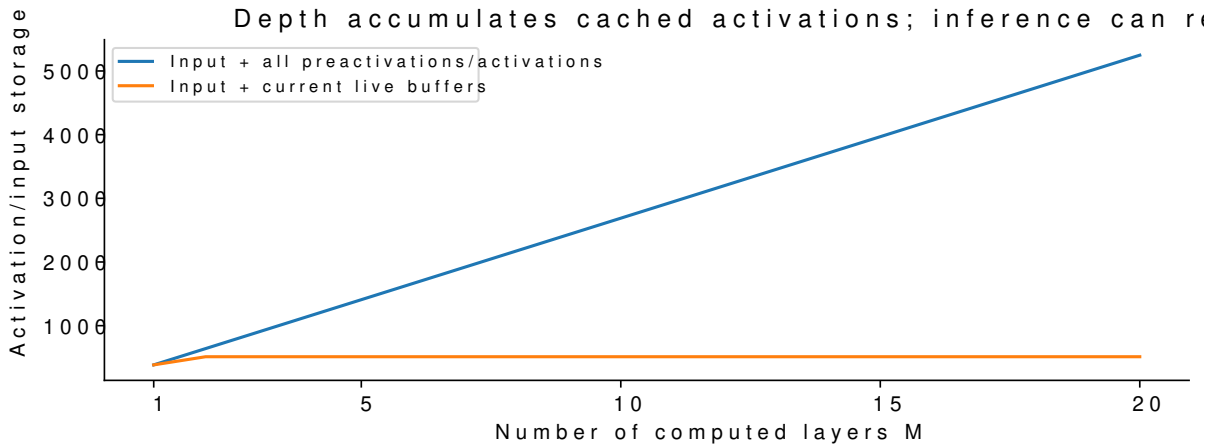


Figure 1.19: Storage for input and activations at $B = 512$, $d = k = 64$. The cached implementation stores both Z and H for every layer. The inference model retains the input and only live layer buffers. Parameters are excluded from this plot to isolate activation lifetime; they must be added for total memory. These are calculated array sizes, not measured runtime allocations.

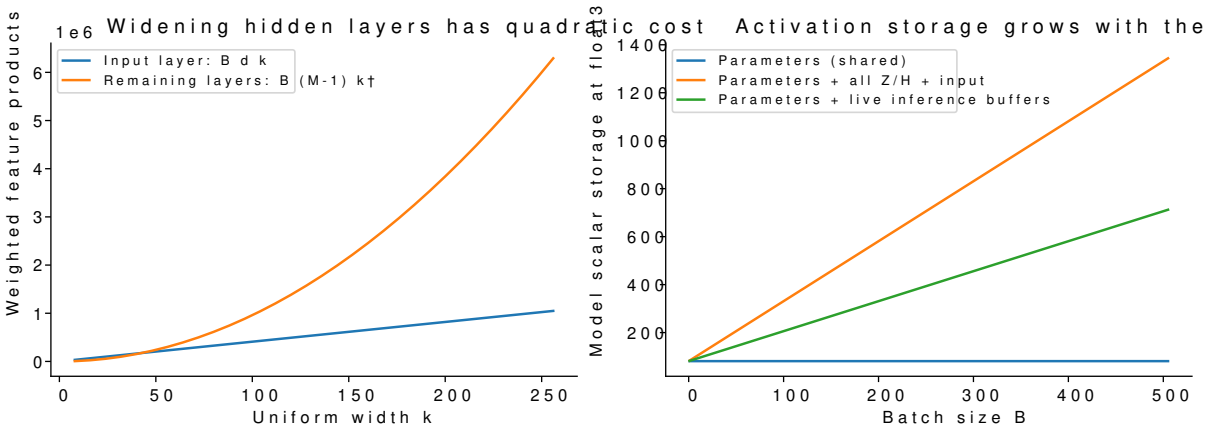


Figure 1.20: Left: ordinary multiplication counts for four width-k computed layers, $B = 32$, $d = 128$. Right: a four-layer model with $d = 128$, $k = 64$ and variable batch size. Weights are reused across examples, while activation/input storage grows with B. Curves are analytic counts, not timing measurements.

What happens to the expanded bias? Mathematically $\mathbf{1}_B\mathbf{b}^\top$ has Bk entries, but broadcasting need not allocate a separate dense repeated copy. There remain only k learned bias parameters. Explicitly materializing the repeated matrix adds a Bk temporary; the activation order is unchanged, but practical memory use increases.

1.5.9 A numerical batch budget and what it teaches

Return to the $12 \to 32 \to 16 \to 1$ grasp model. At $B = 256$, the input has 3,072 entries and all Z/H arrays have $2(256)(49) = 25,088$ entries. Including 961 parameters yields

$$S_{\text{cache}} = 29{,}121 \text{ scalars} = 116{,}484 \text{ bytes at float32.}$$

One conservative inference implementation, keeping the input plus separate live input/Z/H buffers, peaks at 16,384 live intermediate scalars, hence

$$S_{\text{infer}} = 961 + 3{,}072 + 16{,}384 = 20{,}417 \text{ scalars.}$$

The live intermediate maximum is attained by $2B(32)$ in the first layer or $B(32 + 2 \cdot 16)$ in the second. In-place activation or fused kernels can reduce this number. These precise counts belong to the stated storage model, whereas the big-O expressions describe scaling robustly across such constant-factor changes.

Larger batches increase arithmetic and activation storage but not parameter count. Wider hidden layers can increase matrix work quadratically because both ends of a dense connection grow. More layers increase weight storage and computation approximately linearly at fixed width; retained activations also accumulate with depth. These observations help reason about embedded inference and GPU batch sizing before running an experiment.

1.5.10 NumPy implementation: translate the equations literally

```
import numpy as np

X = np.array([[1., 2., 3.], [-1., 0., 2.]])  # (B=2, d=3)
W = np.array([[1., -1., 2.], [0., 3., -1.]]) # (q=2, d=3)
b = np.array([0.5, -0.5])                     # (q=2,)
Z = X @ W.T + b                              # (B=2, q=2)
H = np.maximum(Z, 0.0)                       # (B=2, q=2)
np.testing.assert_allclose(Z, [[5.5, 2.5], [3.5, -2.5]])

# The same calculation in the slide convention:
Xc = X.T                                     # (d=3, B=2)
Ws = W.T                                     # (d=3, q=2)
Zc = Ws.T @ Xc + b[:, None]                  # (q=2, B=2)
np.testing.assert_allclose(Z, Zc.T)
```

For a network, each layer consumes the previous layer's output:

```
H = X
for ell, (W, b) in enumerate(zip(weights, biases)):
    Z = H @ W.T + b
    if ell < len(weights) - 1:
        H = np.maximum(Z, 0.0)               # hidden ReLU
    else:
        H = np.exp(-np.logaddexp(0.0, -Z)) # stable sigmoid
```

This code assumes a binary or independent-sigmoid output, not a mutually exclusive multiclass softmax. No labels are needed to produce predictions. Labels enter when computing a loss.

Beware of `X * W`: it is elementwise multiplication, not a collection of dot products. Also, `np.zeros(d).T` still has shape `(d,)`; transpose does not turn a one-dimensional array into a column. Use `x[:, None]` for `(d,1)` or `x[None, :]` for `(1,d)`. For loss calculations, keep both targets and predictions consistently `(B,)` or consistently `(B,1)`: subtracting arrays of these two different shapes can silently create a `(B,B)` result.

1.5.11 Exercises with worked answers

1. Recover the matrix dimensions. A batch has 8 examples with 6 features and a layer has 5 neurons. In row notation, $X : 8 \times 6$, $W : 5 \times 6$, $b : 5$, $Z, H : 8 \times 5$. In column notation, $X_c : 6 \times 8$, $W_s : 6 \times 5$, and $Z_c, H_c : 5 \times 8$. Both compute 240 scalar weight products and use 35 trainable parameters.

2. Check the boundary cases of the complexity answer. At $M = 1$, $T = O(Bdk)$, and inclusive memory is $O(dk + k + Bd + Bk)$. At $B = 1$, the general batch formulas reduce to single-example formulas. If $d = k$ and $M \geq 1$, time simplifies to $O(BMk^2)$. Checking these cases can reveal an incorrectly counted input layer or an extra hidden layer.

3. Does doubling batch size double total memory? Write $S = P + A(B)$ where P is fixed parameter storage and $A(B)$ is linear in B under a fixed buffering policy. Then $S(2B) = P + 2A(B)$, which is generally less than $2S(B)$. It approaches twice as much only when batch-dependent storage dominates.

4. Why not multiply the parameter count by B? A parameter defines the same function for every example. Batching evaluates that one function repeatedly. For an ordinary MLP, permuting the batch rows permutes output rows in exactly the same way; it does not change the parameters or mix information between examples.

5. Why can a shape-correct expression still be wrong? If $B = d = q$, several incorrect transposes and bias placements produce compatible square shapes. They can combine examples instead of features or associate biases with examples. Evaluate a small nonsquare example against scalar loops to test the intended index semantics. The companion script checks this equivalence, verifies row/column conversion, and compares batched predictions with one-example-at-a-time predictions.

Reproducible companion. Run `python deep_learning/code/mlp_1_4_1_5.py` from the repository root. It verifies the numerical examples and parameter/cache counts, demonstrates affine collapse and XOR, and regenerates all four figures. Its memory plots count array elements under explicit buffering assumptions and do not claim measured CPU/GPU memory or latency.

1.6 Other Activation Functions and Loss Functions

1.6.1 Can neurons really add up to draw a curve?

Yes, with an important distinction between *summing within a layer* and *composing across layers*. For one scalar input x, one hidden layer of m units, and a linear scalar output,

$$h_j(x) = g(a_j x + b_j), \qquad \boxed{f_\theta(x) = c + \sum_{j=1}^{m} v_j g(a_j x + b_j).} \tag{1.36}$$

This is a function of one variable drawn in a two-dimensional (x, y) plot. Each hidden unit supplies a transformed curve; the output weights add those curves, possibly with negative signs. The parameters control four different operations:

- a_j: horizontal scaling and, when negative, reflection;
- b_j: translation of the preactivation, placing its zero at $x = -b_j/a_j$ when $a_j \neq 0$;
- v_j: vertical scaling or reflection of that unit's contribution;
- c: an overall vertical offset.

The quantity $v_j g(a_j x + b_j)$ is the *contribution* to the output; h_j alone is the hidden activation. Distinguishing them matters: positive ReLU outputs can still combine into a decreasing or negative function through negative v_j.

For binary classification, the sum in (1.36) usually gives the *logit*, and a final sigmoid converts it to a probability. For several hidden layers, later units apply g to weighted combinations of earlier features. The model is then a nested composition, not simply a list of unchanged elementary curves added at the end.

The useful curve-building picture

One hidden layer with a linear output is a learned weighted sum of shifted/scaled activation curves. Deep networks additionally compose these learned transformations. Training changes the parameters to reduce a data-dependent objective; it is not tracing a desired graph supplied everywhere in advance.

1.6.2 Build a piecewise-linear curve with exact weights

Let $r(x) = \max(0, x)$ and work on $[0, 4]$. Define

$$f(x) = r(x) - 2r(x-1) + 3r(x-2) - 2r(x-3).$$

This is a $1 \to 4 \to 1$ ReLU network with

$$\mathbf{a} = (1, 1, 1, 1), \quad \mathbf{b} = (0, -1, -2, -3), \quad \mathbf{v} = (1, -2, 3, -2), \quad c = 0.$$

Every added hinge changes the slope when its threshold is crossed:

Interval	Active contributions	Resulting slope
$(0, 1)$	first	1
$(1, 2)$	first two	$1 - 2 = -1$
$(2, 3)$	first three	$1 - 2 + 3 = 2$
$(3, 4)$	all four	$1 - 2 + 3 - 2 = 0$

The curve passes through $(0, 0), (1, 1), (2, 0), (3, 2), (4, 2)$. The network's output weights encode *changes of slope*, rather than directly specifying the heights of those points.

More generally, if a continuous piecewise-linear function has knots $t_1 < \cdots < t_m$ and successive slopes $s_0, s_1, \ldots, s_m$, then

$$f(x) = \alpha + s_0 x + \sum_{j=1}^{m} (s_j - s_{j-1}) r(x - t_j). \tag{1.37}$$

To prove it, differentiate away from the knots. Before t_1, the slope is s_0. After knot t_j, the new hinge adds $s_j - s_{j-1}$, so the sum telescopes to s_j. Choose α to match one value; continuity

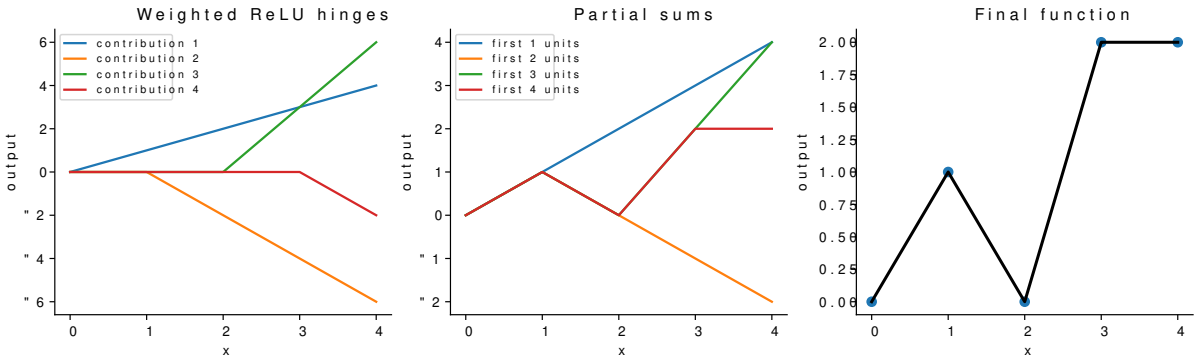


Figure 1.21: Exact, deliberately chosen weights. Left: individual weighted hinges. Middle: their partial sums. Right: the final curve. Adding hidden units adds locations where the slope can change. These are constructed weights, distinct from the trained Iris weights below.

and equal slopes imply the functions agree everywhere. The direct linear term can itself be implemented with $x = r(x) - r(-x)$, so this expression is realizable by an ordinary one-hidden-layer ReLU network without a skip connection.

For a general term $v_j r(a_j x + b_j)$, the knot is $-b_j/a_j$. A positive a_j activates to its right, a negative one to its left. The slope jump from left to right is $v_j|a_j|$. Zero input weights produce constants; a knot outside the observed interval may make a unit affine or inactive throughout that interval.

Sigmoids provide another construction. The difference

$$\sigma(s(x-u)) - \sigma(s(x-v)), \qquad u < v,\ s > 0,$$

is approximately a window: nearly zero outside $[u, v]$, nearly one inside, with smooth transitions. Weighted windows form smooth bumps and plateaus. Larger s sharpens transitions but also narrows the region with substantial sigmoid derivative. This is an intuition for approximation, not a proof of uniform approximation of a discontinuous step at its jump.

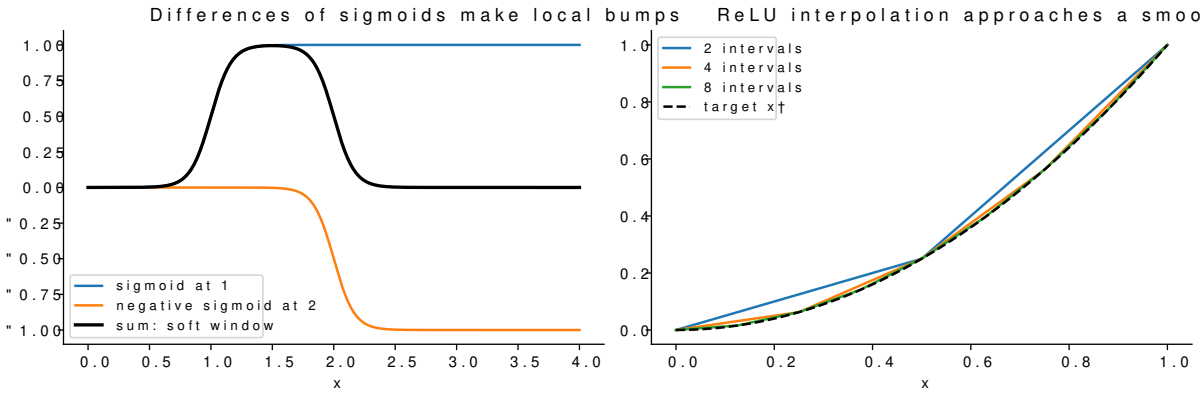


Figure 1.22: Left: one positive and one negative sigmoid combine into a soft local window. Right: piecewise-linear interpolation of x^2 improves as the knots become finer; every interpolant has the hinge representation in (1.37). Section 1.7 turns this observation into a one-dimensional approximation proof.

1.6.3 What should we inspect in an activation function?

An activation has two simultaneous roles: it shapes the class of functions that the network can express, and its derivative affects how parameters can be learned. For one unit $h = g(z)$ and

$z = ax + b$, the chain rule gives

$$\frac{\partial \ell}{\partial a} = \frac{\partial \ell}{\partial h}\, g'(z)x, \qquad \frac{\partial \ell}{\partial b} = \frac{\partial \ell}{\partial h}\, g'(z).$$

A locally flat activation can therefore block a parameter update through that path. Across many layers, Jacobians multiply; small activation derivatives can contribute to vanishing gradients, though weight matrices and the whole computation also matter. Inspect range, saturation, smoothness, sign, and derivative rather than treating an activation's name as a performance guarantee.

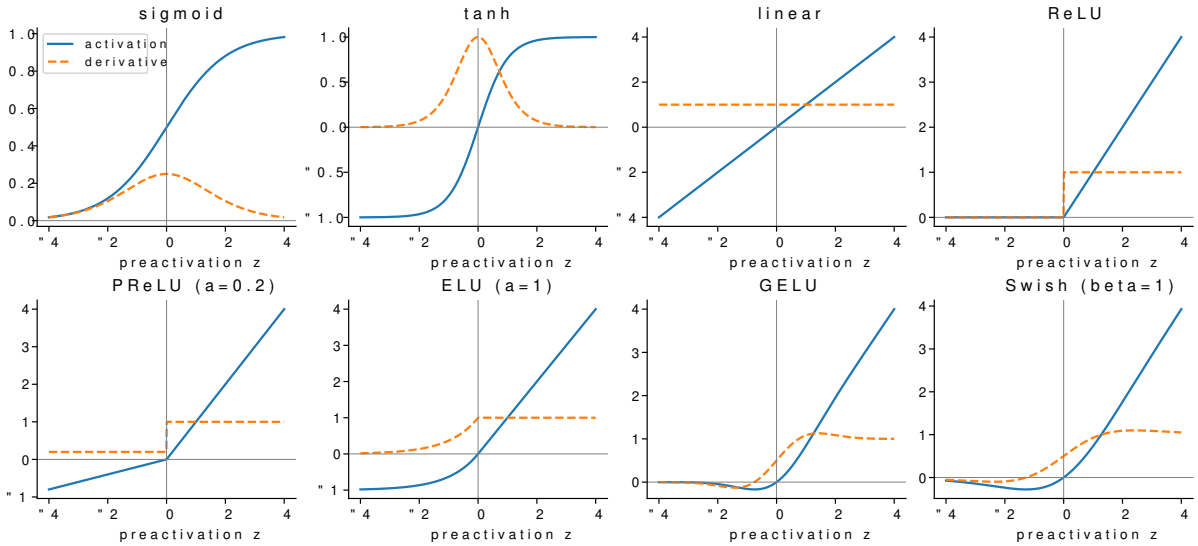


Figure 1.23: Eight activation functions (solid) and their derivatives (dashed). PReLU uses slope $a = 0.2$, ELU uses $\alpha = 1$, and Swish uses $\beta = 1$. Derivative conventions at ReLU/PReLU kinks are implementation choices. Hidden-unit values need not be probabilities.

1.6.4 Sigmoid and hyperbolic tangent

The logistic sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z)).$$

It maps $\mathbb{R}$ into $(0, 1)$, is strictly increasing, and saturates near both endpoints. Its derivative is at most $1/4$, attained at zero. These properties make it a natural Bernoulli probability map, but saturation can impede learning in hidden layers.

The hyperbolic tangent is

$$\tanh z = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} = 2\sigma(2z) - 1.$$

Using the sigmoid derivative and chain rule,

$$\frac{d}{dz} \tanh z = 4\sigma(2z)(1 - \sigma(2z)) = 1 - \tanh^2 z.$$

Its range is $(-1, 1)$, it is odd and centered at zero, and its derivative is one at zero but vanishes in either saturated tail. It is not itself a probability because it can be negative. If $t = \tanh z$, then

$$p = \frac{t + 1}{2} = \sigma(2z)$$

is a valid binary probability. Hence a tanh binary output requires a label/probability convention; inserting a negative tanh value directly into binary cross-entropy is invalid. Tanh scores thresholded at zero can classify labels coded $-1, +1$, while a likelihood must specify the corresponding probabilities.

1.6.5 Linear and rectified linear activations

The linear activation $g(z) = z$ has range $\mathbb{R}$ and derivative one. It is useful for unconstrained real-valued outputs. As proved in Section 1.4, a network of affine maps with only linear activations collapses to one affine map, irrespective of depth.

ReLU is $r(z) = \max(0, z)$, with

$$r'(z) = \begin{cases} 0 & z < 0, \\ 1 & z > 0. \end{cases}$$

It is continuous but not differentiable at zero; a common convention sets the derivative there to zero. It does not saturate on the positive side. A unit whose preactivation stays negative on all training examples receives zero gradient through the ReLU path for its incoming weights and bias. This is the "dead ReLU" mechanism; it does not mean every negative activation makes a neuron permanently dead, since other updates can alter its inputs. ReLU hidden activations are nonnegative, but a linear output with signed weights is unrestricted.

1.6.6 PReLU and ELU: what happens on the negative side?

Parametric ReLU uses a learned negative-side slope:

$$g_a(z) = \begin{cases} z & z \geq 0, \\ az & z < 0, \end{cases} \qquad \frac{\partial g_a}{\partial z} = \begin{cases} 1 & z > 0, \\ a & z < 0, \end{cases} \quad \frac{\partial g_a}{\partial a} = \begin{cases} 0 & z \geq 0, \\ z & z < 0. \end{cases}$$

Thus a learns from negative preactivations as well as affecting gradients flowing to earlier layers. A fixed slope is usually called leaky ReLU; learning it gives PReLU [6]. Parameters may be per unit/channel or shared. With positive a, both sides pass gradients; unconstrained learning does not mathematically force a to remain positive. At $a = 1$ the function is linear, and at $a = 0$ it is ReLU.

For $\alpha > 0$, the exponential linear unit is

$$\mathrm{ELU}_\alpha(z) = \begin{cases} z & z \geq 0, \\ \alpha(e^z - 1) & z < 0, \end{cases} \qquad g'(z) = \begin{cases} 1 & z > 0, \\ \alpha e^z & z < 0. \end{cases}$$

It permits negative outputs and tends to $-\alpha$ as $z \to -\infty$. Unlike a leaky straight line, its negative tail saturates and its derivative tends to zero. It is continuous at zero for all α, but the left and right derivatives agree there only when $\alpha = 1$. ELU was introduced by Clevert et al. [2]; these shape properties do not establish that it will outperform every other activation on a given task.

1.6.7 GELU and Swish: smooth input-dependent gates

Let Φ and ϕ be the standard normal CDF and density. The exact Gaussian Error Linear Unit is

$$\mathrm{GELU}(z) = z\Phi(z), \qquad \mathrm{GELU}'(z) = \Phi(z) + z\phi(z).$$

The derivative follows immediately from the product rule and $\Phi' = \phi$. For $U \sim \mathcal{N}(0, 1)$ and fixed z, $\mathbb{E}[z\mathbf{1}\{U \leq z\}] = z\Phi(z)$: this is an interpretation as the expected value of a random gate. Standard GELU evaluation is deterministic; it does not sample that gate. It is smooth, behaves

approximately like identity far to the right and tends to zero from below far to the left. It is not globally monotone [7].

A common approximation, which must be distinguished from exact GELU, is

$$\mathrm{GELU}(z) \approx \frac{z}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(z + 0.044715z^3)\right)\right].$$

Our plots use the exact CDF via the error function, not this approximation.

Swish is

$$\mathrm{Swish}_\beta(z) = z\sigma(\beta z), \qquad g'(z) = \sigma(\beta z) + \beta z\sigma(\beta z)(1 - \sigma(\beta z)).$$

It uses a logistic rather than a Gaussian gate. For $\beta = 1$ it is also commonly called SiLU. For positive β it has a small negative, nonmonotone portion; at $\beta = 0$ it equals $z/2$, and as $\beta \to +\infty$ it approaches ReLU pointwise. The original Swish study explores fixed or trainable β [9]. Neither smoothness nor a nonzero negative output alone guarantees well-conditioned optimization.

1.6.8 Output activations describe what predictions can mean

Hidden activations construct features. Output activations constrain the final prediction to an appropriate space. A loss then scores that prediction against an observation. A useful choice starts by specifying a conditional distribution for the observation:

Task	Output	Distribution model	Negative log-likelihood
Real regression	identity mean	fixed-variance Gaussian	scaled squared error
Binary class	sigmoid probability	Bernoulli	binary cross-entropy
One of K classes	softmax vector	categorical	categorical cross-entropy
Multiple independent labels	K sigmoids	product of Bernoullis	summed binary cross-entropies

These are natural pairings, not the only possible losses. Absolute error corresponds to a fixed-scale Laplace model, for example. Constraints such as positive means or variable uncertainty may call for other parameterizations. "Linear output" means identity activation on the final logit; the complete input-to-output function can still be highly nonlinear.

1.6.9 Derive squared error from a Gaussian observation model

Assume conditionally independent observations with

$$Y_n \mid x_n \sim \mathcal{N}(f_\theta(x_n), \sigma^2),$$

where $\sigma^2 > 0$ is fixed. The Gaussian density gives

$$-\log p(y_n \mid x_n; \theta) = \tfrac{1}{2}\log(2\pi\sigma^2) + \frac{(y_n - f_\theta(x_n))^2}{2\sigma^2}. \tag{1.38}$$

The first term is independent of θ. Multiplying the average likelihood objective by the positive constant σ^2 gives

$$\boxed{J(\theta) = \frac{1}{2N}\sum_n (f_\theta(x_n) - y_n)^2.}$$

This is half the mean squared error; the factor $1/2$ cancels the derivative's factor two. For identity output $\hat{y} = z$, $\partial\ell/\partial z = \hat{y} - y$. It is a continuous density model, so a density value is not the probability of observing exactly one real number. If variance is learned, the logarithmic variance term cannot be discarded.

Even without a Gaussian assumption, conditional squared error is minimized by the conditional mean when the second moment is finite:

$$\mathbb{E}[(a - Y)^2 \mid X = x] = (a - \mathbb{E}[Y \mid x])^2 + \mathrm{Var}(Y \mid x).$$

The second term cannot be changed by choosing a. This explains why overlapping one-feature Iris observations lead a squared-error model toward an average response, rather than requiring it to pass through every point.

1.6.10 Binary cross-entropy and why the output gradient simplifies

For $p = \sigma(z)$ and $y \in \{0, 1\}$, a Bernoulli model gives

$$\ell = -y \log p - (1 - y) \log(1 - p), \qquad \frac{\partial \ell}{\partial p} = \frac{p - y}{p(1 - p)}.$$

Multiplying by $\partial p / \partial z = p(1 - p)$ yields

$$\boxed{\frac{\partial \ell}{\partial z} = p - y.}$$

In contrast, half squared error on the same sigmoid probability has gradient $(p - y)p(1 - p)$. For a confident wrong prediction $y = 1$, $z \ll 0$, the BCE gradient approaches -1 while the squared-error gradient approaches zero. This illustrates why the likelihood-based pairing can give a more useful correction signal at the output. It does not remove vanishing gradients inside earlier saturated hidden layers.

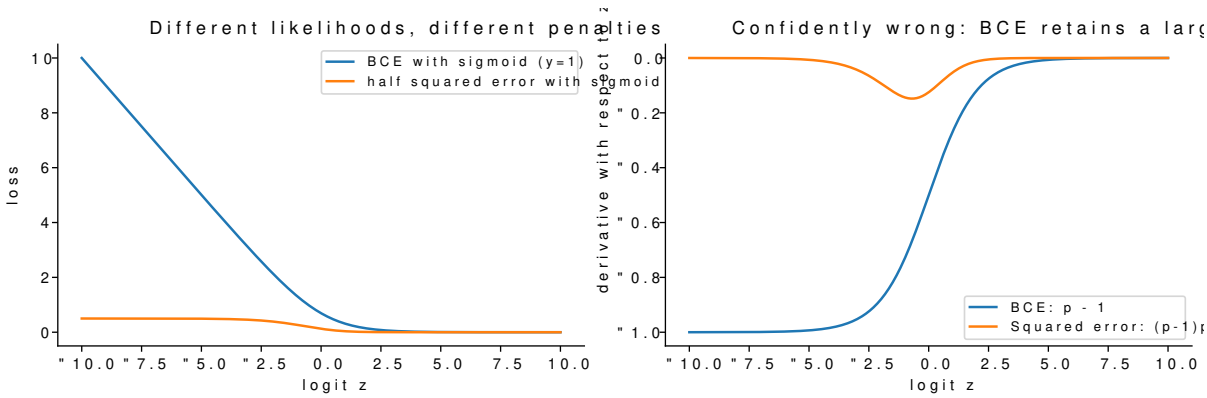


Figure 1.24: For $y = 1$, compare BCE with squared error applied to a sigmoid probability. These losses encode different objectives. Their derivatives with respect to the logit differ sharply for confident mistakes.

A stable binary loss uses logits directly:

$$\ell(z, y) = \log(1 + e^z) - yz = \log(1 + e^{(1-2y)z}) \quad (y \in \{0, 1\}).$$

Compute the latter as `np.logaddexp(0, (1-2*y)*z)`, avoiding explicit logarithms of rounded zero probabilities. For soft targets in $[0, 1]$, the first identity remains valid but the second generally does not.

1.6.11 Derive softmax from relative evidence and normalization

For K mutually exclusive and exhaustive classes, the output must satisfy $p_k > 0$ and $\sum_k p_k = 1$. Let z_k be unrestricted scores. If their differences are modeled as log-odds,

$$\log \frac{p_k}{p_j} = z_k - z_j,$$

then p_k is proportional to e^{z_k}. Normalization forces

$$\boxed{p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}.}$$

This is a useful modeling choice, not the unique possible map into the probability simplex. Adding the same constant to every score leaves all probabilities unchanged. Numerically subtract $m = \max_j z_j$ before exponentiating: $p_k = e^{z_k - m} / \sum_j e^{z_j - m}$. For two classes, $p_1 = \sigma(z_1 - z_0)$, connecting softmax back to sigmoid.

For one-hot targets, $y_k = 1$ only for the observed class. The categorical likelihood is $\prod_k p_k^{y_k}$, so

$$\ell(\mathbf{z}, \mathbf{y}) = -\sum_k y_k \log p_k = \log \sum_j e^{z_j} - \sum_k y_k z_k,$$

where $\sum_k y_k = 1$. Averaging over N examples yields the lecture's multiclass cross-entropy. The same expression can score a soft target distribution, interpreted as an average over possible labels.

By the quotient rule,

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j).$$

Thus changing one logit changes all class probabilities. Combining with $\partial \ell / \partial p_k = -y_k / p_k$ gives

$$\frac{\partial \ell}{\partial z_j} = -\sum_k y_k(\delta_{kj} - p_j) = p_j - y_j.$$

The output correction is again prediction minus target. This does not make an MLP's parameter-space loss convex: hidden nonlinear parameterizations remain.

1.6.12 Transfer question: why not a hard maximum?

Distinguish a maximum value $\max_k z_k$, a winning index $\arg\max_k z_k$, and a one-hot winner vector. A maximum value is not a probability distribution. A one-hot winner assigns exact zero probability to every losing class, so cross-entropy is infinite whenever the wrong class wins. Its output is locally constant away from ties, giving zero ordinary derivative; ties are nondifferentiable. It also discards whether the winner barely won or was overwhelmingly preferred.

Softmax supplies probabilities and a smooth learning signal for every logit. At prediction time, a hard decision is entirely appropriate when one class must be chosen:

$$\arg\max_k p_k = \arg\max_k z_k.$$

The hard decision is usually applied after training, outside the differentiable loss. A temperature $\tau > 0$ gives $p_k \propto e^{z_k/\tau}$; as $\tau \downarrow 0$, mass concentrates on the unique largest score. With ties it splits among the tied maxima. Softmax probabilities are model estimates, not automatically calibrated real-world frequencies.

1.6.13 Transfer question: what if regression has a ReLU output?

Then $\hat{y} = \max(0, z)$ forces predictions to be nonnegative. Negative targets cannot be fitted: for $y < 0$, the best allowable output under squared error is zero, with unavoidable half squared error $y^2/2$.

There is also an optimization effect. Away from zero,

$$\frac{\partial}{\partial z} \frac{1}{2}(\max(0, z) - y)^2 = (\max(0, z) - y)\mathbf{1}\{z > 0\}.$$

If $z < 0$, the gradient is zero even when $y > 0$ and the prediction is wrong. A single example cannot correct its final preactivation through this path. Other examples may update shared weights and move it into the active region, but this is not guaranteed. ReLU output can enforce a meaningful nonnegativity constraint, yet a softplus output $\log(1 + e^z)$ offers a smooth strictly positive alternative. Softplus still has a small derivative far left and cannot produce exact zero with finite z. Choose the output domain and likelihood deliberately.

1.6.14 Real weights on Iris: one feature and a continuous target

The following coefficients are actually fitted, not hand-drawn. We use the 150-row Iris measurements, exported from `sklearn.datasets.load_iris` into a committed CSV. Species IDs are 0=setosa, 1=versicolor, 2=virginica. The original dataset is documented by UCI [4]; the exact CSV and its SHA-256 identify the copy used here.

For a two-axis curve plot, use petal length x in centimetres to predict petal width y in centimetres. This repurposes the measurements for regression; it is not the dataset's usual species-classification task. A fixed seed 2026 splits each species into 40 training and 10 test examples, yielding 120/30 rows. Only training rows determine means and standard deviations. No test error is used to select architecture, training duration, or parameters.

Train a $1 \to 6 \to 1$ ReLU network with linear output using NumPy full-batch Adam for 5,000 steps, learning rate 0.01, seed 11, and a weight-only penalty $\lambda \sum W^2/2$ with $\lambda = 10^{-4}$. The objective is half MSE in standardized target units plus this penalty. Adam's coefficients are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Full code and exact values are saved; this is a reproducible illustration, not a search for a best Iris model.

The training normalization is

$$u = \frac{x - 3.776667}{1.769309}, \qquad t = \frac{y - 1.204167}{0.761839}.$$

If the trained standardized prediction is $\hat{t} = b_o + \sum_j v_j r(w_j u + b_j)$, undo the transformations to obtain

$$\hat{y} = c + \sum_{j=1}^{6} V_j r(A_j x + B_j),$$

with

$$A_j = \frac{w_j}{s_x}, \quad B_j = b_j - \frac{w_j \mu_x}{s_x}, \quad V_j = s_y v_j, \quad c = \mu_y + s_y b_o.$$

These are the fitted coefficients in the original measurement units:

Unit j	A_j	B_j	V_j	Knot $-B_j/A_j$
1	0.00000	-0.80000	0.00000	inactive
2	0.40437	-1.78047	0.52808	4.40312
3	0.41809	-1.84080	0.50510	4.40290
4	-0.59917	3.21392	-0.90287	5.36390
5	-0.60289	3.79894	0.99903	6.30119
6	-0.61062	3.29105	-0.71405	5.38966

Output bias in centimetres: $c = 1.163099$. Values in the table are rounded; the companion JSON preserves full precision. Unit 1 is inactive on the plotted interval, its input/output weights are approximately zero, and its nominal knot is far outside the data. Several other knots cluster: width six does not imply six useful, evenly spaced features. Real training can produce inactive or redundant units.

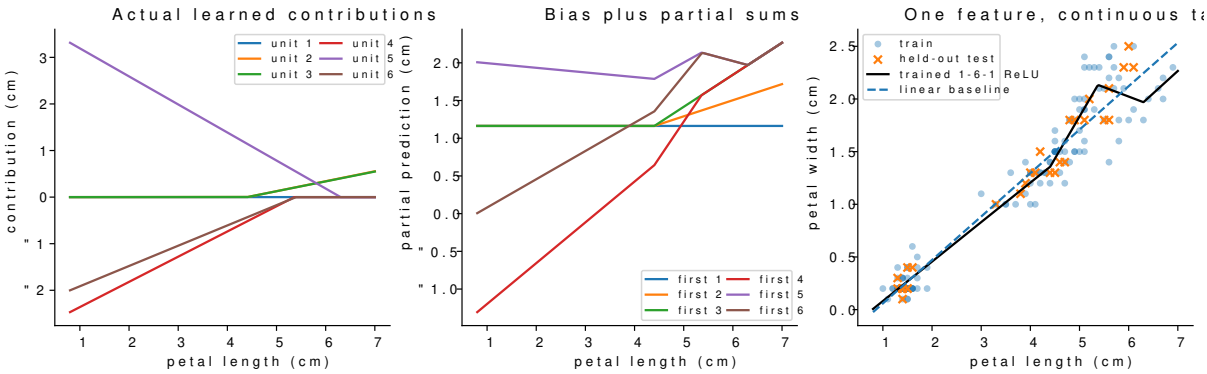


Figure 1.25: Actual fitted Iris weights. Left: the six weighted hidden curves. Middle: bias plus successive contributions; their order is illustrative, not a training sequence. Right: their final sum against training and held-out measurements, with a straight-line baseline. Each model is fitted only on training rows.

Model	Training RMSE (cm)	Test RMSE (cm)
ReLU MLP	0.18696	0.16337
Straight line (training least squares)	—	0.13947
Constant training-mean prediction	—	0.75112

The simpler straight line predicts better on this particular held-out split. We retain this outcome rather than adjusting the example until the MLP wins. A richer function family guarantees neither that optimization finds its best member nor that the selected function generalizes better. A single feature also cannot uniquely determine every petal width: species and other measurements contain information omitted here. Outside the training range, the ReLU network continues its end segments; a smooth-looking fit does not justify extrapolation.

1.6.15 Real weights on Iris: hidden features and softmax species prediction

For classification, use sepal length and petal width, keeping all three species and the same 120/30 split. Standardize with training statistics:

$$u_1 = \frac{x_{\text{sepal}} - 5.865000}{0.844951}, \qquad u_2 = \frac{x_{\text{petalwidth}} - 1.204167}{0.761839}.$$

Fit a $2 \to 6 \to 3$ tanh network with softmax output for 4,000 full-batch Adam steps, seed 19, learning rate 0.01, and weight penalty $\lambda = 0.001$. Its equations are

$$h_j = \tanh(w_{j1}u_1 + w_{j2}u_2 + b_j), \qquad z_k = c_k + \sum_{j=1}^{6} v_{kj}h_j, \qquad p_k = \text{softmax}(\mathbf{z})_k.$$

The actual learned coefficients are:

Unit	w_{j1}	w_{j2}	b_j	v_{0j}	v_{1j}	v_{2j}
1	1.06799	2.89666	-3.32035	-0.04266	-1.73543	1.77591
2	0.55057	-1.85691	1.61886	0.14065	1.44007	-1.57898
3	-0.26192	-1.58740	-1.18726	1.51544	-0.99753	-0.55675
4	-0.20741	-1.65256	-0.02121	0.88022	0.61456	-1.49454
5	0.27034	1.65272	1.24800	-1.56570	1.06228	0.50378
6	0.27040	1.65264	1.24926	-1.56154	1.06322	0.49651

Output biases (class order 0, 1, 2): $(0.83680, -2.25238, 1.63661)$.

For held-out CSV row 28 (zero-based), the selected inputs are $(5.2, 0.2)$ cm. The hidden activations are approximately

$$\mathbf{h} = (-1.0000,\ 0.9986,\ 0.8045,\ 0.9809,\ -0.8155,\ -0.8150),$$

and the logits and probabilities are

$$\mathbf{z} = (5.65199, -1.01143, -4.44544), \qquad \mathbf{p} = (0.998684, 0.001275, 0.000041).$$

Its largest probability is for setosa. For example, the setosa score is a signed weighted sum of the six features plus $c_0 = 0.83680$; it is not a sum of six probabilities.

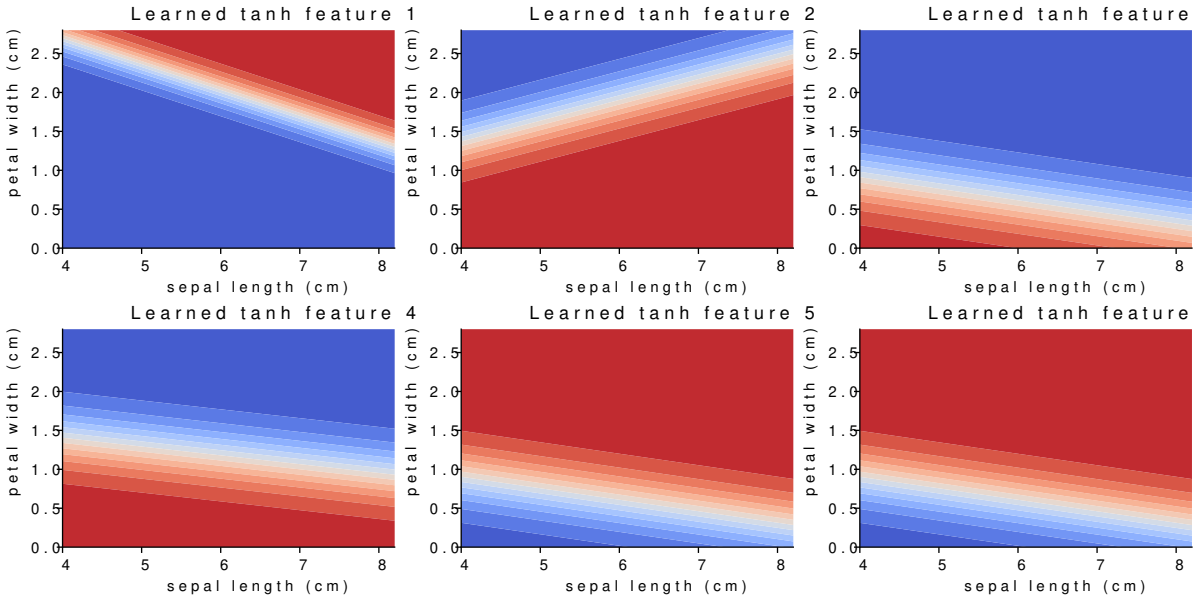


Figure 1.26: The six tanh features from the trained classifier, each plotted over the same two raw measurements. Red denotes values near +1 and blue values near −1. These are internal features, not species probabilities; similar-looking units can be redundant.

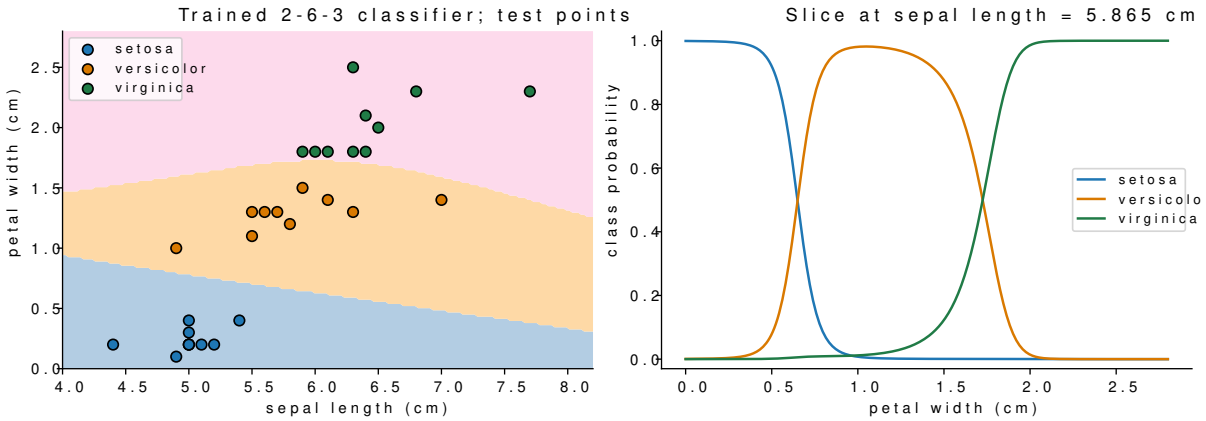


Figure 1.27: Left: predicted class regions and the 30 test observations. Right: class probability curves as petal width varies with sepal length fixed at its training mean. This is a one-dimensional slice of a two-feature classifier, not a different one-feature model.

Training accuracy is 115/120 = 95.83%. Test accuracy is 30/30 = 100%, with average test cross-entropy 0.06792 nats. There are ten test examples per species and the confusion matrix

is diagonal. The perfect test classification is a result on one small split, not a claim of perfect population performance or calibrated probabilities. That test accuracy exceeds training accuracy here is possible because the held-out sample happens to be easier.

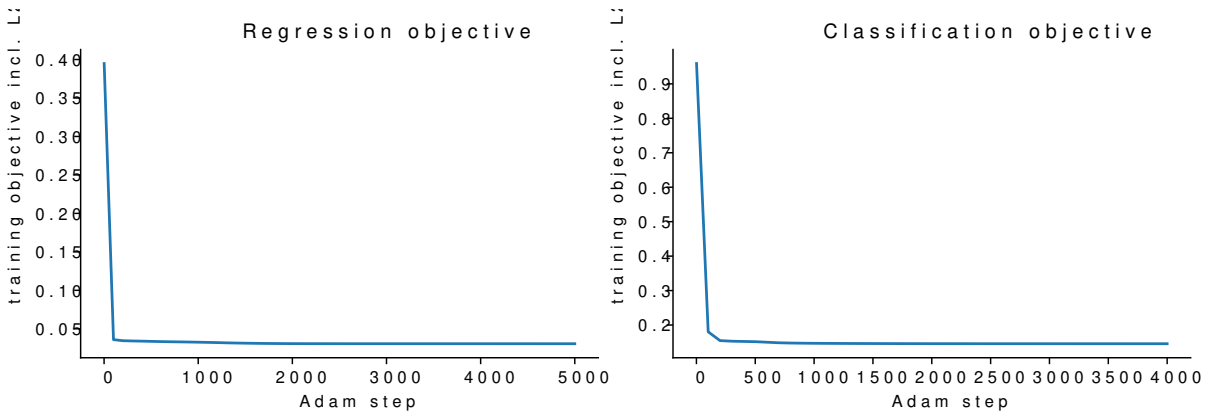


Figure 1.28: Training objectives for the two pre-specified Iris models, including their L2 penalties. Training curves show optimization progress, not generalization performance; the test set was not used for stopping.

1.6.16 Implementation and reconstruction tasks

The companion `activations_1_6_1_7.py` reads the committed CSV and uses NumPy for forward passes, gradients, and Adam. The activation plots use `math.erf` for the exact normal CDF. Finite-difference checks agree with both network gradients to better than 3×10^{-10} in maximum absolute error on the check examples; all eight activation derivatives are checked away from kinks. These numerical checks support the implementation, not a proof that training reaches a global optimum.

```
# One hidden layer, row-batch notation:
z_hidden = X @ W1.T + b1
h = np.maximum(z_hidden, 0.0)  # use np.tanh for the classifier
logits = h @ W2.T + b2

# Stable softmax, if outputs encode mutually exclusive classes:
a = logits - logits.max(axis=1, keepdims=True)
p = np.exp(a)
p /= p.sum(axis=1, keepdims=True)

# Classification backpropagation at the output:
d_logits = p.copy()
d_logits[np.arange(len(y)), y] -= 1
d_logits /= len(y)
```

For the regression network, the analogous output derivative is $(\hat{t} - t)/N$. Multiplication by the output weights and the hidden activation derivative propagates that signal to the first layer; the code includes the weight-penalty derivatives.

To check understanding without rereading, reconstruct the hinge curve from its desired slopes, calculate one Iris prediction from the saved coefficients, and explain why softmax changes every class probability when only one logit changes. Then deliberately remove the hidden nonlinearity: the two weight matrices must collapse into one affine map. These tasks test whether the equations are usable, rather than only recognizable.

1.7 Representational Power of MLPs

1.7.1 Separate three questions that are often mixed together

Representation asks whether some parameters can express or approximate a target function. *Optimization* asks whether a training procedure finds suitable parameters. *Generalization* asks how the resulting function performs on unseen data. A theorem about the first question does not automatically answer the other two.

A sigmoid or threshold perceptron without hidden features uses one affine score. Its binary decision boundary is a hyperplane in the given feature space. XOR fails the inequalities derived in Section 1.4, regardless of how long the perceptron is trained. An identity-output model without hidden layers is affine regression. A softmax linear classifier has pairwise boundaries $z_i = z_j$, hence affine boundaries; the same distinction between logits and probabilities applies.

The limitation depends on the representation supplied to the model. A linear classifier on manually constructed nonlinear features can have a nonlinear boundary in the raw inputs. Thus "only linearly separable functions" means separability in the model's input feature coordinates, not in every possible re-encoding.

1.7.2 Any Boolean function on a finite cube: an explicit construction

Let $f : \{0,1\}^d \to \{0,1\}$ be an arbitrary truth table. For each binary vector v whose label is one, define the number of matching bits

$$s_v(x) = \sum_{i:v_i=1} x_i + \sum_{i:v_i=0} (1 - x_i).$$

This is affine in x. On binary inputs it equals d if $x = v$ and is at most $d-1$ otherwise. Therefore one ReLU unit

$$h_v(x) = \max(0, s_v(x) - d + 1)$$

outputs exactly one at v and zero at all other Boolean inputs. Set

$$F(x) = \sum_{v:f(v)=1} h_v(x).$$

At most one such unit is active on each binary input, so $F(x) = f(x)$ there. This is a one-hidden-layer ReLU network with a linear output and at most 2^d hidden units. The all-zero truth table uses a zero output.

For XOR, this construction gives $h_{10}(x) = r(x_1 - x_2)$ and $h_{01}(x) = r(x_2 - x_1)$, exactly the two features used earlier. The construction is a finite-domain argument, not a claim that a continuous ReLU network equals a discontinuous Boolean indicator everywhere in $\mathbb{R}^d$. With a finite sigmoid output, probabilities cannot equal zero and one exactly, though thresholded labels can be exact and probabilities can approach those endpoints.

This proves existence by constructing weights from the entire truth table. It is not a learning procedure that discovers an unseen truth table from a small subset of its rows. The worst-case width is exponential in d, so even existence may be computationally impractical.

1.7.3 Universal approximation: a precise statement

A classical form of the result of Cybenko [3] is: for a continuous sigmoidal activation σ, every continuous $f : [0,1]^d \to \mathbb{R}$ and every $\varepsilon > 0$ admit a finite width m and real parameters such that

$$F(x) = c + \sum_{j=1}^{m} v_j \sigma(a_j^\top x + b_j), \qquad \sup_{x \in [0,1]^d} |F(x) - f(x)| < \varepsilon. \tag{1.39}$$

The output is a linear combination, not restricted to $[0, 1]$. Other versions cover other suitable activations and domains. ReLU also has universal approximation results; the constructive one-dimensional proof below needs no appeal to the sigmoid theorem.

Read the quantifiers in order: choose a target and an error tolerance; then a sufficiently large *finite* network exists. The required width and weight magnitudes can depend on both. A single fixed-width architecture is not promised arbitrary precision. No infinite network is required for a fixed positive tolerance, although a sequence of increasingly accurate approximations may require unbounded width.

Compactness matters. A continuous function on a compact interval or cube is bounded and uniformly continuous. Merely saying "any bounded continuous function" without specifying the approximation domain and norm is too broad. Uniform approximation on an entire noncompact space is a different assertion. Also, a uniform limit of continuous functions is continuous, so continuous networks cannot uniformly approximate a jump discontinuity to arbitrarily small error across the jump. Weaker integral-error notions can permit approximating discontinuous targets.

1.7.4 A first-principles proof in one dimension using ReLU hinges

Let f be continuous on $[a, b]$. Given $\varepsilon > 0$, uniform continuity supplies $\delta > 0$ such that $|f(x) - f(y)| < \varepsilon$ whenever $|x - y| < \delta$. Choose a partition $a = t_0 < \cdots < t_m = b$ with all intervals shorter than δ.

On $[t_i, t_{i+1}]$, let $\lambda = (x - t_i)/(t_{i+1} - t_i)$ and define the linear interpolant

$$p(x) = (1 - \lambda)f(t_i) + \lambda f(t_{i+1}), \qquad 0 \le \lambda \le 1.$$

Then

$$\begin{aligned} |p(x) - f(x)| &\le (1 - \lambda)|f(t_i) - f(x)| + \lambda|f(t_{i+1}) - f(x)| \\ &< (1 - \lambda)\varepsilon + \lambda\varepsilon = \varepsilon. \end{aligned}$$

At knots the error is zero. The interpolant is continuous and piecewise linear, so the hinge formula (1.37) represents it exactly. Two additional ReLUs can encode its initial affine slope if needed. Thus a finite one-hidden-layer ReLU network uniformly approximates f within ε on this interval.

This is a complete proof for the stated one-dimensional case. It does not constitute a proof of Cybenko's multidimensional theorem. The sigmoid-bump visualization provides another intuition, while the general result requires additional approximation arguments. "Similar to a Taylor expansion" is only a loose analogy: a Taylor series uses derivatives and powers around a point, may fail to converge to a smooth function, and does not explain approximation of every continuous function. Learned affine-transformed activations are a different function family.

For a twice differentiable f with $|f''| \le M$, linear interpolation on an interval of length Δ satisfies

$$|f(x) - p(x)| \le \frac{M\Delta^2}{8}.$$

The interpolation remainder is $f''(\xi)(x - t_i)(x - t_{i+1})/2$, whose absolute product is at most $\Delta^2/4$. For $f(x) = x^2$ on $[0, 1]$, $M = 2$ and uniform spacing $\Delta = 1/m$ gives maximum error $1/(4m^2)$, attained at midpoints. This quantitatively explains the refinement figure in Section 1.6; width buys finer slope changes.

1.7.5 What universal approximation does not say

It does not specify a small sufficient width for every target, a practical sample size, a convergent optimizer, or good behavior outside the chosen domain. It does not remove noise, ambiguity in the inputs, or finite-precision limitations. It does not say that every parameter setting of a large network is useful.

The real Iris regression example is a concrete reminder. A nonlinear model can represent more functions, but its fixed training run had larger test RMSE than a straight line. A theorem that some wide network exists neither selects a good model for these data nor ranks the particular trained models. Fitting a finite training set and recovering its unknown data-generating rule are different achievements.

1.7.6 Depth: repeated composition can reuse a small computation

Consider the tent-shaped map on $[0, 1]$,

$$T(x) = 2r(x) - 4r(x - \tfrac{1}{2}) + 2r(x - 1) = \begin{cases} 2x & 0 \le x \le \frac{1}{2}, \\ 2 - 2x & \frac{1}{2} \le x \le 1. \end{cases}$$

It maps both half-intervals onto the whole interval $[0, 1]$, one in forward order and one in reverse order. Applying T again performs the same fold on each half. Consequently $T \circ T$ has four affine pieces, and $T^{\circ L}$ has 2^L affine pieces. Their slopes have magnitude 2^L, alternating signs.

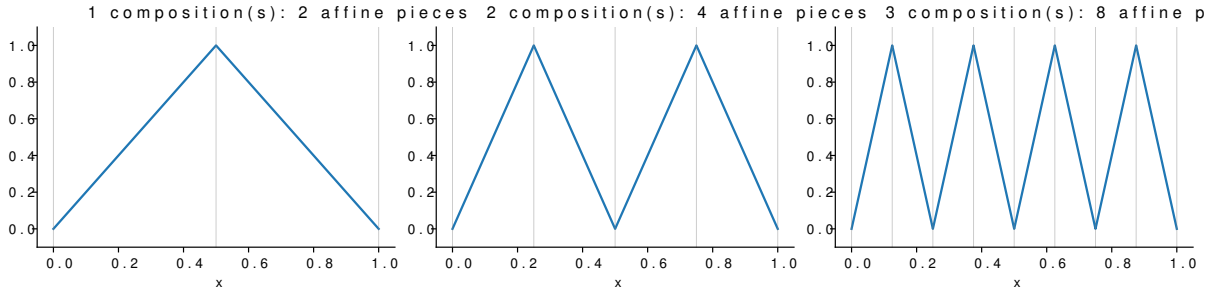


Figure 1.29: One small tent computation reused through composition. Each additional composition splits every current affine interval into two. These continuous triangular waves are exact ReLU constructions; a square-wave cartoon should not be read as an exact discontinuous ReLU output.

To see this is an actual deep network, write $v = (2, -4, 2)^\mathsf{T}$ and $b = (0, -1/2, -1)^\mathsf{T}$. Let

$$h^{(1)} = r(\mathbf{1}_3 x + b), \qquad h^{(\ell)} = r(\mathbf{1}_3 v^\mathsf{T} h^{(\ell-1)} + b), \qquad F(x) = v^\mathsf{T} h^{(L)}.$$

Every hidden layer has width three, and $F = T^{\circ L}$. The linear combination implementing one tent is absorbed into the next layer's affine map. The number of parameters is $O(L)$, while the number of affine pieces is exponential in L.

By contrast, a one-hidden-layer, one-input ReLU network with m hidden units has at most m distinct knots and therefore at most $m + 1$ affine pieces. Each unit contributes at most one knot, and their sum cannot change slope elsewhere. To represent this same tent-composition function *exactly*, it needs at least $2^L - 1$ hidden units. This is a constructive representation-efficiency separation for a particular family, not a claim that every function benefits exponentially from depth or that approximation errors automatically obey the same exact-representation bound.

1.7.7 Interpreting the linear-region theorem in the slide

A linear region is a maximal connected input region where a piecewise-affine network agrees with one affine map. It is not a class label, an independent training example, or a complete measure of learning capacity.

For input dimension n_0, L ReLU hidden layers, and widths $n_i \geq n_0$, Montufar et al. [8] give the constructive lower bound

$$R_{\max} \geq \left(\prod_{i=1}^{L-1} \left\lfloor \frac{n_i}{n_0} \right\rfloor^{n_0} \right) \sum_{j=0}^{n_0} \binom{n_L}{j}.$$

For equal width n divisible by fixed n_0 and $n > n_0$, this displays exponential growth in L. The slide summarizes the growth as

$$\Omega\Big((n/n_0)^{(L-1)n_0} n^{n_0} \Big).$$

The floored expression states the explicit construction; the shorthand should be interpreted with its asymptotic regime. Ω is a lower bound on achievable maximal region count, not an equality for every network. Width and input dimension assumptions matter; for example the displayed base gives no exponential factor at $n = n_0$. The output map must preserve distinctions to retain all these regions.

Depth enables a later computation to act on many portions of the original input after earlier layers map them to common coordinates. Regions are not independent arbitrary pieces: they share parameters through composition. A network with all-zero weights remains constant regardless of depth. More potential regions therefore do not prove better accuracy, successful optimization, or immunity to overfitting.

1.7.8 Capacity comparison and answers to the lecture questions

Architecture	What it can express	What remains unguaranteed
No hidden layer	Affine regression; a sigmoid of an affine score; affine pairwise softmax boundaries	XOR in raw binary coordinates; arbitrary nonlinear dependence
One suitable nonlinear hidden layer	Arbitrary Boolean tables on their finite domain; arbitrary-precision continuous approximation on compact domains when width can grow	Small width, feasible training, accurate unseen predictions
Many nonlinear hidden layers	Compositions and reuse of learned features; efficient representations of some functions requiring many shallow units	An exponential gain for every target, every weight setting, or every data distribution

The universal approximation theorem states an existence result with an error tolerance, a domain, and a suitable activation. It does not state a learning or generalization guarantee. A shallow universal family and a deep efficient family are not contradictory: both may approximate a function, while requiring very different resources.

Reconstruct the argument yourself. For a one-input ReLU network, count possible knots before counting parameters. For a tent composition, count how many old intervals are mapped across the next fold. For a Boolean lookup network, count matching bits. These three different counting arguments explain shallow shape flexibility, compositional efficiency, and finite-domain universality without relying on a slogan.

1.8 Summary, Further Reading, References

1.8.1 Reconstruct the chapter as one connected argument

Start with a binary outcome and an affine score. Modeling its conditional log-odds as affine leads to a sigmoid probability. A Bernoulli likelihood then gives binary cross-entropy. Averaging negative log probabilities under a source gives cross-entropy; subtracting source entropy gives forward KL, when the expressions are well-defined. Replacing an unknown expectation by the sample average leads to maximum likelihood.

Drawing the same sigmoid computation as a neuron changes its organization, not its mathematics. Hidden nonlinear layers then learn the features on which the final prediction depends. Matrix multiplication collects their scalar dot products; dimensions track examples, features, and units. Operation counts and stored array sizes follow from these dimensions, with activation lifetime determining memory beyond parameters.

Activation functions shape both representations and gradient flow. Output activations instead encode the prediction's allowed domain: a real mean, a binary probability, or a probability vector. A likelihood supplies a corresponding loss. One hidden layer with a linear output is a weighted sum of transformed activations, so neurons really can combine to build a curve. Depth adds composition and reuse. Existence and representation efficiency remain separate from optimization and generalization.

1.8.2 Learning goals with precise qualifications

- For a hard observed class label, cross-entropy is the negative log of the *model-assigned probability of that label*. It is not a log of classification accuracy.
- Logistic regression is a sigmoid neuron; the classical hard-threshold perceptron has a different activation and learning rule.
- Conditional cross-entropy equals conditional source entropy plus average conditional forward KL. Empirical likelihood estimates the population objective; continuous empirical point masses require care when writing KL to a density.
- MLPs alternate affine maps and nonlinear activations. Stacking only affine maps yields an affine map.
- Biases are shared across batch examples. Broadcasting describes repeated use, not repeated trainable parameters.
- Time counts arithmetic; memory counts stored objects and their lifetimes. Parameters, input/output, and any retained activations must be included according to the question's convention.
- Sigmoid, tanh, identity, ReLU, PReLU, ELU, GELU, and Swish differ in output ranges, derivatives, saturation, and smoothness. No one shape wins on every task.
- Identity/Gaussian, sigmoid/Bernoulli, and softmax/categorical pairings explain squared error and cross-entropy. Tanh must be mapped to a valid probability before a Bernoulli log loss.
- One-hidden-layer universality allows width to grow and specifies a compact domain and approximation error. Boolean lookup constructions can require exponentially many units.
- Depth can make some representations exponentially more efficient. The linear-region result concerns achievable region counts under assumptions, not guaranteed accuracy gains.

1.8.3 Practice without looking, then check the reasoning

1. Curve reconstruction. A continuous function has slopes $2, -1, 3$ across knots 1 and 2 and satisfies $f(0) = 0$. Write a ReLU representation on $[0, 3]$.

2. Output diagnosis. A regressor has $z = -2$, ReLU output, and target $y = 3$. Find its output, half squared error, and derivative with respect to z. Would a linear output have the same derivative?

3. Relative evidence. Softmax logits are $(1, 2, 3)$. What changes if 100 is added to all logits? What if only the first logit is increased? Derive the answer using normalization.

4. Exact versus approximate. Explain why a finite sigmoid output cannot equal one exactly, yet can classify every row of a finite Boolean truth table correctly. What would a $1/2$ threshold do?

5. Transfer to a new dataset. Choose one input and one real target, split data before normalizing, fit a small MLP and a straight line, and plot the individual weighted hidden curves. Report test error even if the simpler model wins. State whether you trained hidden parameters or only output coefficients.

1.8.4 Worked checks

1. Slopes. $f(x) = 2r(x) - 3r(x - 1) + 4r(x - 2)$ on $[0, 3]$. The changes of slope are -3 and $+4$, not the later slopes themselves. On a general real domain replace the initial $2r(x)$ by $2r(x) - 2r(-x)$ if the initial segment should extend with slope two to negative inputs.

2. A blocked output gradient. The ReLU output is zero, the loss is $9/2$, and its derivative with respect to z is zero. For a linear output $\hat{y} = -2$, the loss is $25/2$ and the derivative is -5. Having a smaller current loss does not imply having a useful gradient for reaching a good solution.

3. Softmax. A shared shift multiplies numerator and denominator by the same factor and cancels. Increasing only the first score raises its probability and lowers the others through the common denominator. For $j \neq 1$, $\partial p_j / \partial z_1 = -p_j p_1$.

4. Classification versus endpoint probabilities. Finite sigmoid outputs lie strictly between zero and one. A finite dataset can still have every positive example above $1/2$ and every negative one below it, so thresholded predictions can be exact without exact endpoint probabilities.

1.8.5 Reproducible materials and further reading

The script `deep_learning/code/activations_1_6_1_7.py` regenerates nine figures and fits both Iris models using NumPy. The committed `deep_learning/data/iris.csv` makes reproduction independent of a dataset download. Its source is the scikit-learn loader for the classic UCI Iris data. The companion `trained_models.json` includes full-precision weights, normalization statistics, zero-based row splits, optimizer settings, histories, test results, and the dataset checksum. Tables are generated from those weights, not manually invented values.

For a connected treatment, consult Goodfellow et al. [5], especially Chapter 6, and Bishop [1]. For precise theorem statements, see Cybenko [3] on sigmoid approximation and Montufar et al. [8] on ReLU linear regions. Original activation papers are He et al. [6], Clevert et al. [2],

Hendrycks and Gimpel [7], and Ramachandran et al. [9]. The explicit hinge, Boolean-match, and tent constructions in these notes make the relevant ideas inspectable without treating a citation as a substitute for reasoning.

Chapter 2

MLPs and Backpropagation

2.1 Recap of Multilayer Perceptrons

2.2 Recap of Chain Rule of Calculus

2.3 Calculating Gradients with Backpropagation

2.4 Backprop for a Linear Layer in Matrix Form

2.5 Full Matrix Form of Backpropagation Equations

2.6 Summary, Further Reading, References

Bibliography

[1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[2] Djork-Arne Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units. *arXiv:1511.07289*, 2015. URL https://arxiv.org/abs/1511.07289.

[3] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989. doi: 10.1007/BF02551274.

[4] R. A. Fisher. Iris. UCI Machine Learning Repository, 1936. URL https://archive.ics.uci.edu/dataset/53/iris.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015. URL https://arxiv.org/abs/1502.01852.

[7] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv:1606.08415*, 2016. URL https://arxiv.org/abs/1606.08415.

[8] Guido Montufar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems*, 2014. URL https://arxiv.org/abs/1402.1869.

[9] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv:1710.05941*, 2017. URL https://arxiv.org/abs/1710.05941.